STUDY GUIDE

# Solutions Manual

Reference solutions and quiz answers for every lesson

Each solution is the exact Java file that was compiled and tested, with an explanation.

Generated 10 October 2026 from the same lesson data that powers the website.
Every Java program in this guide was compiled with --release 17 and run against its expected output.

# Part 0 · Programming & Java Setup

## Exercise solutions

### 0.1 How Java Works: Source, Bytecode, JVM, JRE, JDK

**Exercise 1 · Print your own message**

Ex1.java

```java
public class Ex1 {
 public static void main(String[] args) {
 System.out.println("Write once, run anywhere");
 }
}
```

**Why it works:** A class with a main method that prints the text.

Verified against 1 test case: `(no input)` → `Write once, run anywhere`

### 0.2 The Java Landscape: History, Features, Editions, JVM Memory

**Exercise 1 · Powers of two in memory units**

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long megabytes = sc.nextLong();
 long bytes = megabytes * 1024 * 1024; // megabytes is already a long, so no int overflow
 System.out.println(bytes);
 }
}
```

**Why it works:** Cast to long before multiplying to avoid int overflow.

Verified against 3 test cases: `256⏎` → `268435456`; `1⏎` → `1048576`; `4096⏎` → `4294967296`

### 0.3 Installing Java on macOS and Choosing an Editor

**Exercise 1 · Compile and run it yourself**

Ex1.java

```java
public class Ex1 {
 public static void main(String[] args) {
 System.out.println("JDK ready");
 System.out.println("Let us begin");
 }
}
```

**Why it works:** Two print statements, each followed by a newline.

Verified against 1 test case: `(no input)` → `JDK ready⏎Let us begin`

### 0.4 Your First Program: class, main, statements, comments

**Exercise 1 · Print a box**

**Ex1.java**

```java
public class Ex1 {
 public static void main(String[] args) {
 System.out.println("+--+");
 System.out.println("|Hi|");
 System.out.println("+--+");
 }
}
```

**Why it works:** Each println prints one line.

Verified against 1 test case: `(nc input)` → `+--+⏎|Hi|⏎+--+`

### Exercise 2 · Echo a name

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String name = sc.nextLine();
 System.out.println("Hello, " + name + "!");
 }
}
```

**Why it works:** Read one line, then concatenate.

Verified against 2 test cases: `Asha⏎` → `Hello, Asha!`; `Java Learner⏎` → `Hello, Java Learner!`

### Exercise 3 · Sum two numbers

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int a = sc.nextInt();
 int b = sc.nextInt();
 System.out.println(a + b);
 }
}
```

**Why it works:** Scanner.nextInt() reads whitespace-separated integers.

Verified against 3 test cases: `7 5⏎` → `12`; `-3 10⏎` → `7`; `0 0⏎` → `0`

## 0.5 Compile-time, Runtime and Logic Errors

### Exercise 1 · Correct average

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int a = sc.nextInt();
 int b = sc.nextInt();
 double average = (a + b) / 2.0; // 2.0 forces floating-point division
 System.out.printf("%.1f%n", average);
 }
}
```

**Why it works:** Dividing by the double 2.0 forces floating-point division; printf formats one decimal.

Verified against 3 test cases: `2 3⏎` → `2.5`; `10 20⏎` → `15.0`; `-1 0⏎` → `-0.5`

### Exercise 2 · Safe division

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int a = sc.nextInt();
 int b = sc.nextInt();
 if (b == 0) {
 System.out.println("cannot divide by zero");
 } else {
 System.out.println(a / b);
 }
 }
}
```

**Why it works:** A guard condition prevents the runtime error.

Verified against 3 test cases: `7 2⏎` → `3`; `5 0⏎` → `cannot divide by zero`; `-7 2⏎` → `-3`

## 0.6 What Is an Algorithm?

### Exercise 1 · Find the smallest

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int smallest = sc.nextInt();
 for (int i = 1; i < n; i++) {
 int x = sc.nextInt();
 if (x < smallest) smallest = x;
 }
 System.out.println(smallest);
 }
}
```

**Why it works:** Same scan, opposite comparison.

Verified against 3 test cases: `4⏎5 2 8 1⏎` → `1`; `1⏎7⏎` → `7`; `3⏎-5 -9 -1⏎` → `-9`

### Exercise 2 · Sum of the numbers

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int sum = 0;
 for (int i = 0; i < n; i++) sum += sc.nextInt();
 System.out.println(sum);
 }
}
```

**Why it works:** Add each number to a running total.

Verified against 3 test cases: `4⏎5 2 8 1⏎` → `16`; `2⏎-3 3⏎` → `0`; `1⏎100⏎` → `100`

### Exercise 3 · Is it sorted?

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Is the sequence sorted in non-decreasing order?
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int prev = sc.nextInt();
 boolean sorted = true;
 for (int i = 1; i < n; i++) {
 int x = sc.nextInt();
 if (x < prev) sorted = false;
 prev = x;
 }
 System.out.println(sorted ? "YES" : "NO");
 }
}
```

**Why it works:** One pass remembering the previous element; any decrease means NO.

Verified against 3 test cases: `4⏎1 2 2 3⏎` → `YES`; `3⏎3 2 1⏎` → `NO`; `1⏎5⏎` → `YES`

# Quiz answer key

## 0.1 How Java Works: Source, Bytecode, JVM, JRE, JDK

| | Answer | Why |
|---|---|---|
| Q1 | B. bytecode (.class files) | javac compiles .java source into .class bytecode. |
| Q2 | C. JDK | The JDK contains the compiler and tools. |
| Q3 | B. It is bytecode executed by a platform-specific JVM | The JVM hides the platform differences. |
| Q4 | B. compiling frequently used bytecode to machine code at run time | Hot code is compiled to native code just in time to speed it up. |

## 0.2 The Java Landscape: History, Features, Editions, JVM Memory

| | Answer | Why |
|---|---|---|
| Q1 | B. heap | Objects live on the heap; references to them may live on the stack. |
| Q2 | B. frees objects that are no longer reachable | Automatic memory management. |
| Q3 | C. Java SE | Java SE (Standard Edition). |
| Q4 | B. StackOverflowError | The call stack is exhausted. |

## 0.3 Installing Java on macOS and Choosing an Editor

| | Answer | Why |
|---|---|---|
| Q1 | B. javac Hello.java | javac is the compiler. |
| Q2 | A. java Hello | Use the class name, without extension. |

| | Answer | Why |
|---|---|---|
| Q3 | B. no JDK (compiler) is installed or on the PATH | The compiler is missing from the PATH. |
| Q4 | C. Java 17 | Java 17 (LTS); newer JDKs compile with --release 17. |

## 0.4 Your First Program: class, main, statements, comments

| | Answer | Why |
|---|---|---|
| Q1 | B. the main method | The JVM calls public static void main(String[] args). |
| Q2 | line 3 | A semicolon is missing at the end of the statement. |
| Q3 | class \| void | A class declaration and a void return type. |
| Q4 | ABC D | print does not add a newline; println does. |
| Q5 | B. Counter.java | A public class must be in a file with exactly its name. |

## 0.5 Compile-time, Runtime and Logic Errors

| | Answer | Why |
|---|---|---|
| Q1 | B. a compile-time error | A type mismatch is found by the compiler. |
| Q2 | C. logic error | It runs fine but is wrong. |
| Q3 | B. dividing an int by zero | ArithmeticException happens while running. |
| Q4 | 3 | Integer division truncates the .5. |
| Q5 | B. line number | Line 3 of Hello.java. |

## 0.6 What Is an Algorithm?

| | Answer | Why |
|---|---|---|
| Q1 | B. A set of instructions to solve a problem | The slide's own MCQ: a set of well-defined instructions that solves a problem. |
| Q2 | C. written in Java | It can be expressed in any notation. |
| Q3 | B. impractical: efficiency matters too | Correctness and efficiency are both needed. |
| Q4 | B. n − 1 comparisons | Each of the other n − 1 numbers must be compared at least once. |

# Part 1 · Java Syntax & Fundamentals

## Exercise solutions

### 1.1 Variables, Constants and Primitive Types

#### Exercise 1 · Swap two numbers

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int a = sc.nextInt();
 int b = sc.nextInt();
 int temp = a; // temp remembers a before it is overwritten
 a = b;
 b = temp;
 System.out.println(a + " " + b);
 }
}
```

**Why it works:** temp keeps the first value so it is not lost when a is overwritten.

Verified against 3 test cases: `3 8⏎` → `8 3`; `-1 0⏎` → `0 -1`; `5 5⏎` → `5 5`

#### Exercise 2 · Rectangle

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 double w = sc.nextDouble();
 double h = sc.nextDouble();
 System.out.println("area=" + w * h);
 System.out.println("perimeter=" + 2 * (w + h));
 }
}
```

**Why it works:** Plain arithmetic on doubles; string concatenation prints the value.

Verified against 2 test cases: `2.5 4⏎` → `area=10.0⏎perimeter=13.0`; `1 1⏎` → `area=1.0⏎perimeter=4.0`

#### Exercise 3 · Seconds to h:m:s

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int total = sc.nextInt();
 int hours = total / 3600;
 int minutes = (total % 3600) / 60;
 int seconds = total % 60;
 System.out.println(hours + ":" + minutes + ":" + seconds);
 }
}
```

**Why it works:** Integer division and remainder peel off the units.

Verified against 3 test cases: `3725⏎` → `1:2:5`; `59⏎` → `0:0:59`; `86399⏎` → `23:59:59`

## 1.2 Type Conversion, Casting and Overflow

### Exercise 1 · Big product

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int a = sc.nextInt();
 int b = sc.nextInt();
 long product = (long) a * b; // widen first, then multiply
 System.out.println(product);
 }
}
```

**Why it works:** (long) a promotes the multiplication to long arithmetic.

Verified against 3 test cases: `100000 100000⏎` → `10000000000`; `2147483647 2147483647⏎` → `4611686014132420609`; `-3 4⏎` → `-12`

### Exercise 2 · Shift a letter

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Shift an uppercase letter forward by k positions, wrapping Z -> A.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 char c = sc.next().charAt(0);
 int k = sc.nextInt();
 char shifted = (char) ('A' + (c - 'A' + k) % 26);
 System.out.println(shifted);
 }
}
```

**Why it works:** Convert to 0..25, add k, wrap with %, convert back.

Verified against 3 test cases: `Y 3⏎` → `B`; `A 0⏎` → `A`; `Z 26⏎` → `Z`

### Exercise 3 · Will it overflow?

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Print OK if a + b fits in an int, otherwise OVERFLOW.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long a = sc.nextLong();
 long b = sc.nextLong();
 long sum = a + b;
 boolean fits = sum >= Integer.MIN_VALUE && sum <= Integer.MAX_VALUE;
 System.out.println(fits ? "OK" : "OVERFLOW");
 }
}
```

**Why it works:** A long sum cannot overflow for int-sized inputs; compare it with the int bounds.

Verified against 3 test cases: `2000000000 2000000000⏎` → `OVERFLOW`; `1 2⏎` → `OK`; `-2147483648 -1⏎` → `OVERFLOW`

## 1.3 Arithmetic, Assignment and Increment Operators

### Exercise 1 · Digit sum

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int sum = 0;
 while (n > 0) {
 sum += n % 10;
 n /= 10;
 }
 System.out.println(sum);
 }
}
```

**Why it works:** Repeatedly take the last digit and drop it.

Verified against 3 test cases: `4821⏎` → `15`; `7⏎` → `7`; `1000⏎` → `1`

### Exercise 2 · Reverse the digits

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int reversed = 0;
 while (n > 0) {
 reversed = reversed * 10 + n % 10;
 n /= 10;
 }
 System.out.println(reversed);
 }
}
```

**Why it works:** Push each last digit onto the new number.

Verified against 3 test cases: `1200⏎` → `21`; `12345⏎` → `54321`; `5⏎` → `5`

### Exercise 3 · Days to weeks

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Convert a number of days into "W weeks D days".
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int days = sc.nextInt();
 System.out.println(days / 7 + " weeks " + days % 7 + " days");
 }
}
```

**Why it works:** Quotient and remainder of division by 7.

Verified against 3 test cases: `30⏎` → `4 weeks 2 days`; `7⏎` → `1 weeks 0 days`; `0⏎` → `0 weeks 0 days`

## 1.4 Relational, Logical and Ternary Operators; Precedence

### Exercise 1 · Leap year

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int y = sc.nextInt();
 boolean leap = (y % 4 == 0 && y % 100 != 0) || y % 400 == 0;
 System.out.println(leap ? "leap" : "common");
 }
}
```

**Why it works:** Combine the three divisibility rules with && and ||.

Verified against 4 test cases: `2000⏎` → `leap`; `1900⏎` → `common`; `2024⏎` → `leap`

### Exercise 2 · Maximum of three

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int a = sc.nextInt();
 int b = sc.nextInt();
 int c = sc.nextInt();
 int max = a > b ? (a > c ? a : c) : (b > c ? b : c);
 System.out.println(max);
 }
}
```

**Why it works:** Compare a with b, then the winner with c.

Verified against 3 test cases: `4 9 7⏎` → `9`; `5 5 5⏎` → `5`; `-1 -2 -3⏎` → `-1`

### Exercise 3 · In range?

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Print IN if lo <= x <= hi, otherwise OUT.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int x = sc.nextInt();
 int lo = sc.nextInt();
 int hi = sc.nextInt();
 System.out.println(x >= lo && x <= hi ? "IN" : "OUT");
 }
}
```

**Why it works:** x >= lo && x <= hi.

Verified against 3 test cases: `5 1 10⏎` → `IN`; `11 1 10⏎` → `OUT`; `1 1 10⏎` → `IN`

## 1.5 Bitwise Operators and Shifts

### Exercise 1 · Power of two?

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long n = sc.nextLong();
 System.out.println(n > 0 && (n & (n - 1)) == 0 ? "YES" : "NO");
 }
}
```

**Why it works:** A power of two has exactly one 1 bit, so clearing it leaves 0.

Verified against 4 test cases: `64⏎` → `YES`; `12⏎` → `NO`; `1⏎` → `YES`

### Exercise 2 · Count the 1 bits

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Count the 1 bits (population count) with the "clear lowest set bit" trick.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int count = 0;
 while (n != 0) {
 n &= n - 1;
 count++;
 }
 System.out.println(count);
 }
}
```

**Why it works:** Brian Kernighan's method: the loop runs once per set bit.

Verified against 4 test cases: `255⏎` → `8`; `0⏎` → `0`; `1024⏎` → `1`

### Exercise 3 · Set, clear or toggle

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Read n, i and an operation (set / clear / toggle). Print the result.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int i = sc.nextInt();
 String op = sc.next();
 int mask = 1 << i;
 int result;
 if (op.equals("set")) result = n | mask;
 else if (op.equals("clear")) result = n & ~mask;
 else result = n ^ mask;
 System.out.println(result);
 }
}
```

**Why it works:** Build the one-bit mask and apply the matching operator.

Verified against 3 test cases: `5 1 set⏎` → `7`; `5 0 clear⏎` → `4`; `5 2 toggle⏎` → `1`

## 1.6 Output: print, println, printf and String Concatenation

### Exercise 1 · Times table

Ex1.java

```
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 for (int i = 1; i <= 5; i++) {
 System.out.printf("%d x %d = %d%n", n, i, n * i);
 }
 }
}
```

**Why it works:** A loop with printf.

Verified against 2 test cases: `7⏎` → `7 x 1 = 7⏎7 x 2 = 14⏎7 x 3 = 21⏎7 x 4 = 28⏎7 x 5 = 35`; `1⏎` → `1 x 1 = 1⏎1 x 2 = 2⏎1 x 3 = 3⏎1 x 4 = 4⏎1 x 5 = 5`

### Exercise 2 · Invoice total

Ex2.java

```
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 double price = sc.nextDouble();
 int qty = sc.nextInt();
 System.out.printf("Total: %,.2f%n", price * qty);
 }
}
```

**Why it works:** %,.2f formats with grouping and two decimals.

Verified against 3 test cases: `1999.5 3⏎` → `Total: 5,998.50`; `0.1 3⏎` → `Total: 0.30`; `1000000 1⏎` → `Total: 1,000,000.00`

### Exercise 3 · Predict, then run

Ex3.java

```
public class Ex3 {
 // Predict, then verify: what does this print? (the answer is in the quiz)
 public static void main(String[] args) {
 int a = 2, b = 3;
 System.out.println(a + b + "=" + a + b);
 }
}
```

**Why it works:** 2 + 3 = 5 first; then "=" is joined, then 2 and 3 are joined as text.

Verified against 1 test case: `(no input)` → `5=23`

## 1.7 Input: Scanner, BufferedReader and Command-Line Arguments

### Exercise 1 · Average

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 double sum = 0;
 for (int i = 0; i < n; i++) sum += sc.nextDouble();
 System.out.printf("%.2f%n", sum / n);
 }
}
```

**Why it works:** Accumulate the sum as a double.

Verified against 3 test cases: `3⏎1.5 2.5 3.5⏎` → `2.50`; `1⏎7⏎` → `7.00`; `2⏎1 2⏎` → `1.50`

### Exercise 2 · Sum until the end

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 // Sum all integers until the end of input (the count is not given).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long sum = 0;
 while (sc.hasNextLong()) sum += sc.nextLong();
 System.out.println(sum);
 }
}
```

**Why it works:** hasNextLong() ends the loop at end of input.

Verified against 3 test cases: `4 5 6⏎7⏎` → `22`; `10⏎` → `10`; `-5 5⏎` → `0`

### Exercise 3 · Age then name

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 // A name on the first line, an age on the second: the classic nextInt / nextLine trap.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int age = sc.nextInt();
 sc.nextLine(); // throw away the end of the age line
 String name = sc.nextLine();
 System.out.println(name + " is " + age);
 }
}
```

**Why it works:** Call nextLine() once to discard the rest of the age line.

Verified against 2 test cases: `20⏎Asha Rao⏎` → `Asha Rao is 20`; `7⏎Bo⏎` → `Bo is 7`

## 1.8 Decision Making: if, else, switch

### Exercise 1 · Grade

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int m = sc.nextInt();
 if (m >= 90) System.out.println("A");
 else if (m >= 75) System.out.println("B");
 else if (m >= 50) System.out.println("C");
 else System.out.println("F");
 }
}
```

**Why it works:** An else-if ladder ordered from the highest threshold down.

Verified against 4 test cases: `82⏎` → `B`; `90⏎` → `A`; `49⏎` → `F`

### Exercise 2 · Calculator

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 // A small calculator: "<a> <op> <b>" with op one of + - * /
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int a = sc.nextInt();
 char op = sc.next().charAt(0);
 int b = sc.nextInt();
 switch (op) {
 case '+': System.out.println(a + b); break;
 case '-': System.out.println(a - b); break;
 case '*': System.out.println(a * b); break;
 case '/':
 if (b == 0) System.out.println("error");
 else System.out.println(a / b);
 break;
 default: System.out.println("unknown operator");
 }
 }
}
```

**Why it works:** A switch on the operator character with a guard for division.

Verified against 4 test cases: `12 / 4⏎` → `3`; `5 / 0⏎` → `error`; `6 * 7⏎` → `42`

### Exercise 3 · Triangle type

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int a = sc.nextInt();
 int b = sc.nextInt();
 int c = sc.nextInt();
 if (a + b <= c || a + c <= b || b + c <= a) {
 System.out.println("not a triangle");
 } else if (a == b && b == c) {
 System.out.println("equilateral");
 } else if (a == b || b == c || a == c) {
 System.out.println("isosceles");
 } else {
 System.out.println("scalene");
 }
 }
}
```

**Why it works:** Check validity first, then count equal sides.

Verified against 4 test cases: `5 5 8⏎` → `isosceles`; `3 3 3⏎` → `equilateral`; `3 4 5⏎` → `scalene`

## 1.9 Loops: for, while, do-while, break and continue

### Exercise 1 · Factorial

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long f = 1;
 for (int i = 2; i <= n; i++) f *= i;
 System.out.println(f);
 }
}
```

**Why it works:** An accumulator multiplied in a for loop.

Verified against 3 test cases: `10⏎` → `3628800`; `0⏎` → `1`; `20⏎` → `2432902008176640000`

### Exercise 2 · Pyramid

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 for (int row = 1; row <= n; row++) {
 StringBuilder sb = new StringBuilder();
 for (int c = 1; c <= n - row; c++) sb.append(' ');
 for (int c = 1; c <= 2 * row - 1; c++) sb.append('*');
 System.out.println(sb);
 }
 }
}
```

**Why it works:** Two inner loops per row: spaces, then stars.

Verified against 3 test cases: `3⏎` → `*⏎  ***⏎*****`; `1⏎` → `*`; `2⏎` → `*⏎***`

### Exercise 3 · Collatz steps

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Collatz: n -> n/2 if even, 3n+1 if odd. Count the steps until n becomes 1.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long n = sc.nextLong();
 int steps = 0;
 while (n != 1) {
 n = (n % 2 == 0) ? n / 2 : 3 * n + 1;
 steps++;
 }
 System.out.println(steps);
 }
}
```

**Why it works:** The loop condition is "n != 1"; the update depends on parity.

Verified against 3 test cases: `6⏎` → `8`; `1⏎` → `0`; `27⏎` → `111`

## 1.10 Methods: Parameters, Return Values, Scope and Overloading

### Exercise 1 · isPrime method

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 static boolean isPrime(int n) {
 if (n < 2) return false;
 for (int d = 2; (long) d * d <= n; d++) {
 if (n % d == 0) return false;
 }
 return true;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 System.out.println(isPrime(n) ? "prime" : "not prime");
 }
}
```

**Why it works:** Trial division up to √n returns false at the first divisor.

Verified against 4 test cases: `97⏎` → `prime`; `1⏎` → `not prime`; `91⏎` → `not prime`

### Exercise 2 · Overloaded power

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 static long power(long base, int exp) {
 long result = 1;
 for (int i = 0; i < exp; i++) result *= base;
 return result;
 }

 static double power(double base, int exp) { // overloaded version for decimals
 double result = 1;
 for (int i = 0; i < exp; i++) result *= base;
 return result;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long b = sc.nextLong();
 int e = sc.nextInt();
 System.out.println(power(b, e));
 System.out.println(power(2.5, 2));
 }
}
```

**Why it works:** Overloading chooses the version from the argument types.

Verified against 2 test cases: `3 4⏎` → `81⏎6.25`; `2 10⏎` → `1024⏎6.25`

### Exercise 3 · Digits helpers

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 // Return the number of digits and the digit sum through two separate methods.
 static int digitCount(long n) {
 int c = 0;
 do {
 c++;
 n /= 10;
 } while (n > 0);
 return c;
 }

 static int digitSum(long n) {
 int s = 0;
 while (n > 0) {
 s += n % 10;
 n /= 10;
 }
 return s;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long n = sc.nextLong();
 System.out.println(digitCount(n) + " " + digitSum(n));
 }
}
```

**Why it works:** Two small methods are easier to test than one big main.

Verified against 3 test cases: `90210⏎` → `5 12`; `0⏎` → `1 0`; `7⏎` → `1 7`

## 1.11 Tracing and Debugging Programs

### Exercise 1 · Trace by hand

Ex1.java

```java
public class Ex1 {
 // Trace by hand first. What does this print? (the answer is part of the lesson quiz)
 public static void main(String[] args) {
 int x = 1;
 for (int i = 0; i < 4; i++) {
 x = x * 2 + i;
 }
 System.out.println(x);
 }
}
```

**Why it works:** See the table in the lesson: 2, 5, 12, 27.

Verified against 1 test case: `(nc input)` → `27`

### Exercise 2 · Print the trace

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Print the trace table "i value" for the loop value = value * 2 + 1 starting from the input.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int value = sc.nextInt();
 int steps = sc.nextInt();
 for (int i = 1; i <= steps; i++) {
 value = value * 2 + 1;
 System.out.println(i + " " + value);
 }
 }
}
```

**Why it works:** The printed lines are the rows of the trace table.

Verified against 2 test cases: `1 3⏎` → `1 3⏎2 7⏎3 15`; `0 2⏎` → `1 1⏎2 3`

### Exercise 3 · Fix the maximum

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Fixed version of a buggy "largest of n numbers" that started with max = 0.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int max = Integer.MIN_VALUE;
 for (int i = 0; i < n; i++) {
 int x = sc.nextInt();
 if (x > max) max = x;
 }
 System.out.println(max);
 }
}
```

**Why it works:** The initial value must be smaller than every possible input.

Verified against 3 test cases: `3⏎-4 -2 -9⏎` → `-2`; `1⏎0⏎` → `0`; `2⏎5 3⏎` → `5`

# Quiz answer key

## 1.1 Variables, Constants and Primitive Types

| | Answer | Why |
|---|---|---|
| Q1 | C. long | It exceeds the int limit of 2.1 billion. |
| Q2 | line 3 | A double cannot be assigned to an int without a cast (lossy conversion). |
| Q3 | C. totalMarks | It cannot start with a digit, contain "-", or be a keyword. |
| Q4 | 20 | LIMIT * 2 = 20. |
| Q5 | C. String | String is a class (reference type). |

## 1.2 Type Conversion, Casting and Overflow

| | Answer | Why |
|---|---|---|
| Q1 | 6.5 | 3 + 3.5 = 6.5. |
| Q2 | true | Overflow wraps to −2147483648. |
| Q3 | C. long p = 100000L * 100000; | One operand is long, so the multiplication happens in long. |
| Q4 | C | Compound assignment includes an implicit cast. |
| Q5 | A. 9 | Casting truncates toward zero. |

## 1.3 Arithmetic, Assignment and Increment Operators

| | Answer | Why |
|---|---|---|
| Q1 | 12 7 | x++ yields 5 (x→6); ++x makes x 7 and yields 7. y = 12, x = 7. |
| Q2 | -2 | The remainder has the sign of the dividend. |
| Q3 | B. n % 10 | n % 10. |
| Q4 | 2 | 15 → 30 → 2. |
| Q5 | B. 7 | ^ is XOR: 101 ^ 010 = 111 = 7. |

## 1.4 Relational, Logical and Ternary Operators; Precedence

| | Answer | Why |
|---|---|---|
| Q1 | false | b != 0 is false, so a / b is never evaluated (no exception). |
| Q2 | true | && is evaluated before \|\|. |
| Q3 | B. 1 <= x && x <= 10 | Two comparisons joined by &&. |
| Q4 | B. "yes" | The condition is true, so the first value. |
| Q5 | line 2 | `=` assigns; use `==` to compare. |

## 1.5 Bitwise Operators and Shifts

| | Answer | Why |
|---|---|---|
| Q1 | 8 | 1100 & 1010 = 1000. |
| Q2 | 6 | 101 ^ 011 = 110. |
| Q3 | B. x \| (1 << 3) | OR with the mask forces the bit to 1. |

| | Answer | Why |
|---|---|---|
| Q4 | -4 | Arithmetic shift divides by 4 keeping the sign. |
| Q5 | A. clears the lowest set bit | Subtracting 1 flips the lowest set bit and everything below it. |

## 1.6 Output: print, println, printf and String Concatenation

| | Answer | Why |
|---|---|---|
| Q1 | 3345 | 1 + 2 = 3, then "3" joins → "33", then "4", "5" → 3345. |
| Q2 | 66 | char + int is an int. |
| Q3 | B. printf("%.2f", 3.14159) | %.2f formats a decimal with two digits. |
| Q4 | C. print | print leaves the cursor on the same line. |
| Q5 | B. error messages | The standard error stream. |

## 1.7 Input: Scanner, BufferedReader and Command-Line Arguments

| | Answer | Why |
|---|---|---|
| Q1 | B. an empty string (the rest of the number's line) | The newline is still waiting. |
| Q2 | 18 | String concatenation. |
| Q3 | C. BufferedReader | Buffered input is the fastest of the three. |
| Q4 | hasNextInt | hasNextInt() is true while another int is available. |

## 1.8 Decision Making: if, else, switch

| | Answer | Why |
|---|---|---|
| Q1 | B. compile-time error | incompatible types: int cannot be converted to boolean. |
| Q2 | many | No case matches 9, so default runs. |
| Q3 | bcd | Fall-through from case 2. |
| Q4 | B. nothing — ; is an empty statement | The empty statement is legal. |
| Q5 | line 4 | break is only allowed inside a loop or switch. |

## 1.9 Loops: for, while, do-while, break and continue

| | Answer | Why |
|---|---|---|
| Q1 | 7 | 1 + 2 + 4 = 7 (3 is skipped). |
| Q2 | 12 | 3 × 4. |
| Q3 | C. do-while | do-while tests after the body. |
| Q4 | 243 | 1, 3, 9, 27, 81, 243 — stops when i ≥ 100. |
| Q5 | line 4 | n++ moves n away from 0; it should be n--. |

## 1.10 Methods: Parameters, Return Values, Scope and Overloading

| | Answer | Why |
|---|---|---|
| Q1 | 5 | The method changed its own copy. |
| Q2 | C. int f(int a) and int f(double a) | Different parameter types; return type alone does not distinguish. |

| | Answer | Why |
|---|---|---|
| Q3 | line 5 | Not all paths return a value: missing return statement. |
| Q4 | B. only inside that loop body | Scope is the enclosing block. |

## 1.11 Tracing and Debugging Programs

| | Answer | Why |
|---|---|---|
| Q1 | n after pass 1 = 5; n after pass 2 = 2; final count = 3 | 10 → 5 → 2 → 1: three passes. |
| Q2 | A. runs the current line without entering called methods | Step into enters methods; step over treats them as one step. |
| Q3 | B. trace a small failing input | Reproduce and observe. |
| Q4 | 2 | It skips a[0] = 2 and counts only 4 and 6. |

# Part 2 · Core Programming Skills

## Exercise solutions

### 2.1 One-Dimensional Arrays

#### Exercise 1 · Print reversed

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 StringBuilder sb = new StringBuilder();
 for (int i = n - 1; i >= 0; i--) sb.append(i == n - 1 ? "" : " ").append(a[i]);
 System.out.println(sb);
 }
}
```

**Why it works:** A downward loop; no need to change the array.

Verified against 3 test cases: `5⏎1 2 3 4 5⏎` → `5 4 3 2 1`; `1⏎7⏎` → `7`; `3⏎-1 0 1⏎` → `1 0 -1`

#### Exercise 2 · Digit frequencies

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Frequency of each value 0..9 in the array.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] freq = new int[10];
 for (int i = 0; i < n; i++) freq[sc.nextInt()]++;
 for (int d = 0; d < 10; d++) {
 if (freq[d] > 0) System.out.println(d + ": " + freq[d]);
 }
 }
}
```

**Why it works:** The value itself is the index of its counter.

Verified against 2 test cases: `6⏎3 1 3 0 9 1⏎` → `0: 1⏎1: 2⏎3: 2⏎9: 1`; `1⏎5⏎` → `5: 1`

#### Exercise 3 · Second largest distinct

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 // Second largest DISTINCT value, or -1 if there is none. One pass, O(1) space.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long first = Long.MIN_VALUE, second = Long.MIN_VALUE;
 for (int i = 0; i < n; i++) {
 long x = sc.nextLong();
 if (x > first) {
 second = first;
 first = x;
 } else if (x > second && x < first) {
 second = x;
 }
 }
 System.out.println(second == Long.MIN_VALUE ? "-1" : String.valueOf(second));
 }
}
```

**Why it works:** Two variables updated in one pass; duplicates of the maximum must not become the second.

Verified against 3 test cases: `5⏎4 9 9 2 7⏎` → `7`; `3⏎5 5 5⏎` → `-1`; `2⏎-3 -1⏎` → `-3`

## 2.2 Two-Dimensional Arrays (Matrices)

### Exercise 1 · Sum of a matrix

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int r = sc.nextInt();
 int c = sc.nextInt();
 long sum = 0;
 for (int i = 0; i < r; i++) for (int j = 0; j < c; j++) sum += sc.nextInt();
 System.out.println(sum);
 }
}
```

**Why it works:** Accumulate while reading.

Verified against 2 test cases: `2 3⏎1 2 3⏎4 5 6⏎` → `21`; `1 1⏎-4⏎` → `-4`

### Exercise 2 · Transpose

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 // Print the transpose of an r x c matrix.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int r = sc.nextInt();
 int c = sc.nextInt();
 int[][] m = new int[r][c];
 for (int i = 0; i < r; i++) for (int j = 0; j < c; j++) m[i][j] = sc.nextInt();
 for (int j = 0; j < c; j++) {
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < r; i++) sb.append(i == 0 ? "" : " ").append(m[i][j]);
 System.out.println(sb);
 }
 }
}
```

**Why it works:** Print column j of the original as row j.

Verified against 2 test cases: `2 3⏎1 2 3⏎4 5 6⏎` → `1 4⏎2 5⏎3 6`; `1 2⏎7 8⏎` → `7⏎8`

### Exercise 3 · Spiral order

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 // Print an n x n matrix in clockwise spiral order, space separated.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[][] m = new int[n][n];
 for (int i = 0; i < n; i++) for (int j = 0; j < n; j++) m[i][j] = sc.nextInt();
 StringBuilder sb = new StringBuilder();
 int top = 0, bottom = n - 1, left = 0, right = n - 1;
 while (top <= bottom && left <= right) {
 for (int j = left; j <= right; j++) sb.append(m[top][j]).append(' ');
 top++;
 for (int i = top; i <= bottom; i++) sb.append(m[i][right]).append(' ');
 right--;
 if (top <= bottom) {
 for (int j = right; j >= left; j--) sb.append(m[bottom][j]).append(' ');
 bottom--;
 }
 if (left <= right) {
 for (int i = bottom; i >= top; i--) sb.append(m[i][left]).append(' ');
 left++;
 }
 }
 System.out.println(sb.toString().trim());
 }
}
```

**Why it works:** Peel the outer ring, then recurse inwards by moving the boundaries.

Verified against 3 test cases: `3⏎1 2 3⏎4 5 6⏎7 8 9⏎` → `1 2 3 6 9 8 7 4 5`; `1⏎5⏎` → `5`; `2⏎1 2⏎3 4⏎` → `1 2 4 3`

## 2.3 Strings, StringBuilder and Characters

### Exercise 1 · Count vowels

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.nextLine().toLowerCase();
 int v = 0;
 for (char c : s.toCharArray()) {
 if ("aeiou".indexOf(c) >= 0) v++;
 }
 System.out.println(v);
 }
}
```

**Why it works:** Lower-case the text and test each char for membership.

Verified against 3 test cases: `Programming Language⏎` → `7`; `rhythm⏎` → `0`; `AEIOU aeiou⏎` → `10`

### Exercise 2 · Anagrams

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Are the two words anagrams (same letters, same counts, ignoring case)?
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String a = sc.next().toLowerCase();
 String b = sc.next().toLowerCase();
 int[] count = new int[26];
 for (char c : a.toCharArray()) count[c - 'a']++;
 for (char c : b.toCharArray()) count[c - 'a']--;
 boolean ok = a.length() == b.length();
 for (int x : count) if (x != 0) ok = false;
 System.out.println(ok ? "YES" : "NO");
 }
}
```

**Why it works:** Equal multisets of letters give all-zero counters.

Verified against 3 test cases: `Listen Silent⏎` → `YES`; `abc abd⏎` → `NO`; `aa a⏎` → `NO`

### Exercise 3 · Run-length encoding

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 // Run-length encode: aaabccdddd -> a3b1c2d4
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 StringBuilder out = new StringBuilder();
 int i = 0;
 while (i < s.length()) {
 int j = i;
 while (j < s.length() && s.charAt(j) == s.charAt(i)) j++;
 out.append(s.charAt(i)).append(j - i);
 i = j;
 }
 System.out.println(out);
 }
}
```

**Why it works:** Find the end j of the run starting at i, append char and count, continue at j.

Verified against 3 test cases: `aaabccdddd⏎` → `a3b1c2d4`; `a⏎` → `a1`; `abab⏎` → `a1b1a1b1`

## 2.4 Number Systems, Binary and Two's Complement

### Exercise 1 · To binary

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 if (n == 0) {
 System.out.println("0");
 return;
 }
 StringBuilder sb = new StringBuilder();
 while (n > 0) {
 sb.append(n % 2);
 n /= 2;
 }
 System.out.println(sb.reverse());
 }
}
```

**Why it works:** Remainders are the bits from low to high.

Verified against 4 test cases: `156⏎` → `10011100`; `0⏎` → `0`; `1⏎` → `1`

### Exercise 2 · From any base

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 int base = sc.nextInt();
 long value = 0;
 for (char c : s.toCharArray()) value = value * base + Character.digit(c, base);
 System.out.println(value);
 }
}
```

**Why it works:** Horner's rule over the digits.

Verified against 3 test cases: `9C 16⏎` → `156`; `1010 2⏎` → `10`; `777 8⏎` → `511`

### Exercise 3 · Add two binary strings

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 // Add two binary numbers given as strings (no conversion to int).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String a = sc.next();
 String b = sc.next();
 StringBuilder sb = new StringBuilder();
 int i = a.length() - 1, j = b.length() - 1, carry = 0;
 while (i >= 0 || j >= 0 || carry > 0) {
 int sum = carry;
 if (i >= 0) sum += a.charAt(i--) - '0';
 if (j >= 0) sum += b.charAt(j--) - '0';
 sb.append(sum % 2);
 carry = sum / 2;
 }
 System.out.println(sb.reverse());
 }
}
```

**Why it works:** Column addition: sum = a + b + carry; digit = sum % 2; carry = sum / 2.

Verified against 4 test cases: `1010 1110⏎` → `11000`; `1 1⏎` → `10`; `0 0⏎` → `0`

## 2.5 Divisibility, GCD/LCM, Primes and Modular Arithmetic

### Exercise 1 · gcd and lcm

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long a = sc.nextLong(), b = sc.nextLong();
 long x = a, y = b;
 while (y != 0) {
 long r = x % y;
 x = y;
 y = r;
 }
 System.out.println(x + " " + (a / x * b));
 }
}
```

**Why it works:** Euclid for the gcd; divide first to avoid overflow in the lcm.

Verified against 3 test cases: `12 18⏎` → `6 36`; `7 13⏎` → `1 91`; `1000000000 1000000000⏎` → `1000000000 1000000000`

### Exercise 2 · All divisors

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 // Print all divisors of n in increasing order using the sqrt trick (pairs d, n/d).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long n = sc.nextLong();
 java.util.TreeSet<Long> divisors = new java.util.TreeSet<>();
 for (long d = 1; d * d <= n; d++) {
 if (n % d == 0) {
 divisors.add(d);
 divisors.add(n / d);
 }
 }
 StringBuilder sb = new StringBuilder();
 for (long d : divisors) sb.append(sb.length() == 0 ? "" : " ").append(d);
 System.out.println(sb);
 }
}
```

**Why it works:** Divisors pair up; a TreeSet keeps them sorted and removes the duplicate when d = n/d.

Verified against 3 test cases: `36⏎` → `1 2 3 4 6 9 12 18 36`; `1⏎` → `1`; `97⏎` → `1 97`

### Exercise 3 · Modular power

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 // a^b mod m with fast exponentiation
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long a = sc.nextLong(), b = sc.nextLong(), m = sc.nextLong();
 long result = 1 % m;
 a %= m;
 while (b > 0) {
 if ((b & 1) == 1) result = result * a % m;
 a = a * a % m;
 b >>= 1;
 }
 System.out.println(result);
 }
}
```

**Why it works:** O(log b) squarings; each product is below $m^2 \le 10^{18}$ so long suffices.

Verified against 3 test cases: `3 200 13⏎` → `9`; `2 10 1000⏎` → `24`; `5 0 7⏎` → `1`

## 2.6 Recursion: Base Case, Recursive Case and the Call Stack

### Exercise 1 · Recursive digit sum

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 static int sumDigits(int n) {
 if (n < 10) return n;
 return n % 10 + sumDigits(n / 10);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 System.out.println(sumDigits(sc.nextInt()));
 }
}
```

**Why it works:** The last digit plus the digit sum of the rest.

Verified against 3 test cases: `4821⏎` → `15`; `0⏎` → `0`; `999⏎` → `27`

### Exercise 2 · Towers of Hanoi

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 // Towers of Hanoi: print the moves for n disks from rod A to rod C using B.
 static int moves = 0;
 static StringBuilder out = new StringBuilder();

 static void hanoi(int n, char from, char to, char via) {
 if (n == 0) return;
 hanoi(n - 1, from, via, to);
 out.append("Move disk ").append(n).append(" from ").append(from).append(" to ").append(to)
     .append('\n');
 moves++;
 hanoi(n - 1, via, to, from);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 hanoi(n, 'A', 'C', 'B');
 System.out.print(out);
 System.out.println("total moves: " + moves);
 }
}
```

**Why it works:** T(n) = 2T(n−1) + 1 = $2^n$ − 1 moves.

Verified against 2 test cases: `2⏎` → `Move disk 1 from A to B⏎Move disk 2 from A to C⏎Move disk 1 from B to C⏎total moves: 3`; `1⏎` → `Move disk 1 from A to C⏎total moves: 1`

### Exercise 3 · All subsets

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 // Print every subset of {1..n} (in the order: without the element first, then with it).
 static StringBuilder out = new StringBuilder();

 static void subsets(int i, int n, java.util.ArrayList<Integer> cur) {
 if (i > n) {
 out.append(cur.toString()).append('\n');
 return;
 }
 subsets(i + 1, n, cur); // skip i
 cur.add(i);
 subsets(i + 1, n, cur); // take i
 cur.remove(cur.size() - 1); // undo
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 subsets(1, sc.nextInt(), new java.util.ArrayList<>());
 System.out.print(out);
 }
}
```

**Why it works:** Each element is either excluded or included: $2^n$ leaves.

Verified against 2 test cases: `2⏎` → `[]⏎[2]⏎[1]⏎[1, 2]`; `1⏎` → `[]⏎[1]`

## 2.7 References, Pass-by-Value and Memory

### Exercise 1 · Swap inside an array

Ex1.java

```java
import java.util.Arrays;
import java.util.Scanner;

public class Ex1 {
 // Write a method that swaps a[i] and a[j] IN the array (this works because arrays are objects).
 static void swap(int[] a, int i, int j) {
 int t = a[i];
 a[i] = a[j];
 a[j] = t;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int k = 0; k < n; k++) a[k] = sc.nextInt();
 int i = sc.nextInt(), j = sc.nextInt();
 swap(a, i, j);
 System.out.println(Arrays.toString(a));
 }
}
```

**Why it works:** The parameter references the caller's array.

Verified against 2 test cases: `4⏎1 2 3 4⏎0 3⏎` → `[4, 2, 3, 1]`; `2⏎5 6⏎0 0⏎` → `[5, 6]`

### Exercise 2 · Return a new array

Ex2.java

```java
import java.util.Arrays;
import java.util.Scanner;

public class Ex2 {
 // Return a NEW array with every element doubled; the input array must stay unchanged.
 static int[] doubled(int[] a) {
 int[] r = new int[a.length];
 for (int i = 0; i < a.length; i++) r[i] = a[i] * 2;
 return r;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int k = 0; k < n; k++) a[k] = sc.nextInt();
 int[] d = doubled(a);
 System.out.println(Arrays.toString(a) + " " + Arrays.toString(d));
 }
}
```

**Why it works:** Creating a new array keeps the caller's data intact.

Verified against 2 test cases: `3⏎1 2 3⏎` → `[1, 2, 3] [2, 4, 6]`; `1⏎0⏎` → `[0] [0]`

### Exercise 3 · Alias or copy?

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Prints "same" or "different" for two array variables that refer to the same / different arrays.
 // Mode 1: b = a (alias). Mode 2: b = a.clone() (copy).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int mode = sc.nextInt();
 int[] a = {1, 2, 3};
 int[] b = mode == 1 ? a : a.clone();
 b[0] = 99;
 System.out.println(a[0] == 99 ? "same" : "different");
 }
}
```

**Why it works:** An alias shares the object; a clone does not.

Verified against 2 test cases: `1⏎` → `same`; `2⏎` → `different`

## 2.8 Boundary Conditions, Validation and Common Mistakes

### Exercise 1 · Sum the valid numbers

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // Read tokens; print the sum of the ones that are valid integers and the count of invalid tokens.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long sum = 0;
 int bad = 0;
 while (sc.hasNext()) {
 String t = sc.next();
 try {
 sum += Integer.parseInt(t);
 } catch (NumberFormatException e) {
 bad++;
 }
 }
 System.out.println(sum + " " + bad);
 }
}
```

**Why it works:** Each token is validated by attempting the conversion.

Verified against 3 test cases: `5 x 7 3.5 -2⏎` → `10 2`; `a b⏎` → `0 2`; `10 20⏎` → `30 0`

### Exercise 2 · Maximum or empty

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 // Largest element, or `empty` when n = 0.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 if (n == 0) {
 System.out.println("empty");
 return;
 }
 int max = Integer.MIN_VALUE;
 for (int i = 0; i < n; i++) max = Math.max(max, sc.nextInt());
 System.out.println(max);
 }
}
```

**Why it works:** A guard for the empty case; MIN_VALUE as the initial maximum.

Verified against 3 test cases: `0⏎` → `empty`; `3⏎-5 -2 -9⏎` → `-2`; `1⏎7⏎` → `7`

### Exercise 3 · Safe element access

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 // Safe element access: print a[i] if 0 <= i < n else `out of range`.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int k = 0; k < n; k++) a[k] = sc.nextInt();
 int i = sc.nextInt();
 System.out.println(i >= 0 && i < n ? String.valueOf(a[i]) : "out of range");
 }
}
```

**Why it works:** The bounds check comes before the access.

Verified against 3 test cases: `3⏎10 20 30⏎3⏎` → `out of range`; `3⏎10 20 30⏎0⏎` → `10`; `3⏎10 20 30⏎-1⏎` → `out of range`

# Quiz answer key

## 2.1 One-Dimensional Arrays

| | Answer | Why |
|---|---|---|
| Q1 | 5 1 | length is 5; index 3 holds 1. |
| Q2 | 9 | b is an alias of a. |
| Q3 | B. three zeros | Numeric arrays are zero-initialised. |
| Q4 | C. 0 … n−1 | Zero-based indexing. |

## 2.2 Two-Dimensional Arrays (Matrices)

| | Answer | Why |
|---|---|---|
| Q1 | 3 2 6 | 3 rows, 2 columns, element in row 2 col 1. |
| Q2 | B. i == j | Row index equals column index. |

| | Answer | Why |
|---|---|---|
| Q3 | B. O(R·C) | Every cell once. |
| Q4 | B. new int[3][] | Only the first dimension is required. |

## 2.3 Strings, StringBuilder and Characters

| | Answer | Why |
|---|---|---|
| Q1 | el2 | substring(1,3) = "el"; first l is at index 2. |
| Q2 | false | Different objects. |
| Q3 | B. StringBuilder | It avoids $O(n^2)$ copying. |
| Q4 | 3 | char arithmetic: 100 − 97. |
| Q5 | B. immutable | Their contents never change after creation. |

## 2.4 Number Systems, Binary and Two's Complement

| | Answer | Why |
|---|---|---|
| Q1 | 11 | 1·8 + 0·4 + 1·2 + 1·1. |
| Q2 | 11111101 | Flip → 11111100, add 1 → 11111101. |
| Q3 | C. 4 | $16 = 2^4$. |
| Q4 | C. −128..127 | $-2^7 \ldots 2^7 - 1$. |
| Q5 | B. the least significant bit | It is n mod 2. |

## 2.5 Divisibility, GCD/LCM, Primes and Modular Arithmetic

| | Answer | Why |
|---|---|---|
| Q1 | B. 432 | It equals 18 × 24 = 432. |
| Q2 | 2 | −7 = −3·3 + 2. |
| Q3 | C. √n | A composite has a divisor ≤ √n. |
| Q4 | 1 | 2 · 3 = 6, 6 mod 5 = 1. |

## 2.6 Recursion: Base Case, Recursive Case and the Call Stack

| | Answer | Why |
|---|---|---|
| Q1 | 10 | 4 + 3 + 2 + 1 + 0. |
| Q2 | B. StackOverflowError | Frames pile up until the stack is exhausted. |
| Q3 | C. 15 | calls(n) = calls(n−1) + calls(n−2) + 1: 1,1,3,5,9,15. |
| Q4 | B. O(n) | n stack frames at the deepest point. |
| Q5 | B. memoisation / dynamic programming | Cache each sub-answer once. |

## 2.7 References, Pass-by-Value and Memory

| | Answer | Why |
|---|---|---|
| Q1 | 7 | The element change is seen; the reassignment is not. |
| Q2 | B. by value (references are copied) | Always by value; for objects the value is the reference. |

| | Answer | Why |
|---|---|---|
| Q3 | B. NullPointerException | NullPointerException. |
| Q4 | B. heap | The heap; references to them may be on the stack. |

## 2.8 Boundary Conditions, Validation and Common Mistakes

| | Answer | Why |
|---|---|---|
| Q1 | true | The absolute value overflows back to MIN_VALUE. |
| Q2 | B. lo + (hi - lo) / 2 | It cannot overflow when lo ≤ hi. |
| Q3 | B. an empty array | There is no maximum of nothing. |
| Q4 | B. NumberFormatException | It is not a valid int. |

# Part 3 · Object-Oriented Programming

## Exercise solutions

### 3.1 Classes, Objects, Fields, Constructors and this

#### Exercise 1 · Rectangle class

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 static class Rectangle {
 double width, height;

 Rectangle(double width, double height) {
 this.width = width;
 this.height = height;
 }

 double area() { return width * height; }
 double perimeter() { return 2 * (width + height); }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 Rectangle r = new Rectangle(sc.nextDouble(), sc.nextDouble());
 System.out.println(r.area() + " " + r.perimeter());
 }
}
```

**Why it works:** Fields + constructor + two methods that use the fields.

Verified against 2 test cases: `2.5 4⏎` → `10.0 13.0`; `1 1⏎` → `1.0 4.0`

#### Exercise 2 · Counter

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 static class Counter {
 private int count;

 void increment() { count++; }
 void add(int k) { count += k; }
 int get() { return count; }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 Counter c = new Counter();
 for (int i = 0; i < n; i++) {
 String op = sc.next();
 if (op.equals("inc")) c.increment();
 else c.add(sc.nextInt());
 }
 System.out.println(c.get());
 }
}
```

**Why it works:** State in a field changed only through methods.

Verified against 2 test cases: `4⏎inc⏎add 5⏎inc⏎add 10⏎` → `17`; `1⏎inc⏎` → `1`

### Exercise 3 · Distance between points

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 static class Point {
 final int x, y;

 Point(int x, int y) {
 this.x = x;
 this.y = y;
 }

 double distanceTo(Point other) {
 int dx = x - other.x;
 int dy = y - other.y;
 return Math.sqrt(dx * dx + dy * dy);
 }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 Point a = new Point(sc.nextInt(), sc.nextInt());
 Point b = new Point(sc.nextInt(), sc.nextInt());
 System.out.printf("%.3f%n", a.distanceTo(b));
 }
}
```

**Why it works:** A method that takes another object of the same class.

Verified against 3 test cases: `0 0 3 4⏎` → `5.000`; `1 1 1 1⏎` → `0.000`; `-1 -1 2 3⏎` → `5.000`

## 3.2 Encapsulation and Access Modifiers

### Exercise 1 · Validated temperature

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 static class Temperature {
 private double celsius;

 void setCelsius(double c) {
 if (c < -273.15) throw new IllegalArgumentException("below absolute zero");
 celsius = c;
 }

 double getCelsius() { return celsius; }
 double getFahrenheit() { return celsius * 9 / 5 + 32; }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 Temperature t = new Temperature();
 try {
 t.setCelsius(sc.nextDouble());
 System.out.printf("%.1f%n", t.getFahrenheit());
 } catch (IllegalArgumentException e) {
 System.out.println("invalid: " + e.getMessage());
 }
 }
}
```

**Why it works:** The setter protects the invariant.

Verified against 3 test cases: `36.6⏎` → `97.9`; `-300⏎` → `invalid: below absolute zero`; `100⏎` → `212.0`

### Exercise 2 · Bounded stack

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 static class Stack {
 private final int[] data;
 private int size;

 Stack(int capacity) { data = new int[capacity]; }

 boolean push(int x) {
 if (size == data.length) return false;
 data[size++] = x;
 return true;
 }

 int pop() {
 if (size == 0) throw new IllegalStateException("empty");
 return data[--size];
 }

 boolean isEmpty() { return size == 0; }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 Stack s = new Stack(3);
 int n = sc.nextInt();
 StringBuilder out = new StringBuilder();
 for (int i = 0; i < n; i++) {
 String op = sc.next();
 if (op.equals("push")) {
 out.append(s.push(sc.nextInt()) ? "ok " : "full ");
 } else {
 out.append(s.isEmpty() ? "empty " : s.pop() + " ");
 }
 }
 System.out.println(out.toString().trim());
 }
}
```

**Why it works:** The array is hidden: callers cannot corrupt size.

Verified against 2 test cases: `5⏎push 1⏎push 2⏎pop⏎pop⏎pop⏎` → `ok ok 2 1 empty`; `5⏎push 1⏎push 2⏎push 3⏎push 4⏎pop⏎` → `ok ok ok full 3`

### Exercise 3 · Access levels

**Ex3.java**

```java
public class Ex3 {
 // Which members are accessible? A tiny demonstration with output you can predict.
 static class Vault {
 private int secret = 42;
 int packageLevel = 7;
 protected int family = 3;
 public int open = 1;

 int reveal() { return secret; } // inside the class: private is accessible
 }

 public static void main(String[] args) {
 Vault v = new Vault();
 // v.secret would be fine here only because Vault is a nested class of Ex3; the lesson explains
     the rule.
 System.out.println(v.reveal() + " " + v.packageLevel + " " + v.family + " " + v.open);
 }
}
```

**Why it works:** Nested classes share access with their enclosing class; across top-level classes in different packages only public (and protected for subclasses) would be visible.

Verified against 1 test case: `(nc input)` → `42 7 3 1`

## 3.3 Inheritance, Overriding and Polymorphism

### Exercise 1 · Animal sounds

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 static class Animal {
 String sound() { return "..."; }
 }

 static class Dog extends Animal {
 @Override String sound() { return "Woof"; }
 }

 static class Cat extends Animal {
 @Override String sound() { return "Meow"; }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) {
 String kind = sc.next();
 Animal a = kind.equals("dog") ? new Dog() : kind.equals("cat") ? new Cat() : new Animal();
 sb.append(sb.length() == 0 ? "" : " ").append(a.sound());
 }
 System.out.println(sb);
 }
}
```

**Why it works:** Polymorphic call: the object's class decides the output.

Verified against 2 test cases: `3⏎dog⏎cat⏎fish⏎` → `Woof Meow ...`; `1⏎cat⏎` → `Meow`

### Exercise 2 · Payroll

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 static class Employee {
 final String name;
 final double base;

 Employee(String name, double base) {
 this.name = name;
 this.base = base;
 }

 double pay() { return base; }
 }

 static class Manager extends Employee {
 Manager(String name, double base) { super(name, base); }

 @Override double pay() { return base * 1.5; } // 50% bonus
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 double total = 0;
 for (int i = 0; i < n; i++) {
 String role = sc.next();
 String name = sc.next();
 double base = sc.nextDouble();
 Employee e = role.equals("manager") ? new Manager(name, base) : new Employee(name, base);
 total += e.pay();
 }
 System.out.printf("%.2f%n", total);
 }
}
```

**Why it works:** The same loop works for every employee type.

Verified against 2 test cases: `2⏎manager Ann 1000⏎staff Bob 800⏎` → `2300.00`; `1⏎staff Z 10⏎` → `10.00`

### Exercise 3 · Predict the order

**Ex3.java**

```java
public class Ex3 {
 // Predict the output: constructors run parent-first; the overridden method is chosen by the
     OBJECT's type.
 static class A {
 A() { System.out.println("A()"); }
 String who() { return "A"; }
 }

 static class B extends A {
 B() { System.out.println("B()"); }
 @Override String who() { return "B"; }
 }

 public static void main(String[] args) {
 A x = new B();
 System.out.println(x.who());
 }
}
```

**Why it works:** new B() runs A() then B(); who() is dispatched on the actual object (B).

Verified against 1 test case: `(nc input)` → `A()⏎B()⏎B`

## 3.4 Abstract Classes and Interfaces

### Exercise 1 · Total area

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 interface Shape {
 double area();
 }

 static class Circle implements Shape {
 final double r;
 Circle(double r) { this.r = r; }
 public double area() { return Math.PI * r * r; }
 }

 static class Square implements Shape {
 final double s;
 Square(double s) { this.s = s; }
 public double area() { return s * s; }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 double total = 0;
 for (int i = 0; i < n; i++) {
 String kind = sc.next();
 double v = sc.nextDouble();
 Shape s = kind.equals("circle") ? new Circle(v) : new Square(v);
 total += s.area();
 }
 System.out.printf("%.2f%n", total);
 }
}
```

**Why it works:** The loop sees only Shape; each object supplies its own area.

Verified against 2 test cases: `3⏎circle 1⏎square 2⏎circle 2⏎` → `19.71`; `1⏎square 3⏎` → `9.00`

### Exercise 2 · Abstract Vehicle

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 static abstract class Vehicle {
 abstract int wheels();

 String describe() {
 return getClass().getSimpleName() + " has " + wheels() + " wheels";
 }
 }

 static class Bike extends Vehicle { int wheels() { return 2; } }
 static class Car extends Vehicle { int wheels() { return 4; } }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 Vehicle v = sc.next().equals("bike") ? new Bike() : new Car();
 System.out.println(v.describe());
 }
}
```

**Why it works:** Shared code in the abstract class, variable part in the subclass.

Verified against 2 test cases: `bike⏎` → `Bike has 2 wheels`; `car⏎` → `Car has 4 wheels`

### Exercise 3 · Comparable versions

**Ex3.java**

```java
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Scanner;

public class Ex3 {
 // Implementing Comparable<T> gives a class a natural ordering usable by sort().
 static class Version implements Comparable<Version> {
 final int major, minor;

 Version(String s) {
 String[] p = s.split("\\.");
 major = Integer.parseInt(p[0]);
 minor = Integer.parseInt(p[1]);
 }

 @Override
 public int compareTo(Version o) {
 if (major != o.major) return Integer.compare(major, o.major);
 return Integer.compare(minor, o.minor);
 }

 @Override
 public String toString() {
 return major + "." + minor;
 }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 List<Version> list = new ArrayList<>();
 for (int i = 0; i < n; i++) list.add(new Version(sc.next()));
 Collections.sort(list);
 StringBuilder sb = new StringBuilder();
 for (Version v : list) sb.append(sb.length() == 0 ? "" : " ").append(v);
 System.out.println(sb);
 }
}
```

**Why it works:** compareTo defines the natural order used by Collections.sort.

Verified against 2 test cases: `4⏎1.10⏎1.2⏎2.0⏎1.9⏎` → `1.2 1.9 1.10 2.0`; `2⏎3.1⏎3.0⏎` → `3.0 3.1`

## 3.5 static, final and Initialisation Order

### Exercise 1 · ID generator

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 static class IdGenerator {
 private static int next = 1;

 static int nextId() {
 return next++;
 }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) sb.append(sb.length() == 0 ? "" : " ").append(IdGenerator.nextId());
 System.out.println(sb);
 }
}
```

**Why it works:** One shared counter increments on every call.

Verified against 2 test cases: `4⏎` → `1 2 3 4`; `1⏎` → `1`

### Exercise 2 · A constant

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 static final double PI_APPROX = 3.14159;

 static double circleArea(double r) {
 return PI_APPROX * r * r;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 System.out.printf("%.3f%n", circleArea(sc.nextDouble()));
 }
}
```

**Why it works:** A named constant avoids magic numbers.

Verified against 2 test cases: `2⏎` → `12.566`; `1⏎` → `3.142`

### Exercise 3 · Initialisation order

**Ex3.java**

```java
public class Ex3 {
 // Predict the output of the initialisation order.
 static class Init {
 static { System.out.println("static block"); }
 { System.out.println("instance block"); }
 Init() { System.out.println("constructor"); }
 }

 public static void main(String[] args) {
 new Init();
 new Init();
 }
}
```

**Why it works:** The class is loaded once; each object repeats instance init + constructor.

Verified against 1 test case: `(nc input)` → `static block⏎instance block⏎constructor⏎instance block⏎constructor`

## 3.6 Packages and Imports

### Exercise 1 · Static import

**Ex1.java**

```java
import java.util.Scanner;
import static java.lang.Math.sqrt;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 double a = sc.nextDouble();
 System.out.println(sqrt(a));
 }
}
```

**Why it works:** The static import lets you write sqrt(x).

Verified against 2 test cases: `16⏎` → `4.0`; `2⏎` → `1.4142135623730951`

### Exercise 2 · Sort with Arrays

**Ex2.java**

```java
import java.util.Arrays;
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 Arrays.sort(a);
 System.out.println(Arrays.toString(a));
 }
}
```

**Why it works:** Two utility methods from one imported class.

Verified against 2 test cases: `4⏎9 3 7 1⏎` → `[1, 3, 7, 9]`; `1⏎5⏎` → `[5]`

### Exercise 3 · Same name, two packages

**Ex3.java**

```java
public class Ex3 {
 // Two classes named Date live in different packages; use fully qualified names to tell them
    apart.
 public static void main(String[] args) {
 java.util.Date util = new java.util.Date(0);
 java.sql.Date sql = new java.sql.Date(0);
 System.out.println(util.getClass().getName());
 System.out.println(sql.getClass().getName());
 }
}
```

**Why it works:** Fully qualified names disambiguate clashes.

Verified against 1 test case: `(nc input)` → `java.util.Date⏎java.sql.Date`

## 3.7 Exceptions: try, catch, finally, throw and throws

### Exercise 1 · Safe division

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String a = sc.next();
 String b = sc.next();
 try {
 int x = Integer.parseInt(a);
 int y = Integer.parseInt(b);
 System.out.println(x / y);
 } catch (NumberFormatException e) {
 System.out.println("bad number");
 } catch (ArithmeticException e) {
 System.out.println("division by zero");
 }
 }
}
```

**Why it works:** parseInt throws NumberFormatException; the division throws ArithmeticException.

Verified against 3 test cases: `12 4⏎` → `3`; `5 0⏎` → `division by zero`; `x 2⏎` → `bad number`

### Exercise 2 · Custom exception

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 static class NegativeAgeException extends RuntimeException {
 NegativeAgeException(int age) {
 super("age cannot be negative: " + age);
 }
 }

 static void check(int age) {
 if (age < 0) throw new NegativeAgeException(age);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 for (int i = 0; i < n; i++) {
 int age = sc.nextInt();
 try {
 check(age);
 System.out.println("ok " + age);
 } catch (NegativeAgeException e) {
 System.out.println(e.getMessage());
 }
 }
 }
}
```

**Why it works:** throw inside the check method; catch in the loop.

Verified against 2 test cases: `3⏎20⏎-5⏎0⏎` → `ok 20⏎age cannot be negative: -5⏎ok 0`; `1⏎-1⏎` → `age cannot be negative: -1`

### Exercise 3 · try / finally order

Ex3.java

```java
public class Ex3 {
 // Predict: which lines print, and in what order?
 static int f() {
 try {
 System.out.println("try");
 return 1;
 } finally {
 System.out.println("finally");
 }
 }

 public static void main(String[] args) {
 System.out.println(f());
 }
}
```

**Why it works:** The return value is computed, finally runs, then the value returns.

Verified against 1 test case: `(nc input)` → `try⏎finally⏎1`

## 3.8 Generics and Lists: ArrayList and LinkedList

### Exercise 1 · Remove duplicates

Ex1.java

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ex1 {
 // Remove all duplicates keeping the first occurrence (order preserved), using a List only.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 List<Integer> out = new ArrayList<>();
 for (int i = 0; i < n; i++) {
 int x = sc.nextInt();
 if (!out.contains(x)) out.add(x);
 }
 StringBuilder sb = new StringBuilder();
 for (int x : out) sb.append(sb.length() == 0 ? "" : " ").append(x);
 System.out.println(sb);
 }
}
```

**Why it works:** contains is O(n), so this is O(n²) — a HashSet makes it O(n) (next lesson).

Verified against 2 test cases: `6⏎3 1 3 2 1 4⏎` → `3 1 2 4`; `3⏎7 7 7⏎` → `7`

### Exercise 2 · Remove a value

**Ex2.java**

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ex2 {
 // Delete every element equal to the given value from a list, safely (no
     ConcurrentModificationException).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 List<Integer> list = new ArrayList<>();
 for (int i = 0; i < n; i++) list.add(sc.nextInt());
 int target = sc.nextInt();
 list.removeIf(x -> x == target);
 System.out.println(list);
 }
}
```

**Why it works:** removeIf iterates safely and in one pass.

Verified against 2 test cases: `6⏎3 1 3 2 1 4⏎3⏎` → `[1, 2, 1, 4]`; `2⏎5 5⏎5⏎` → `[]`

### Exercise 3 · Generic maximum

**Ex3.java**

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ex3 {
 // Generic method: return the largest element of any list of Comparable items.
 static <T extends Comparable<T>> T largest(List<T> list) {
 T best = list.get(0);
 for (T x : list) if (x.compareTo(best) > 0) best = x;
 return best;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 List<String> words = new ArrayList<>();
 for (int i = 0; i < n; i++) words.add(sc.next());
 System.out.println(largest(words));
 }
}
```

**Why it works:** A bounded generic method works for any comparable type.

Verified against 2 test cases: `4⏎pear apple zebra fig⏎` → `zebra`; `1⏎x⏎` → `x`

## 3.9 Sets and Maps: HashSet, TreeSet, HashMap, TreeMap

### Exercise 1 · Distinct count

**Ex1.java**

```java
import java.util.HashSet;
import java.util.Scanner;
import java.util.Set;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 Set<Integer> set = new HashSet<>();
 for (int i = 0; i < n; i++) set.add(sc.nextInt());
 System.out.println(set.size());
 }
}
```

**Why it works:** A set keeps each value once.

Verified against 2 test cases: `6⏎3 1 3 2 1 4⏎` → `4`; `1⏎9⏎` → `1`

### Exercise 2 · Word frequency

**Ex2.java**

```java
import java.util.Scanner;
import java.util.TreeMap;

public class Ex2 {
 // Word frequencies, words printed in alphabetical order.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 TreeMap<String, Integer> freq = new TreeMap<>();
 while (sc.hasNext()) {
 freq.merge(sc.next().toLowerCase(), 1, Integer::sum);
 }
 for (var e : freq.entrySet()) {
 System.out.println(e.getKey() + " " + e.getValue());
 }
 }
}
```

**Why it works:** A TreeMap gives sorted iteration for free.

Verified against 2 test cases: `the cat and The hat and the bat⏎` → `and 2⏎bat 1⏎cat 1⏎hat 1⏎the 3`; `a⏎` → `a 1`

### Exercise 3 · Two-sum

**Ex3.java**

```java
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

public class Ex3 {
 // Two-sum: print the indices (smaller first) of the first pair that adds up to target, or
     `none`.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int target = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 Map<Integer, Integer> seen = new HashMap<>();
 for (int i = 0; i < n; i++) {
 Integer j = seen.get(target - a[i]);
 if (j != null) {
 System.out.println(j + " " + i);
 return;
 }
 seen.put(a[i], i);
 }
 System.out.println("none");
 }
}
```

**Why it works:** A map from value to index finds the complement in O(1).

Verified against 3 test cases: `4 18⏎2 7 11 15⏎` → `1 2`; `3 100⏎1 2 3⏎` → `none`; `4 6⏎3 2 4 3⏎` → `1 2`

## 3.10 Queues, Deques, Stacks and PriorityQueue

### Exercise 1 · Balanced brackets

**Ex1.java**

```java
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 Deque<Character> st = new ArrayDeque<>();
 boolean ok = true;
 for (char c : s.toCharArray()) {
 if ("([{".indexOf(c) >= 0) st.push(c);
 else {
 if (st.isEmpty()) { ok = false; break; }
 char o = st.pop();
 if ("([{".indexOf(o) != ")]}".indexOf(c)) { ok = false; break; }
 }
 }
 System.out.println(ok && st.isEmpty() ? "YES" : "NO");
 }
}
```

**Why it works:** A stack holds the currently open brackets in order.

Verified against 4 test cases: `{[()]}⏎` → `YES`; `([)]⏎` → `NO`; `((⏎` → `NO`

### Exercise 2 · Josephus with a queue

Ex2.java

```java
import java.util.ArrayDeque;
import java.util.Queue;
import java.util.Scanner;

public class Ex2 {
 // Josephus with a queue: n people, every k-th leaves; print the order of leaving.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int k = sc.nextInt();
 Queue<Integer> q = new ArrayDeque<>();
 for (int i = 1; i <= n; i++) q.add(i);
 StringBuilder sb = new StringBuilder();
 while (!q.isEmpty()) {
 for (int i = 1; i < k; i++) q.add(q.poll()); // rotate k-1 people to the back
 sb.append(sb.length() == 0 ? "" : " ").append(q.poll());
 }
 System.out.println(sb);
 }
}
```

**Why it works:** A circle is a queue whose front keeps rotating to the back.

Verified against 2 test cases: `5 2⏎` → `2 4 1 5 3`; `7 3⏎` → `3 6 2 7 5 1 4`

### Exercise 3 · k-th largest

Ex3.java

```java
import java.util.PriorityQueue;
import java.util.Scanner;

public class Ex3 {
 // k-th largest element using a min-heap of size k.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int k = sc.nextInt();
 PriorityQueue<Integer> heap = new PriorityQueue<>();
 for (int i = 0; i < n; i++) {
 heap.add(sc.nextInt());
 if (heap.size() > k) heap.poll(); // drop the smallest: the heap keeps the k largest
 }
 System.out.println(heap.peek());
 }
}
```

**Why it works:** The heap always contains the k largest seen so far; its top is the k-th largest.

Verified against 3 test cases: `6 2⏎3 2 1 5 6 4⏎` → `5`; `3 3⏎9 8 7⏎` → `7`; `1 1⏎4⏎` → `4`

## 3.11 Sorting with Comparable and Comparator

### Exercise 1 · Descending

**Ex1.java**

```java
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 List<Integer> list = new ArrayList<>();
 for (int i = 0; i < n; i++) list.add(sc.nextInt());
 Collections.sort(list, Collections.reverseOrder());
 StringBuilder sb = new StringBuilder();
 for (int x : list) sb.append(sb.length() == 0 ? "" : " ").append(x);
 System.out.println(sb);
 }
}
```

**Why it works:** A reversed natural order.

Verified against 2 test cases: `5⏎3 9 1 7 5⏎` → `9 7 5 3 1`; `1⏎2⏎` → `2`

### Exercise 2 · By length then alphabet

**Ex2.java**

```java
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Scanner;

public class Ex2 {
 // Sort words by length, then alphabetically.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 List<String> words = new ArrayList<>();
 for (int i = 0; i < n; i++) words.add(sc.next());
 words.sort(Comparator.comparingInt(String::length).thenComparing(Comparator.naturalOrder()));
 System.out.println(String.join(" ", words));
 }
}
```

**Why it works:** A two-key comparator.

Verified against 2 test cases: `5⏎pear fig apple kiwi plum⏎` → `fig kiwi pear plum apple`; `2⏎b a⏎` → `a b`

### Exercise 3 · Merge intervals

**Ex3.java**

```java
import java.util.Arrays;
import java.util.Scanner;

public class Ex3 {
 // Intervals: sort by start, then merge overlapping ones.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[][] iv = new int[n][2];
 for (int i = 0; i < n; i++) {
 iv[i][0] = sc.nextInt();
 iv[i][1] = sc.nextInt();
 }
 Arrays.sort(iv, (a, b) -> Integer.compare(a[0], b[0]));
 StringBuilder sb = new StringBuilder();
 int start = iv[0][0], end = iv[0][1];
 for (int i = 1; i < n; i++) {
 if (iv[i][0] <= end) {
 end = Math.max(end, iv[i][1]);
 } else {
 sb.append('[').append(start).append(',').append(end).append("] ");
 start = iv[i][0];
 end = iv[i][1];
 }
 }
 sb.append('[').append(start).append(',').append(end).append(']');
 System.out.println(sb);
 }
}
```

**Why it works:** After sorting by start, overlapping intervals are adjacent: extend the end while the next start ≤ end.

Verified against 3 test cases: `4⏎1 3⏎8 10⏎2 6⏎15 18⏎` → `[1,6] [8,10] [15,18]`; `2⏎1 4⏎4 5⏎` → `[1,5]`; `1⏎3 7⏎` → `[3,7]`

## 3.12 equals(), hashCode() and Immutability

### Exercise 1 · Distinct people

**Ex1.java**

```java
import java.util.HashSet;
import java.util.Objects;
import java.util.Scanner;
import java.util.Set;

public class Ex1 {
 static final class Person {
 final String name;
 final int age;

 Person(String name, int age) {
 this.name = name;
 this.age = age;
 }

 @Override
 public boolean equals(Object o) {
 if (this == o) return true;
 if (!(o instanceof Person)) return false;
 Person p = (Person) o;
 return age == p.age && name.equals(p.name);
 }

 @Override
 public int hashCode() {
 return Objects.hash(name, age);
 }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 Set<Person> set = new HashSet<>();
 for (int i = 0; i < n; i++) set.add(new Person(sc.next(), sc.nextInt()));
 System.out.println(set.size());
 }
}
```

**Why it works:** With equals + hashCode, the HashSet de-duplicates by content.

Verified against 2 test cases: `4⏎Asha 20⏎Ravi 21⏎Asha 20⏎Asha 22⏎` → `3`; `2⏎A 1⏎A 1⏎` → `1`

### Exercise 2 · Immutable money

**Ex2.java**

```
public class Ex2 {
 // An immutable "with" method: returns a NEW object instead of changing this one.
 static final class Money {
 private final long cents;

 Money(long cents) { this.cents = cents; }

 Money plus(Money other) { return new Money(cents + other.cents); }

 long cents() { return cents; }
 }

 public static void main(String[] args) {
 Money a = new Money(250);
 Money b = a.plus(new Money(175));
 System.out.println(a.cents() + " " + b.cents());
 }
}
```

**Why it works:** An immutable class returns new instances.

Verified against 1 test case: `(no input)` → `250 425`

### Exercise 3 · Defensive copying

**Ex3.java**

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Ex3 {
 // An "immutable" class that leaks its list: fix by copying defensively.
 static final class Team {
 private final List<String> members;

 Team(List<String> members) {
 this.members = new ArrayList<>(members); // defensive copy in
 }

 List<String> getMembers() {
 return Collections.unmodifiableList(members); // read-only view out
 }
 }

 public static void main(String[] args) {
 List<String> source = new ArrayList<>(List.of("Asha", "Ravi"));
 Team t = new Team(source);
 source.add("Intruder"); // does not affect the Team
 System.out.println(t.getMembers());
 try {
 t.getMembers().add("Hacker");
 } catch (UnsupportedOperationException e) {
 System.out.println("read-only");
 }
 }
}
```

**Why it works:** Copy in, read-only view out.

Verified against 1 test case: `(no input)` → `[Asha, Ravi]⏎read-only`

# Quiz answer key

## 3.1 Classes, Objects, Fields, Constructors and this

| | Answer | Why |
|---|---|---|
| Q1 | B. a class is a blueprint; an object is an instance of it | new Student(...) builds an object from the Student class. |
| Q2 | 5 | a and b are the same object. |
| Q3 | B. has the same name as the class and no return type | Written like a method without a return type. |
| Q4 | B. the object the method was called on | The current object. |

## 3.2 Encapsulation and Access Modifiers

| | Answer | Why |
|---|---|---|
| Q1 | C. private | private. |
| Q2 | B. so the class can enforce its rules and change internals later | Encapsulation protects invariants. |
| Q3 | B. in the same package | Package-private. |
| Q4 | line 2 | A public mutable field can be set to anything. |

## 3.3 Inheritance, Overriding and Polymorphism

| | Answer | Why |
|---|---|---|
| Q1 | B | Dynamic dispatch uses the actual type. |
| Q2 | B. super | super(...). |
| Q3 | B. Car has an Engine | "has-a". |
| Q4 | B. parent constructor | Parent first. |

## 3.4 Abstract Classes and Interfaces

| | Answer | Why |
|---|---|---|
| Q1 | B. implement several interfaces | One superclass, many interfaces. |
| Q2 | B. has no body and must be implemented by a concrete subclass | It declares a requirement. |
| Q3 | B. Comparable<T> | compareTo defines the order. |
| Q4 | B. an interface | An interface. |

## 3.5 static, final and Initialisation Order

| | Answer | Why |
|---|---|---|
| Q1 | B. the JVM calls it before any object exists | No object is needed to start the program. |
| Q2 | B. one per class | A single shared copy. |
| Q3 | 3 | The static counter is shared. |
| Q4 | A. cannot be re-assigned but elements can change | final protects the reference. |

### 3.6 Packages and Imports

| | Answer | Why |
|---|---|---|
| Q1 | C. java.lang | java.lang. |
| Q2 | B. first in the file | It is the first statement (comments aside). |
| Q3 | B. no — sub-packages are separate | Wildcards cover one package only. |
| Q4 | B. com/shop/billing/ | Dots become folder separators. |

### 3.7 Exceptions: try, catch, finally, throw and throws

| | Answer | Why |
|---|---|---|
| Q1 | C. IOException | IOException must be handled or declared. |
| Q2 | ACD | B is skipped; the catch and finally run. |
| Q3 | B. declares that the method may throw a checked exception | It warns callers. |
| Q4 | B. AutoCloseable | close() is called automatically. |

### 3.8 Generics and Lists: ArrayList and LinkedList

| | Answer | Why |
|---|---|---|
| Q1 | [5, 7] | remove(int) removes index 1 (the 6). |
| Q2 | B. ArrayList | ArrayList is O(1). |
| Q3 | B. generics need reference types; use Integer | Type arguments must be objects. |
| Q4 | false | Outside the −128..127 cache, boxed values are different objects. |

### 3.9 Sets and Maps: HashSet, TreeSet, HashMap, TreeMap

| | Answer | Why |
|---|---|---|
| Q1 | B. false | false — the set is unchanged. |
| Q2 | B. TreeMap | TreeMap, in O(log n). |
| Q3 | A. O(1) | Hashing jumps to the bucket. |
| Q4 | 2 | merge sums existing and new values. |

### 3.10 Queues, Deques, Stacks and PriorityQueue

| | Answer | Why |
|---|---|---|
| Q1 | 3 2 | LIFO: 3 comes out first; 2 is now on top. |
| Q2 | B. queue | FIFO explores level by level. |
| Q3 | C. the smallest | Natural ordering is a min-heap. |
| Q4 | B. O(log n) | Sift up through the heap. |

### 3.11 Sorting with Comparable and Comparator

| | Answer | Why |
|---|---|---|
| Q1 | A. a goes before b | Negative: first argument first. |

| | Answer | Why |
|---|---|---|
| Q2 | B. integer overflow | The subtraction can overflow. |
| Q3 | B. keeps equal elements in their original order | List.sort and Collections.sort are stable. |
| Q4 | B. O(n log n) | Merge-sort-like TimSort. |

## 3.12 equals(), hashCode() and Immutability

| | Answer | Why |
|---|---|---|
| Q1 | A. a.hashCode() == b.hashCode() must hold | That is the contract. |
| Q2 | B. private final fields and no setters | State cannot change after construction. |
| Q3 | 2 | Almost certainly: different hash codes put them in different buckets, so equals is never consulted. |
| Q4 | C. String | String never changes. |

# Part 4 · Data Structures & Algorithm Foundations

## Exercise solutions

### 4.1 Data Structures Overview: Arrays, Linked Lists, Stacks, Queues, Hash Tables

#### Exercise 1 · Reverse a linked list

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 static class Node {
 int v;
 Node next;
 Node(int v) { this.v = v; }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 Node head = null;
 Node tail = null;
 for (int i = 0; i < n; i++) {
 Node node = new Node(sc.nextInt());
 if (head == null) head = tail = node;
 else {
 tail.next = node;
 tail = node;
 }
 }
 // reverse
 Node prev = null;
 for (Node c = head; c != null; ) {
 Node nx = c.next;
 c.next = prev;
 prev = c;
 c = nx;
 }
 StringBuilder sb = new StringBuilder();
 for (Node c = prev; c != null; c = c.next) sb.append(sb.length() == 0 ? "" : " ").append(c.v);
 System.out.println(sb);
 }
}
```

**Why it works:** One pass flipping each link.

Verified against 2 test cases: `5⏎1 2 3 4 5⏎` → `5 4 3 2 1`; `1⏎9⏎` → `9`

#### Exercise 2 · Palindromic list

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 // Is a linked list of n numbers a palindrome? Use an array to hold the values (simple version).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 boolean ok = true;
 for (int i = 0, j = n - 1; i < j; i++, j--) if (a[i] != a[j]) ok = false;
 System.out.println(ok ? "YES" : "NO");
 }
}
```

**Why it works:** Compare the i-th from the front with the i-th from the back.

Verified against 3 test cases: `5⏎1 2 3 2 1⏎` → `YES`; `3⏎1 2 3⏎` → `NO`; `1⏎7⏎` → `YES`

### Exercise 3 · Circular queue

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 // A circular queue (ring buffer) of capacity c. Commands: enq x -> ok/full ; deq -> value/empty.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int cap = sc.nextInt();
 int n = sc.nextInt();
 int[] data = new int[cap];
 int head = 0, size = 0;
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) {
 String op = sc.next();
 if (op.equals("enq")) {
 int x = sc.nextInt();
 if (size == cap) sb.append("full ");
 else {
 data[(head + size) % cap] = x;
 size++;
 sb.append("ok ");
 }
 } else {
 if (size == 0) sb.append("empty ");
 else {
 sb.append(data[head]).append(' ');
 head = (head + 1) % cap;
 size--;
 }
 }
 }
 System.out.println(sb.toString().trim());
 }
}
```

**Why it works:** Modulo arithmetic turns the array into a ring.

Verified against 2 test cases: `2 6⏎enq 1⏎enq 2⏎enq 3⏎deq⏎enq 3⏎deq⏎` → `ok ok full 1 ok 2`; `1 2⏎deq⏎enq 5⏎` → `empty ok`

## 4.2 Trees, Graphs and Heaps

### Exercise 1 · Height of a BST

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 static class T { int k; T l, r; T(int k) { this.k = k; } }

 static T ins(T t, int k) {
 if (t == null) return new T(k);
 if (k < t.k) t.l = ins(t.l, k);
 else if (k > t.k) t.r = ins(t.r, k);
 return t;
 }

 static int height(T t) { return t == null ? 0 : 1 + Math.max(height(t.l), height(t.r)); }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 T root = null;
 for (int i = 0; i < n; i++) root = ins(root, sc.nextInt());
 System.out.println(height(root));
 }
}
```

**Why it works:** Recursive definition mirrors the tree.

Verified against 3 test cases: `7⏎50 30 70 20 40 60 80⏎` → `3`; `5⏎1 2 3 4 5⏎` → `5`; `1⏎7⏎` → `1`

### Exercise 2 · Connected components

**Ex2.java**

```java
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.List;
import java.util.Queue;
import java.util.Scanner;

public class Ex2 {
 // Number of connected components of an undirected graph.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int m = sc.nextInt();
 List<List<Integer>> adj = new ArrayList<>();
 for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
 for (int i = 0; i < m; i++) {
 int a = sc.nextInt(), b = sc.nextInt();
 adj.get(a).add(b);
 adj.get(b).add(a);
 }
 boolean[] seen = new boolean[n];
 int components = 0;
 for (int s = 0; s < n; s++) {
 if (seen[s]) continue;
 components++;
 Queue<Integer> q = new ArrayDeque<>();
 q.add(s);
 seen[s] = true;
 while (!q.isEmpty()) {
 int u = q.poll();
 for (int v : adj.get(u)) {
 if (!seen[v]) {
 seen[v] = true;
 q.add(v);
 }
 }
 }
 }
 System.out.println(components);
 }
}
```

**Why it works:** Each BFS/DFS discovers exactly one component.

Verified against 3 test cases: `6 3⏎0 1⏎1 2⏎3 4⏎` → `3`; `3 0⏎` → `3`; `2 1⏎0 1⏎` → `1`

### Exercise 3 · Shortest path length

**Ex3.java**

```java
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.Queue;
import java.util.Scanner;

public class Ex3 {
 // Fewest edges from vertex 0 to vertex n-1 in an undirected graph, or -1.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int m = sc.nextInt();
 List<List<Integer>> adj = new ArrayList<>();
 for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
 for (int i = 0; i < m; i++) {
 int a = sc.nextInt(), b = sc.nextInt();
 adj.get(a).add(b);
 adj.get(b).add(a);
 }
 int[] dist = new int[n];
 Arrays.fill(dist, -1);
 Queue<Integer> q = new ArrayDeque<>();
 q.add(0);
 dist[0] = 0;
 while (!q.isEmpty()) {
 int u = q.poll();
 for (int v : adj.get(u)) {
 if (dist[v] == -1) {
 dist[v] = dist[u] + 1;
 q.add(v);
 }
 }
 }
 System.out.println(dist[n - 1]);
 }
}
```

**Why it works:** BFS reaches each vertex first by a shortest path.

Verified against 3 test cases: `5 5⏎0 1⏎0 2⏎1 3⏎2 3⏎3 4⏎` → `3`; `3 1⏎0 1⏎` → `-1`; `1 0⏎` → `0`

## 4.3 Searching: Linear and Binary Search

### Exercise 1 · First occurrence

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int target = sc.nextInt();
 int found = -1;
 for (int i = 0; i < n; i++) {
 int x = sc.nextInt();
 if (x == target && found == -1) found = i;
 }
 System.out.println(found);
 }
}
```

**Why it works:** Linear search remembers the first match.

Verified against 2 test cases: `5 7⏎4 9 7 1 7⏎` → `2`; `3 5⏎1 2 3⏎` → `-1`

### Exercise 2 · Binary search

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 // Binary search in a sorted array; print the index or -1.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int target = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 int lo = 0, hi = n - 1, ans = -1;
 while (lo <= hi) {
 int mid = lo + (hi - lo) / 2;
 if (a[mid] == target) { ans = mid; break; }
 if (a[mid] < target) lo = mid + 1;
 else hi = mid - 1;
 }
 System.out.println(ans);
 }
}
```

**Why it works:** O(log n) with the standard invariant.

Verified against 3 test cases: `6 9⏎1 3 5 7 9 11⏎` → `4`; `3 4⏎1 2 3⏎` → `-1`; `1 5⏎5⏎` → `0`

### Exercise 3 · Count occurrences

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 // Count how many times target occurs in a sorted array in O(log n): upper bound − lower bound.
 static int lower(int[] a, int t) {
 int lo = 0, hi = a.length;
 while (lo < hi) {
 int mid = lo + (hi − lo) / 2;
 if (a[mid] < t) lo = mid + 1;
 else hi = mid;
 }
 return lo;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int target = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 System.out.println(lower(a, target + 1) − lower(a, target));
 }
}
```

**Why it works:** Two lower-bound searches bracket the run of equal values.

Verified against 3 test cases: `8 2⏎1 2 2 2 2 3 5 9⏎` → `4`; `3 7⏎1 2 3⏎` → `0`; `4 1⏎1 1 1 1⏎` → `4`

## 4.4 Elementary Sorting: Bubble, Insertion, Selection; Stability

### Exercise 1 · Bubble sort with swap count

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 // Bubble sort; print the sorted array and the number of swaps (= number of inversions).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 int swaps = 0;
 for (int pass = 0; pass < n − 1; pass++) {
 boolean any = false;
 for (int i = 0; i < n − 1 − pass; i++) {
 if (a[i] > a[i + 1]) {
 int t = a[i]; a[i] = a[i + 1]; a[i + 1] = t;
 swaps++;
 any = true;
 }
 }
 if (!any) break;
 }
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) sb.append(i == 0 ? "" : " ").append(a[i]);
 System.out.println(sb);
 System.out.println(swaps);
 }
}
```

**Why it works:** Each swap removes exactly one inversion.

Verified against 3 test cases: `5⏎4 3 2 1 5⏎` → `1 2 3 4 5⏎`6; `3⏎1 2 3⏎` → `1 2 3⏎`0; `2⏎2 1⏎` → `1 2⏎1`

### Exercise 2 · Insertion steps

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 // Insertion sort: print the array after every insertion step i = 1..n-1.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 for (int i = 1; i < n; i++) {
 int key = a[i];
 int j = i - 1;
 while (j >= 0 && a[j] > key) {
 a[j + 1] = a[j];
 j--;
 }
 a[j + 1] = key;
 StringBuilder sb = new StringBuilder();
 for (int k = 0; k < n; k++) sb.append(k == 0 ? "" : " ").append(a[k]);
 System.out.println(sb);
 }
 }
}
```

**Why it works:** The sorted prefix grows by one element per step.

Verified against 2 test cases: `4⏎5 2 4 1⏎` → `2 5 4 1⏎`2 4 5 1⏎1 2 4 5; `2⏎1 2⏎` → `1 2`

### Exercise 3 · Stable by key

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 // Is the sort stable? Input: n pairs "key label". Sort by key with insertion sort and print the
     labels.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] key = new int[n];
 String[] label = new String[n];
 for (int i = 0; i < n; i++) {
 key[i] = sc.nextInt();
 label[i] = sc.next();
 }
 for (int i = 1; i < n; i++) {
 int k = key[i];
 String l = label[i];
 int j = i - 1;
 while (j >= 0 && key[j] > k) {
 key[j + 1] = key[j];
 label[j + 1] = label[j];
 j--;
 }
 key[j + 1] = k;
 label[j + 1] = l;
 }
 System.out.println(String.join(" ", label));
 }
}
```

**Why it works:** Equal keys never pass each other, so labels with equal keys keep input order.

Verified against 2 test cases: `5⏎2 a⏎1 b⏎2 c⏎1 d⏎2 e⏎` → `b d a c e`; `2⏎1 x⏎1 y⏎` → `x y`

## 4.5 Merge Sort and Divide and Conquer

### Exercise 1 · Merge sort

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 static void sort(int[] a, int lo, int hi, int[] t) {
 if (hi - lo < 2) return;
 int mid = (lo + hi) >>> 1;
 sort(a, lo, mid, t);
 sort(a, mid, hi, t);
 int i = lo, j = mid, k = lo;
 while (i < mid && j < hi) t[k++] = a[i] <= a[j] ? a[i++] : a[j++];
 while (i < mid) t[k++] = a[i++];
 while (j < hi) t[k++] = a[j++];
 System.arraycopy(t, lo, a, lo, hi - lo);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 sort(a, 0, n, new int[n]);
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) sb.append(i == 0 ? "" : " ").append(a[i]);
 System.out.println(sb);
 }
}
```

**Why it works:** Standard top-down merge sort.

Verified against 3 test cases: `7⏎38 27 43 3 9 82 10⏎` → `3 9 10 27 38 43 82`; `1⏎5⏎` → `5`; `4⏎2 2 1 1⏎` → `1 1 2 2`

### Exercise 2 · Merge two sorted arrays

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 // Merge two sorted arrays into one sorted array.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int m = sc.nextInt();
 int[] a = new int[n], b = new int[m];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 for (int i = 0; i < m; i++) b[i] = sc.nextInt();
 StringBuilder sb = new StringBuilder();
 int i = 0, j = 0;
 while (i < n || j < m) {
 int v = (j >= m || (i < n && a[i] <= b[j])) ? a[i++] : b[j++];
 sb.append(sb.length() == 0 ? "" : " ").append(v);
 }
 System.out.println(sb);
 }
}
```

**Why it works:** The merge step on its own: O(n + m).

Verified against 2 test cases: `3 4⏎1 4 9⏎2 3 10 11⏎` → `1 2 3 4 9 10 11`; `1 1⏎5⏎5⏎` → `5 5`

### Exercise 3 · Count inversions

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 static long count(int[] a, int lo, int hi, int[] t) {
 if (hi - lo < 2) return 0;
 int mid = (lo + hi) >>> 1;
 long inv = count(a, lo, mid, t) + count(a, mid, hi, t);
 int i = lo, j = mid, k = lo;
 while (i < mid && j < hi) {
 if (a[i] <= a[j]) t[k++] = a[i++];
 else {
 inv += mid - i;
 t[k++] = a[j++];
 }
 }
 while (i < mid) t[k++] = a[i++];
 while (j < hi) t[k++] = a[j++];
 System.arraycopy(t, lo, a, lo, hi - lo);
 return inv;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 System.out.println(count(a, 0, n, new int[n]));
 }
}
```

**Why it works:** Each right-half element that jumps ahead passes all remaining left-half elements.

Verified against 3 test cases: `5⏎2 4 1 3 5⏎` → `3`; `3⏎3 2 1⏎` → `3`; `4⏎1 2 3 4⏎` → `0`

## 4.6 Iteration versus Recursion

### Exercise 1 · gcd both ways

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 // gcd both ways; print iterative then recursive result
 static int gcdLoop(int a, int b) {
 while (b != 0) {
 int t = a % b;
 a = b;
 b = t;
 }
 return a;
 }

 static int gcdRec(int a, int b) {
 return b == 0 ? a : gcdRec(b, a % b);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int a = sc.nextInt(), b = sc.nextInt();
 System.out.println(gcdLoop(a, b) + " " + gcdRec(a, b));
 }
}
```

**Why it works:** while (b != 0) vs return gcd(b, a % b).

Verified against 2 test cases: `84 36⏎` → `12 12`; `7 13⏎` → `1 1`

### Exercise 2 · Climbing stairs

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 // Climbing stairs: ways to climb n steps taking 1 or 2 at a time (Fibonacci in disguise),
     iteratively.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long a = 1, b = 1;
 for (int i = 2; i <= n; i++) {
 long next = a + b;
 a = b;
 b = next;
 }
 System.out.println(b);
 }
}
```

**Why it works:** Bottom-up with two variables.

Verified against 3 test cases: `10⏎` → `89`; `1⏎` → `1`; `2⏎` → `2`

### Exercise 3 · Reverse a string

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 // Binary-search count of steps both ways would be the same; here: convert the recursive
 // "reverse a string" into a loop and print both results.
 static String revRec(String s) {
 return s.isEmpty() ? "" : revRec(s.substring(1)) + s.charAt(0);
 }

 static String revLoop(String s) {
 StringBuilder sb = new StringBuilder();
 for (int i = s.length() - 1; i >= 0; i--) sb.append(s.charAt(i));
 return sb.toString();
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 System.out.println(revRec(s) + " " + revLoop(s));
 }
}
```

**Why it works:** Same result; the recursion creates a new string at each level.

Verified against 2 test cases: `stressed⏎` → `desserts desserts`; `a⏎` → `a a`

## 4.7 Time Complexity and Big-O Notation

### Exercise 1 · Count the triangle

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 // Count how many times the inner statement runs for a double loop i in 1..n, j in 1..i.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long count = 0;
 for (int i = 1; i <= n; i++)
 for (int j = 1; j <= i; j++) count++;
 System.out.println(count);
 }
}
```

**Why it works:** Triangular numbers: O(n²).

Verified against 3 test cases: `5⏎` → `15`; `1⏎` → `1`; `100⏎` → `5050`

### Exercise 2 · Passes of a halving loop

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 // Count the passes of: while (n > 1) n /= 2; (the floor of log2 n)
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long n = sc.nextLong();
 int passes = 0;
 while (n > 1) {
 n /= 2;
 passes++;
 }
 System.out.println(passes);
 }
}
```

**Why it works:** Each pass halves n; $\lfloor \log_2 1000 \rfloor = 9$.

Verified against 3 test cases: `1000⏎` → `9`; `1⏎` → `0`; `1024⏎` → `10`

### Exercise 3 · The crossover point

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 // Smallest n for which n^2 exceeds 10 * n * log2(n) ... i.e. when does a quadratic algorithm
     become
 // slower than an n log n one that does 10x more work per step? Print that n (n >= 2).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int factor = sc.nextInt();
 int n = 2;
 while (!((double) n * n > factor * n * (Math.log(n) / Math.log(2)))) n++;
 System.out.println(n);
 }
}
```

**Why it works:** $n^2 > \text{factor} \cdot n \cdot \log_2 n \Leftrightarrow n > \text{factor} \cdot \log_2 n$; for factor 10 this first holds at n = 59.

Verified against 2 test cases: `10⏎` → `59`; `2⏎` → `5`

## 4.8 Algorithm Design Paradigms

### Exercise 1 · Activity selection (greedy)

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 // Greedy activity selection: maximum number of non-overlapping intervals (sort by END time).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[][] iv = new int[n][2];
 for (int i = 0; i < n; i++) {
 iv[i][0] = sc.nextInt();
 iv[i][1] = sc.nextInt();
 }
 java.util.Arrays.sort(iv, (a, b) -> Integer.compare(a[1], b[1]));
 int count = 0, lastEnd = Integer.MIN_VALUE;
 for (int[] x : iv) {
 if (x[0] >= lastEnd) {
 count++;
 lastEnd = x[1];
 }
 }
 System.out.println(count);
 }
}
```

**Why it works:** The earliest finish leaves the most room — a provable greedy choice.

Verified against 3 test cases: `5⏎1 3⏎2 4⏎3 5⏎0 6⏎5 7⏎` → `3`; `1⏎0 1⏎` → `1`; `3⏎1 5⏎2 3⏎3 4⏎` → `2`

### Exercise 2 · Fewest coins (DP)

**Ex2.java**

```java
import java.util.Arrays;
import java.util.Scanner;

public class Ex2 {
 // Fewest coins to make the amount (any coin system), or -1.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int k = sc.nextInt();
 int[] coins = new int[k];
 for (int i = 0; i < k; i++) coins[i] = sc.nextInt();
 int amount = sc.nextInt();
 int INF = Integer.MAX_VALUE;
 int[] best = new int[amount + 1];
 Arrays.fill(best, INF);
 best[0] = 0;
 for (int x = 1; x <= amount; x++)
 for (int c : coins)
 if (c <= x && best[x - c] != INF) best[x] = Math.min(best[x], best[x - c] + 1);
 System.out.println(best[amount] == INF ? -1 : best[amount]);
 }
}
```

**Why it works:** Bottom-up over amounts 1..amount.

Verified against 3 test cases: `3⏎4 3 1⏎6⏎` → `2`; `1⏎2⏎3⏎` → `-1`; `3⏎25 10 1⏎30⏎` → `3`

### Exercise 3 · No two adjacent ones (backtracking)

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 // Backtracking: print all binary strings of length n with no two consecutive 1s, in order.
 static StringBuilder out = new StringBuilder();

 static void gen(int n, String s, boolean lastOne) {
 if (s.length() == n) {
 out.append(s).append('\n');
 return;
 }
 gen(n, s + "0", false);
 if (!lastOne) gen(n, s + "1", true);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 gen(sc.nextInt(), "", false);
 System.out.print(out);
 }
}
```

**Why it works:** The constraint is checked as the string grows, pruning invalid branches immediately.

Verified against 3 test cases: `3⏎` → `000⏎001⏎010⏎100⏎101`; `1⏎` → `0⏎1`; `2⏎` → `00⏎01⏎10`

## 4.9 Analysing Loops: Counting Iterations

### Exercise 1 · Tripling outer loop

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 // How many times does "count++" execute? for (i = 1; i <= n; i *= 3) for (j = 0; j < n; j++)
     count++;
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long count = 0;
 for (long i = 1; i <= n; i *= 3)
 for (int j = 0; j < n; j++) count++;
 System.out.println(count);
 }
}
```

**Why it works:** 4 outer iterations (1, 3, 9, 27) × 30 = 120: O(n log n).

Verified against 3 test cases: `30⏎` → `120`; `1⏎` → `1`; `100⏎` → `500`

### Exercise 2 · Half-linear times log

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 // for (i = n/2; i >= 1; i--) for (j = 1; j <= n; j *= 2) count++;
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long count = 0;
 for (int i = n / 2; i >= 1; i--)
 for (long j = 1; j <= n; j *= 2) count++;
 System.out.println(count);
 }
}
```

**Why it works:** 8 × 5 = 40: O(n log n).

Verified against 3 test cases: `16⏎` → `40`; `2⏎` → `2`; `100⏎` → `350`

### Exercise 3 · Measure the growth ratio

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 // Measure the growth: print count(2n) / count(n) rounded to 1 decimal for a double loop where j
     runs i times.
 static long count(int n) {
 long c = 0;
 for (int i = 1; i <= n; i++)
 for (int j = 1; j <= i; j++) c++;
 return c;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 System.out.printf("%.1f%n", (double) count(2 * n) / count(n));
 }
}
```

**Why it works:** Doubling n quadruples n(n+1)/2 (almost exactly), the signature of O(n²).

Verified against 2 test cases: `500⏎` → `4.0`; `1000⏎` → `4.0`

## 4.10 Analysing Recursion: Recurrence Relations

### Exercise 1 · Count the calls

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 static long calls;

 static int f(int n) {
 calls++;
 if (n <= 1) return 1;
 return f(n - 1) + f(n - 1); // two calls, not one!
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 f(sc.nextInt());
 System.out.println(calls);
 }
}
```

**Why it works:** A complete binary tree of depth n−1.

Verified against 3 test cases: `10⏎` → `1023`; `1⏎` → `1`; `5⏎` → `31`

### Exercise 2 · Two half calls

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 static long calls;

 static int g(int n) {
 calls++;
 if (n <= 1) return 0;
 return g(n / 2) + g(n / 2); // two calls on HALF the size: T(n) = 2T(n/2) + 1
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 g(sc.nextInt());
 System.out.println(calls);
 }
}
```

**Why it works:** T(n) = 2T(n/2) + 1 = O(n): a full binary tree with n leaves.

Verified against 3 test cases: `16⏎` → `31`; `1⏎` → `1`; `8⏎` → `15`

### Exercise 3 · Karatsuba work

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 static long work;

 // T(n) = 3T(n/2) + n (Karatsuba shape): count the "n" work terms for n a power of two.
 static void t(int n) {
 if (n <= 1) {
 work += 1;
 return;
 }
 work += n;
 t(n / 2);
 t(n / 2);
 t(n / 2);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 t(sc.nextInt());
 System.out.println(work);
 }
}
```

**Why it works:** Σ n(3/2)^k + 3^(log n) = Θ(n^1.585).

Verified against 3 test cases: `8⏎` → `65`; `2⏎` → `5`; `1⏎` → `1`

## 4.11 Space Complexity and the Call Stack

### Exercise 1 · Maximum recursion depth

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 static int depth, maxDepth;

 static long f(int n) {
 depth++;
 maxDepth = Math.max(maxDepth, depth);
 long r = n <= 1 ? 1 : f(n - 1) + 1;
 depth--;
 return r;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 f(sc.nextInt());
 System.out.println(maxDepth);
 }
}
```

**Why it works:** A chain of n calls is all active at the base case.

Verified against 3 test cases: `50⏎` → `50`; `1⏎` → `1`; `200⏎` → `200`

### Exercise 2 · Fibonacci in O(1) space

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 // Space-efficient: nth Fibonacci with O(1) space (two variables) instead of an array of n values.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long a = 0, b = 1;
 for (int i = 0; i < n; i++) {
 long c = a + b;
 a = b;
 b = c;
 }
 System.out.println(a);
 }
}
```

**Why it works:** The loop only needs F(i−1) and F(i).

Verified against 4 test cases: `50⏎` → `12586269025`; `0⏎` → `0`; `1⏎` → `1`

### Exercise 3 · Duplicates with O(1) extra space

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 // Does the array contain a duplicate? O(n log n) time but only O(1) extra space apart from sorting in place.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 java.util.Arrays.sort(a);
 boolean dup = false;
 for (int i = 1; i < n; i++) if (a[i] == a[i - 1]) dup = true;
 System.out.println(dup ? "YES" : "NO");
 }
}
```

**Why it works:** Trade time (O(n log n) sort) for space (no extra structure).

Verified against 3 test cases: `5⏎4 1 7 1 9⏎` → `YES`; `3⏎1 2 3⏎` → `NO`; `1⏎5⏎` → `NO`

## 4.12 Best, Average and Worst Case

### Exercise 1 · Comparisons in a linear search

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 // Comparisons a linear search makes for the target (n if absent).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int target = sc.nextInt();
 int comparisons = 0;
 boolean found = false;
 for (int i = 0; i < n; i++) {
 int x = sc.nextInt();
 if (!found) {
 comparisons++;
 if (x == target) found = true;
 }
 }
 System.out.println(comparisons);
 }
}
```

**Why it works:** The count equals index + 1 for a hit and n for a miss.

Verified against 3 test cases: `5 7⏎4 9 7 1 7⏎` → `3`; `3 9⏎1 2 3⏎` → `3`; `2 1⏎1 2⏎` → `1`

### Exercise 2 · Average successful search

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 // Average comparisons of a successful linear search over n elements when every position is
     equally likely.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 System.out.printf("%.1f%n", (n + 1) / 2.0);
 }
}
```

**Why it works:** Average of 1..n.

Verified against 3 test cases: `9⏎` → `5.0`; `1⏎` → `1.0`; `10⏎` → `5.5`

### Exercise 3 · Insertion sort comparisons

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 // Insertion sort comparisons for the given array (the lesson's counting rule).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 long c = 0;
 for (int i = 1; i < n; i++) {
 int key = a[i], j = i - 1;
 while (j >= 0) {
 c++;
 if (a[j] <= key) break;
 a[j + 1] = a[j];
 j--;
 }
 a[j + 1] = key;
 }
 System.out.println(c);
 }
}
```

**Why it works:** Each element is compared with every larger predecessor plus one stopping comparison (when it exists).

Verified against 3 test cases: `5⏎5 4 3 2 1⏎` → `10`; `5⏎1 2 3 4 5⏎` → `4`; `1⏎1⏎` → `0`

# Quiz answer key

## 4.1 Data Structures Overview: Arrays, Linked Lists, Stacks, Queues, Hash Tables

| | Answer | Why |
|---|---|---|
| Q1 | array index lookup → O(1); linked list lookup by position → O(n); hash table lookup (average) → O(1); sorted tree lookup → O(log n) | Arrays jump by arithmetic; lists walk; hashing jumps by hash; trees halve. |
| Q2 | B. stack | The most recent action is undone first: LIFO. |
| Q3 | A. O(1) | Re-wire two references. |
| Q4 | B. queue | FIFO explores by levels. |

## 4.2 Trees, Graphs and Heaps

| | Answer | Why |
|---|---|---|
| Q1 | A. sorted keys | left, node, right. |
| Q2 | B. queue | FIFO queue. |
| Q3 | B. 7 and 8 | 2·3+1 = 7 and 2·3+2 = 8. |
| Q4 | C. $O(V^2)$ | One cell per vertex pair. |

## 4.3 Searching: Linear and Binary Search

| | Answer | Why |
|---|---|---|
| Q1 | B. 20 | $\lceil \log_2 10^6 \rceil = 20$. |
| Q2 | B. sorted | Sorted order lets us discard a half. |
| Q3 | first mid = 3; a[first mid] = 7 | mid = 3 and a[3] = 7: found at the first probe. |
| Q4 | B. O(log n) | The range halves each step. |

## 4.4 Elementary Sorting: Bubble, Insertion, Selection; Stability

| | Answer | Why |
|---|---|---|
| Q1 | C. selection | Selection sort swaps across equal elements. |
| Q2 | A. O(n) | One comparison per element. |
| Q3 | 3 | 3 inversions → 3 swaps. |
| Q4 | B. O(1) extra memory | Rearranges within the input. |

## 4.5 Merge Sort and Divide and Conquer

| | Answer | Why |
|---|---|---|
| Q1 | B. O(n log n) | Always n log n. |
| Q2 | C. O(n) | The temporary array. |
| Q3 | B. divide, conquer, combine | Divide, conquer (recurse), combine. |
| Q4 | 2 | 3 > 1 and 3 > 2; 1 < 2. |

## 4.6 Iteration versus Recursion

| | Answer | Why |
|---|---|---|
| Q1 | B. loop | Recursion needs a stack frame per level. |
| Q2 | B. $O(2^n)$ to O(n) | Each value is computed once. |
| Q3 | B. all subsets of a set | Branching choices form a tree. |
| Q4 | B. StackOverflowError | The call stack is exhausted. |

## 4.7 Time Complexity and Big-O Notation

| | Answer | Why |
|---|---|---|
| Q1 | binary search → O(log n); find max in unsorted array → O(n); merge sort → O(n log n); bubble sort → $O(n^2)$; all subsets → $O(2^n)$; all permutations → O(n!) | The lecture's Big-O cheat sheet. |
| Q2 | B. $O(n^2)$ | The $n^2$ term dominates ($n^3$ is true but loose). |
| Q3 | C. Big-Θ | Θ is both an upper and a lower bound. |
| Q4 | B. $2^n$ | Exponential beats every polynomial. |
| Q5 | A. O(1) | Constant: address arithmetic. |

## 4.8 Algorithm Design Paradigms

| | Answer | Why |
|---|---|---|
| Q1 | N-Queens → backtracking; merge sort → divide and conquer; shortest string of given weight → greedy; grid paths count → dynamic programming | See the table in the lesson. |
| Q2 | 2 | 25 + 5. |
| Q3 | B. the best local choice leads to a global optimum | Provability is the key. |
| Q4 | B. dynamic programming | Store and reuse answers. |

## 4.9 Analysing Loops: Counting Iterations

| | Answer | Why |
|---|---|---|
| Q1 | B. $\log_2 n$ times | Geometric growth. |
| Q2 | C. $O(n^2)$ | n(n+1)/2. |
| Q3 | B. O(n log n) | Harmonic series: n·H(n). |
| Q4 | B. $O(n\sqrt{n})$ | n outer × √n inner. |
| Q5 | B. $\log_{10} n$ times (the number of digits) | One iteration per digit. |

## 4.10 Analysing Recursion: Recurrence Relations

| | Answer | Why |
|---|---|---|
| Q1 | T(n) = T(n−1) + 1 → O(n); T(n) = T(n/2) + 1 → O(log n); T(n) = 2T(n/2) + n → O(n log n); T(n) = 2T(n−1) + 1 → $O(2^n)$ | See the table. |
| Q2 | C. O(n^1.585) | Leaves dominate: $n^{\log_2 3}$. |
| Q3 | B. log n | Halving. |
| Q4 | C. $1.618^n$ calls | Exponential in the golden ratio. |

## 4.11 Space Complexity and the Call Stack

| | Answer | Why |
|---|---|---|
| Q1 | int add(int n) recursive sum → O(n); loop with sum += pairSum(i, i+1) → O(1); new int[n] filled in a loop → O(n); int sum; for (i<n) sum += i → O(1); recursive fibonacci(n) (stack) → O(n); linked list with n inserts → O(n) | The six "predict the space complexity" slides. |
| Q2 | B. the extra space used besides the input | Temporary memory. |
| Q3 | B. O(log n) | Depth $\log_2 n$. |
| Q4 | B. 4 MB | $10^6$ × 4 bytes. |

## 4.12 Best, Average and Worst Case

| | Answer | Why |
|---|---|---|
| Q1 | B. reverse sorted | Every element moves to the front. |
| Q2 | B. already sorted data | Unbalanced partitions. |

| | Answer | Why |
|---|---|---|
| Q3 | C. worst | The worst case bounds all inputs. |
| Q4 | B. all O(n log n) | It always splits evenly. |

# Part 5 · Archive Algorithm Mastery

## Exercise solutions

### 5.1 Euclid's Algorithm (GCD)

#### Exercise 1 · Reduce a fraction

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 static int gcd(int a, int b) {
 while (b != 0) {
 int r = a % b;
 a = b;
 b = r;
 }
 return a;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int numerator = sc.nextInt();
 int denominator = sc.nextInt();
 int g = gcd(numerator, denominator);
 System.out.println((numerator / g) + "/" + (denominator / g));
 }
}
```

**Why it works:** Reduce by the gcd: dividing numerator and denominator by their greatest common divisor leaves no common factor, so the fraction is in lowest terms.

Verified against 3 test cases: `12 18⏎` → `2/3`; `7 5⏎` → `7/5`; `100 25⏎` → `4/1`

#### Exercise 2 · LCM of a list

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 static long gcd(long a, long b) {
 while (b != 0) {
 long r = a % b;
 a = b;
 b = r;
 }
 return a;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long lcm = 1;
 for (int i = 0; i < n; i++) {
 long x = sc.nextLong();
 // divide first, then multiply: keeps the intermediate value small
 lcm = lcm / gcd(lcm, x) * x;
 }
 System.out.println(lcm);
 }
}
```

**Why it works:** The LCM of a list is built one number at a time with lcm(x, y) = x / gcd(x, y) · y. Dividing first keeps intermediate values as small as possible.

Verified against 3 test cases: `3⏎4 6 8⏎` → `24`; `2⏎7 13⏎` → `91`; `4⏎1 2 3 4⏎` → `12`

### Exercise 3 · Modular inverse

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 // returns {g, x, y} with a*x + b*y = g = gcd(a, b)
 static long[] extendedGcd(long a, long b) {
 if (b == 0) {
 return new long[] {a, 1, 0};
 }
 long[] next = extendedGcd(b, a % b);
 long g = next[0];
 long x = next[2];
 long y = next[1] - (a / b) * next[2];
 return new long[] {g, x, y};
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long a = sc.nextLong();
 long m = sc.nextLong();
 long[] r = extendedGcd(a, m);
 if (r[0] != 1) {
 System.out.println(-1); // a and m share a factor: no inverse exists
 } else {
 long inverse = ((r[1] % m) + m) % m; // shift a possibly-negative x into [0, m-1]
 System.out.println(inverse);
 }
 }
}
```

**Why it works:** Extended Euclid carries two extra numbers back up the recursion so that a·x + b·y = gcd(a, b) holds at every level. When the gcd is 1, a·x ≡ 1 (mod m), so x is the inverse.

Verified against 3 test cases: `3 11⏎` → `4`; `10 17⏎` → `12`; `6 9⏎` → `-1`

## 5.2 Euler's Phi (Totient) Function

### Exercise 1 · List the coprime numbers

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 static int gcd(int a, int b) {
 while (b != 0) {
 int r = a % b;
 a = b;
 b = r;
 }
 return a;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 StringBuilder sb = new StringBuilder();
 for (int k = 1; k <= n; k++) {
 if (gcd(k, n) == 1) {
 sb.append(k).append(' ');
 }
 }
 System.out.println(sb.toString().trim());
 }
}
```

**Why it works:** For each k test gcd(k, n) == 1 and collect the passes.

Verified against 3 test cases: `24⏎` → `1 5 7 11 13 17 19 23`; `12⏎` → `1 5 7 11`; `7⏎` → `1 2 3 4 5 6`

### Exercise 2 · Sum of totients

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] phi = new int[n + 1];
 for (int i = 0; i <= n; i++) {
 phi[i] = i;
 }
 for (int p = 2; p <= n; p++) {
 if (phi[p] == p) { // p is prime
 for (int m = p; m <= n; m += p) {
 phi[m] -= phi[m] / p;
 }
 }
 }
 long sum = 0;
 for (int i = 1; i <= n; i++) {
 sum += phi[i];
 }
 System.out.println(sum);
 }
}
```

**Why it works:** The sieve-style table computes all φ values in O(n log log n); the sum is a plain loop.

Verified against 3 test cases: `10⏎` → `32`; `1⏎` → `1`; `100⏎` → `3044`

### Exercise 3 · Coprime pairs

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 // Number of ordered pairs (a, b) with 1 <= a, b <= n and gcd(a, b) = 1.
 // For each larger value b = 2..n there are phi(b) smaller partners a; double for order; add (1, 1).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] phi = new int[n + 1];
 for (int i = 0; i <= n; i++) {
 phi[i] = i;
 }
 for (int p = 2; p <= n; p++) {
 if (phi[p] == p) {
 for (int m = p; m <= n; m += p) {
 phi[m] -= phi[m] / p;
 }
 }
 }
 long sum = 0;
 for (int i = 1; i <= n; i++) {
 sum += phi[i];
 }
 System.out.println(2 * sum - 1);
 }
}
```

**Why it works:** For each b ≥ 2 there are φ(b) values a < b that are coprime to it. Counting both orders gives 2·Σφ(b) for b = 2..n, plus the pair (1,1): 2·(Σ_{i=1..n} φ(i)) − 1.

Verified against 3 test cases: `3⏎` → `7`; `5⏎` → `19`; `1⏎` → `1`

## 5.3 Chinese Remainder Theorem

### Exercise 1 · Two congruences

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int a = sc.nextInt();
 int r1 = sc.nextInt();
 int b = sc.nextInt();
 int r2 = sc.nextInt();
 for (int x = 0; x < a * b; x++) {
 if (x % a == r1 && x % b == r2) {
 System.out.println(x);
 return;
 }
 }
 }
}
```

**Why it works:** CRT guarantees exactly one solution in [0, a·b). A simple loop finds it.

Verified against 3 test cases: `3 2 5 3⏎` → `8`; `4 3 9 5⏎` → `23`; `7 0 11 0⏎` → `0`

### Exercise 2 · CRT with the formula

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 static long[] extGcd(long a, long b) {
 if (b == 0) {
 return new long[] {a, 1, 0};
 }
 long[] r = extGcd(b, a % b);
 return new long[] {r[0], r[2], r[1] - (a / b) * r[2]};
 }

 static long inverse(long a, long m) {
 long x = extGcd(((a % m) + m) % m, m)[1];
 return ((x % m) + m) % m;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long[] mod = new long[n];
 long[] rem = new long[n];
 long M = 1;
 for (int i = 0; i < n; i++) {
 mod[i] = sc.nextLong();
 rem[i] = sc.nextLong();
 M *= mod[i];
 }
 long x = 0;
 for (int i = 0; i < n; i++) {
 long Mi = M / mod[i];
 long term = rem[i] % M * Mi % M * inverse(Mi, mod[i]) % M;
 x = (x + term) % M;
 }
 System.out.println(x);
 }
}
```

**Why it works:** Each term r_i·M_i·inv(M_i) is ≡ r_i for modulus m_i and ≡ 0 for all others. Summing and reducing mod M gives the unique answer.

Verified against 3 test cases: `3⏎3 2⏎5 3⏎7 2⏎` → `23`; `2⏎5 4⏎11 7⏎` → `29`; `1⏎13 6⏎` → `6`

### Exercise 3 · Non-coprime moduli

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 // Moduli need NOT be coprime. Walk through x = r1, r1 + m1, r1 + 2*m1, ... : only these satisfy
 // the first congruence. The remainders mod m2 repeat after m2 steps, so m2 tries are enough.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long r1 = sc.nextLong();
 long m1 = sc.nextLong();
 long r2 = sc.nextLong();
 long m2 = sc.nextLong();
 long x = r1 % m1;
 for (long k = 0; k < m2; k++) {
 if (x % m2 == r2 % m2) {
 System.out.println(x);
 return;
 }
 x += m1;
 }
 System.out.println(-1);
 }
}
```

**Why it works:** Walk through the candidates r1 + k·m1. Their residues mod m2 form a cycle of length ≤ m2, so after m2 candidates every possibility has been checked.

Verified against 3 test cases: `2 6 4 10⏎` → `14`; `2 6 1 4⏎` → `-1`; `0 5 0 5⏎` → `0`

## 5.4 Combinations (nCr)

### Exercise 1 · nCr as a number

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int r = sc.nextInt();
 long result = 1;
 for (int i = 1; i <= r; i++) {
 result = result * (n - r + i) / i; // always exact: result is C(n-r+i, i)
 }
 System.out.println(result);
 }
}
```

**Why it works:** The loop keeps the exact value C(n−r+i, i) after step i, so each division is exact.

Verified against 3 test cases: `5 2⏎` → `10`; `30 15⏎` → `155117520`; `7 0⏎` → `1`

### Exercise 2 · nCr modulo 1 000 000 007

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 final long MOD = 1_000_000_007L;
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int r = sc.nextInt();
 long[][] c = new long[n + 1][r + 1];
 for (int i = 0; i <= n; i++) {
 c[i][0] = 1;
 for (int j = 1; j <= Math.min(i, r); j++) {
 c[i][j] = (c[i - 1][j - 1] + (j <= i - 1 ? c[i - 1][j] : 0)) % MOD;
 }
 }
 System.out.println(c[n][r]);
 }
}
```

**Why it works:** Pascal's recurrence uses only additions, so applying `% MOD` at each step keeps every value small.

Verified against 3 test cases: `10 3⏎` → `120`; `1000 500⏎` → `159835829`; `5 5⏎` → `1`

### Exercise 3 · List all combinations

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 static StringBuilder out = new StringBuilder();

 // choose(start, chosen): extend the current selection with numbers >= start
 static void choose(int n, int r, int start, int[] picked, int count) {
 if (count == r) {
 for (int i = 0; i < r; i++) {
 out.append(i == 0 ? "" : " ").append(picked[i]);
 }
 out.append('\n');
 return;
 }
 for (int v = start; v <= n; v++) {
 picked[count] = v;
 choose(n, r, v + 1, picked, count + 1);
 }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int r = sc.nextInt();
 choose(n, r, 1, new int[r], 0);
 System.out.print(out);
 }
}
```

**Why it works:** Recursive loop from `start` upwards: choose a value, recurse with start = value + 1. Because values only increase, each subset appears once and in lexicographic order.

Verified against 3 test cases: `4 2⏎` → `1 2⏎1 3⏎1 4⏎2 3⏎2 4⏎3 4`; `3 3⏎` → `1 2 3`; `3 1⏎` → `1⏎2⏎3`

## 5.5 Karatsuba's Multiplication

### Exercise 1 · School-book multiplication

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // School-book multiplication of two digit strings: O(n * m) digit multiplications.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String x = sc.next();
 String y = sc.next();
 int[] result = new int[x.length() + y.length()];
 for (int i = x.length() - 1; i >= 0; i--) {
 for (int j = y.length() - 1; j >= 0; j--) {
 int product = (x.charAt(i) - '0') * (y.charAt(j) - '0');
 int pos = i + j + 1; // units position of this partial product
 int sum = product + result[pos];
 result[pos] = sum % 10;
 result[pos - 1] += sum / 10; // carry into the next position to the left
 }
 }
 StringBuilder sb = new StringBuilder();
 for (int digit : result) {
 if (!(sb.length() == 0 && digit == 0)) {
 sb.append(digit);
 }
 }
 System.out.println(sb.length() == 0 ? "0" : sb);
 }
}
```

**Why it works:** The classic method: n·m single-digit products, each added into the right position with carry.

Verified against 3 test cases: `12345 6789⏎` → `83810205`; `0 98765⏎` → `0`; `99999999999 99999999999⏎` → `9999999999800000000001`

### Exercise 2 · Count the base multiplications

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 static int baseCases = 0;

 static long pow10(int k) {
 long p = 1;
 for (int i = 0; i < k; i++) p *= 10;
 return p;
 }

 static long multiply(long x, long y) {
 if (x < 10 || y < 10) {
 baseCases++;
 return x * y;
 }
 int n = Math.max(Long.toString(x).length(), Long.toString(y).length());
 int half = n / 2;
 long p = pow10(half);
 long a = x / p, b = x % p, c = y / p, d = y % p;
 long ac = multiply(a, c);
 long bd = multiply(b, d);
 long abcd = multiply(a + b, c + d);
 return ac * p * p + (abcd - ac - bd) * p + bd;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 multiply(sc.nextLong(), sc.nextLong());
 System.out.println(baseCases);
 }
}
```

**Why it works:** A static counter in the base case counts leaf calls of the recursion tree.

Verified against 3 test cases: `1234 5678⏎` → `11`; `5 7⏎` → `1`; `12 34⏎` → `3`

### Exercise 3 · Big Karatsuba

**Ex3.java**

```java
import java.math.BigInteger;
import java.util.Scanner;

public class Ex3 {
 static BigInteger multiply(BigInteger x, BigInteger y) {
 int n = Math.max(x.bitLength(), y.bitLength());
 if (n <= 30) { // both factors < 2^30, so the product < 2^60 fits in a long
 return BigInteger.valueOf(x.longValue() * y.longValue());
 }
 int half = n / 2;
 BigInteger a = x.shiftRight(half);
 BigInteger b = x.subtract(a.shiftLeft(half));
 BigInteger c = y.shiftRight(half);
 BigInteger d = y.subtract(c.shiftLeft(half));
 BigInteger ac = multiply(a, c);
 BigInteger bd = multiply(b, d);
 BigInteger mid = multiply(a.add(b), c.add(d)).subtract(ac).subtract(bd);
 return ac.shiftLeft(2 * half).add(mid.shiftLeft(half)).add(bd);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 BigInteger x = new BigInteger(sc.next());
 BigInteger y = new BigInteger(sc.next());
 System.out.println(multiply(x, y));
 }
}
```

**Why it works:** Same three-product identity as the long version, but with BigInteger and bit shifts; the base case is a plain 30-bit multiplication.

Verified against 3 test cases: `123456789 987654321⏎` → `121932631112635269`; `10000000000000000000 10000000000000000000⏎` → `100000000000000000000000000000000000000000`; `0 5⏎` → `0`

## 5.6 Booth's Algorithm (signed binary multiplication)

### Exercise 1 · n-bit two's complement

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 // Print x in n-bit two's complement (works for negative x too).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int x = sc.nextInt();
 int n = sc.nextInt();
 StringBuilder sb = new StringBuilder();
 for (int bit = n - 1; bit >= 0; bit--) {
 sb.append((x >> bit) & 1); // >> on a negative int keeps the sign, so high bits come out as 1s
 }
 System.out.println(sb);
 }
}
```

**Why it works:** Java stores int in two's complement, so shifting and masking one bit at a time reads off the pattern directly.

Verified against 3 test cases: `-5 8⏎` → `11111011`; `7 4⏎` → `0111`; `-1 4⏎` → `1111`

### Exercise 2 · Booth's operations

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 // For an n-bit multiplier given as a bit string, list Booth's operation for each cycle
     (rightmost bit first).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String q = sc.next();
 int previous = 0; // Q-1 starts as 0
 StringBuilder sb = new StringBuilder();
 for (int i = q.length() - 1; i >= 0; i--) {
 int current = q.charAt(i) - '0';
 String op;
 if (current == 1 && previous == 0) {
 op = "-M";
 } else if (current == 0 && previous == 1) {
 op = "+M";
 } else {
 op = "0";
 }
 sb.append(sb.length() == 0 ? "" : " ").append(op);
 previous = current;
 }
 System.out.println(sb);
 }
}
```

**Why it works:** The current bit with the previously processed (lower) bit forms the pair $Q_0Q_{-1}$; the table of conditions gives the action.

Verified against 3 test cases: `0110⏎` → `0 -M 0 +M`; `0011⏎` → `-M 0 +M 0`; `1111⏎` → `-M 0 0 0`

### Exercise 3 · Booth multiplication

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int m = sc.nextInt();
 int q = sc.nextInt();
 int width = 16; // inputs fit in 16-bit signed numbers
 if (m == -(1 << (width - 1))) {
 width++;
 }
 long mask = (1L << width) - 1;
 long M = m & mask;
 long negM = (-m) & mask;
 long ac = 0;
 long qr = q & mask;
 int qm1 = 0;
 for (int i = 0; i < width; i++) {
 int q0 = (int) (qr & 1);
 if (q0 == 1 && qm1 == 0) {
 ac = (ac + negM) & mask;
 } else if (q0 == 0 && qm1 == 1) {
 ac = (ac + M) & mask;
 }
 qm1 = (int) (qr & 1);
 qr = ((qr >> 1) | ((ac & 1) << (width - 1))) & mask;
 ac = (ac >> 1) | (ac & (1L << (width - 1)));
 }
 long raw = (ac << width) | qr;
 long sign = 1L << (2 * width - 1);
 long product = (raw & sign) != 0 ? raw - (1L << (2 * width)) : raw; // read the 2n-bit pattern
     as signed
 System.out.println(product);
 }
}
```

**Why it works:** Direct simulation of the hardware loop; the final 2n-bit pattern is converted to a signed number.

Verified against 3 test cases: `-3 4⏎` → `-12`; `7 3⏎` → `21`; `-8 -8⏎` → `64`

## 5.7 Simple Sieve of Eratosthenes

### Exercise 1 · Count primes

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 boolean[] composite = new boolean[n + 1];
 int count = 0;
 for (int i = 2; i <= n; i++) {
 if (!composite[i]) {
 count++;
 for (long m = (long) i * i; m <= n; m += i) {
 composite[(int) m] = true;
 }
 }
 }
 System.out.println(count);
 }
}
```

**Why it works:** Standard sieve; every i not crossed out when reached is prime and is counted, then its multiples are crossed out.

Verified against 3 test cases: `100⏎` → `25`; `1⏎` → `0`; `1000000⏎` → `78498`

### Exercise 2 · The k-th prime

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int k = sc.nextInt();
 int limit = 2_000_000; // the 100 000th prime is 1 299 709, so this is enough
 boolean[] composite = new boolean[limit + 1];
 int seen = 0;
 for (int i = 2; i <= limit; i++) {
 if (!composite[i]) {
 seen++;
 if (seen == k) {
 System.out.println(i);
 return;
 }
 for (long m = (long) i * i; m <= limit; m += i) {
 composite[(int) m] = true;
 }
 }
 }
 }
}
```

**Why it works:** A single sieve to 2 000 000 contains the 100 000th prime (1 299 709); count survivors until the k-th.

Verified against 3 test cases: `1000⏎` → `7919`; `1⏎` → `2`; `100000⏎` → `1299709`

### Exercise 3 · Prime factorisation queries

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 public static void main(String[] args) {
 int limit = 1_000_000;
 int[] spf = new int[limit + 1]; // spf[x] = smallest prime factor of x
 for (int i = 2; i <= limit; i++) {
 if (spf[i] == 0) {
 for (int m = i; m <= limit; m += i) {
 if (spf[m] == 0) {
 spf[m] = i;
 }
 }
 }
 }
 Scanner sc = new Scanner(System.in);
 int q = sc.nextInt();
 StringBuilder out = new StringBuilder();
 for (int t = 0; t < q; t++) {
 int x = sc.nextInt();
 int original = x;
 StringBuilder line = new StringBuilder();
 while (x > 1) {
 int p = spf[x];
 int e = 0;
 while (x % p == 0) {
 x /= p;
 e++;
 }
 if (line.length() > 0) {
 line.append(" * ");
 }
 line.append(p);
 if (e > 1) {
 line.append('^').append(e);
 }
 }
 out.append(original).append(" = ").append(original == 1 ? "1" : line).append('\n');
 }
 System.out.print(out);
 }
}
```

**Why it works:** A "smallest prime factor" sieve stores for each number its smallest prime divisor, so every factorisation takes O(log x) steps.

Verified against 2 test cases: `3⏎12⏎97⏎360⏎` → `12 = 2^2 * 3⏎97 = 97⏎360 = 2^3 * 3^2 * 5`; `2⏎2⏎1000000⏎` → `2 = 2⏎1000000 = 2^6 * 5^6`

## 5.8 Segmented & Incremental Sieve

### Exercise 1 · Primes in a small range

**Ex4.java**

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ex4 {
 // Count primes in [L, R] where R can be as large as 10^12 but R - L <= 10^6.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long low = sc.nextLong();
 long high = sc.nextLong();
 int root = (int) Math.sqrt((double) high);
 while ((long) root * root > high) root--;
 while ((long) (root + 1) * (root + 1) <= high) root++;

 boolean[] small = new boolean[root + 1];
 List<Integer> base = new ArrayList<>();
 for (int i = 2; i <= root; i++) {
 if (!small[i]) {
 base.add(i);
 for (long m = (long) i * i; m <= root; m += i) {
 small[(int) m] = true;
 }
 }
 }
 boolean[] composite = new boolean[(int) (high - low + 1)];
 for (int p : base) {
 long start = Math.max((long) p * p, (low + p - 1) / p * p);
 for (long m = start; m <= high; m += p) {
 composite[(int) (m - low)] = true;
 }
 }
 int count = 0;
 for (long n = Math.max(low, 2); n <= high; n++) {
 if (!composite[(int) (n - low)]) {
 count++;
 }
 }
 System.out.println(count);
 }
}
```

**Why it works:** This reference solution already uses the segmented method, which also covers small ranges.

Verified against 3 test cases: `10 30⏎` → `6`; `1 1⏎` → `0`; `100 130⏎` → `6`

### Exercise 2 · Primes in a huge range

**Ex4.java**

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ex4 {
 // Count primes in [L, R] where R can be as large as 10^12 but R - L <= 10^6.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long low = sc.nextLong();
 long high = sc.nextLong();
 int root = (int) Math.sqrt((double) high);
 while ((long) root * root > high) root--;
 while ((long) (root + 1) * (root + 1) <= high) root++;

 boolean[] small = new boolean[root + 1];
 List<Integer> base = new ArrayList<>();
 for (int i = 2; i <= root; i++) {
 if (!small[i]) {
 base.add(i);
 for (long m = (long) i * i; m <= root; m += i) {
 small[(int) m] = true;
 }
 }
 }
 boolean[] composite = new boolean[(int) (high - low + 1)];
 for (int p : base) {
 long start = Math.max((long) p * p, (low + p - 1) / p * p);
 for (long m = start; m <= high; m += p) {
 composite[(int) (m - low)] = true;
 }
 }
 int count = 0;
 for (long n = Math.max(low, 2); n <= high; n++) {
 if (!composite[(int) (n - low)]) {
 count++;
 }
 }
 System.out.println(count);
 }
}
```

**Why it works:** Segmented sieve: base primes up to √R (≤ 10⁶) and a window array of at most 10⁶ + 1 cells.

Verified against 3 test cases: `1000000000000 1000000000100⏎` → `4`; `2 2⏎` → `1`; `999999999900 1000000000000⏎` → `4`

### Exercise 3 · Largest prime gap in a window

**Ex5.java**

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ex5 {
 // Largest gap between two consecutive primes inside [L, R]; print -1 if fewer than two primes.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long low = sc.nextLong();
 long high = sc.nextLong();
 int root = (int) Math.sqrt((double) high);
 while ((long) root * root > high) root--;
 while ((long) (root + 1) * (root + 1) <= high) root++;
 boolean[] small = new boolean[root + 1];
 List<Integer> base = new ArrayList<>();
 for (int i = 2; i <= root; i++) {
 if (!small[i]) {
 base.add(i);
 for (long m = (long) i * i; m <= root; m += i) {
 small[(int) m] = true;
 }
 }
 }
 boolean[] composite = new boolean[(int) (high - low + 1)];
 for (int p : base) {
 long start = Math.max((long) p * p, (low + p - 1) / p * p);
 for (long m = start; m <= high; m += p) {
 composite[(int) (m - low)] = true;
 }
 }
 long prev = -1;
 long best = -1;
 for (long n = Math.max(low, 2); n <= high; n++) {
 if (!composite[(int) (n - low)]) {
 if (prev != -1) {
 best = Math.max(best, n - prev);
 }
 prev = n;
 }
 }
 System.out.println(best);
 }
}
```

**Why it works:** After the segmented sieve, one pass over the window tracks the last prime seen and the maximum gap.

Verified against 3 test cases: `1 100⏎` → `8`; `90 96⏎` → `-1`; `2 3⏎` → `1`

## 5.9 Binary Palindrome

### Exercise 1 · Is it a binary palindrome?

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int x = sc.nextInt();
 int rev = 0, rest = x;
 while (rest > 0) {
 rev = (rev << 1) | (rest & 1);
 rest >>= 1;
 }
 System.out.println(rev == x ? "YES" : "NO");
 }
}
```

**Why it works:** Reverse the bits into a second variable with shifts and compare. 0 never enters the loop, reversed stays 0 and equals n.

Verified against 4 test cases: `5⏎` → `YES`; `10⏎` → `NO`; `0⏎` → `YES`

### Exercise 2 · Count binary palindromes up to n

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 static boolean pal(int x) {
 int rev = 0, rest = x;
 while (rest > 0) {
 rev = (rev << 1) | (rest & 1);
 rest >>= 1;
 }
 return rev == x;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int count = 0;
 for (int i = 1; i <= n; i++) if (pal(i)) count++;
 System.out.println(count);
 }
}
```

**Why it works:** Test every i in 1..n with the O(log i) check. Total O(n log n).

Verified against 3 test cases: `10⏎` → `5`; `1⏎` → `1`; `100⏎` → `19`

### Exercise 3 · Next binary palindrome

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 static boolean pal(long x) {
 long rev = 0, rest = x;
 while (rest > 0) {
 rev = (rev << 1) | (rest & 1);
 rest >>= 1;
 }
 return rev == x;
 }

 // Smallest binary palindrome strictly greater than n.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long n = sc.nextLong();
 long x = n + 1;
 while (!pal(x)) x++;
 System.out.println(x);
 }
}
```

**Why it works:** Simple forward search is fast enough because the next palindrome is always within about 2^(len/2) steps (for a 30-bit number, at most ~33 000 steps).

Verified against 3 test cases: `7⏎` → `9`; `0⏎` → `1`; `51⏎` → `63`

## 5.10 Swap Two Nibbles in a Byte

### Exercise 1 · Swap the nibbles

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int x = sc.nextInt() & 0xFF;
 System.out.println(((x & 0xF0) >>> 4) | ((x & 0x0F) << 4));
 }
}
```

**Why it works:** Mask each nibble, shift into the other's place, OR together.

Verified against 4 test cases: `171⏎` → `186`; `0⏎` → `0`; `255⏎` → `255`

### Exercise 2 · Swap adjacent bits

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 // Swap every pair of adjacent bits (bit0<->bit1, bit2<->bit3, ...) of a 32-bit value; print as unsigned.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long input = sc.nextLong();
 int x = (int) input;
 int even = x & 0x55555555; // bits 0, 2, 4, ...
 int odd = x & 0xAAAAAAAA; // bits 1, 3, 5, ...
 int swapped = (even << 1) | (odd >>> 1);
 System.out.println(swapped & 0xFFFFFFFFL);
 }
}
```

**Why it works:** The two masks split the word into non-overlapping bit sets; each set moves by one position into the other set's slots.

Verified against 3 test cases: `23⏎` → `43`; `0⏎` → `0`; `4294967295⏎` → `4294967295`

### Exercise 3 · Reverse 32 bits

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 // Reverse the 32 bits of an unsigned 32-bit integer.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long input = sc.nextLong();
 int x = (int) input;
 int result = 0;
 for (int i = 0; i < 32; i++) {
 result = (result << 1) | (x & 1);
 x >>>= 1;
 }
 System.out.println(result & 0xFFFFFFFFL);
 }
}
```

**Why it works:** Same peel-and-push technique as the binary-palindrome lesson, but for exactly 32 iterations so leading zeros are included.

Verified against 3 test cases: `43261596⏎` → `964176192`; `1⏎` → `2147483648`; `0⏎` → `0`

## 5.11 Longest Sequence of 1s (and after flipping a bit)

### Exercise 1 · Longest run of 1s

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int x = sc.nextInt();
 int cur = 0, best = 0;
 for (int i = 0; i < 32; i++) {
 if ((x & 1) == 1) best = Math.max(best, ++cur);
 else cur = 0;
 x >>>= 1;
 }
 System.out.println(best);
 }
}
```

**Why it works:** Count while bits are 1, reset on 0, remember the maximum.

Verified against 4 test cases: `56825⏎` → `6`; `0⏎` → `0`; `-1⏎` → `32`

### Exercise 2 · Flip one bit

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int x = sc.nextInt();
 int best = 0;
 for (int bit = 0; bit < 32; bit++) {
 int y = x | (1 << bit);
 int run = 0;
 for (int i = 0; i < 32; i++) {
 if (((y >>> i) & 1) == 1) best = Math.max(best, ++run);
 else run = 0;
 }
 }
 System.out.println(best);
 }
}
```

**Why it works:** Set each bit in turn (OR with 1 << bit) and measure the longest run; the maximum over 32 trials is the answer. The one-pass method from the lesson gives the same result.

Verified against 4 test cases: `478⏎` → `8`; `0⏎` → `1`; `-1⏎` → `32`

### Exercise 3 · Flip up to k zeros

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 // Longest run of 1s after flipping at most k zeros: sliding window over the 32 bits containing
     <= k zeros.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int x = sc.nextInt();
 int k = sc.nextInt();
 int left = 0, zeros = 0, best = 0;
 for (int right = 0; right < 32; right++) {
 if (((x >>> right) & 1) == 0) zeros++;
 while (zeros > k) {
 if (((x >>> left) & 1) == 0) zeros--;
 left++;
 }
 best = Math.max(best, right - left + 1);
 }
 System.out.println(best);
 }
}
```

**Why it works:** Classic two-pointer window: the right end moves one step per bit; the left end only moves forward, so total work is O(32).

Verified against 3 test cases: `478 2⏎` → `9`; `0 5⏎` → `5`; `-1 0⏎` → `32`

## 5.12 Josephus Problem

### Exercise 1 · Elimination order

**Ex1.java**

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int k = sc.nextInt();
 List<Integer> circle = new ArrayList<>();
 for (int i = 1; i <= n; i++) circle.add(i);
 StringBuilder sb = new StringBuilder();
 int idx = 0;
 while (!circle.isEmpty()) {
 idx = (idx + k - 1) % circle.size();
 sb.append(sb.length() == 0 ? "" : " ").append(circle.remove(idx));
 }
 System.out.println(sb);
 }
}
```

**Why it works:** Direct simulation; the last remaining person is removed at the end of the loop so the output ends with the survivor.

Verified against 3 test cases: `5 2⏎` → `2 4 1 5 3`; `7 3⏎` → `3 6 2 7 5 1 4`; `1 5⏎` → `1`

### Exercise 2 · Survivor for large N

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long n = sc.nextLong();
 long k = sc.nextLong();
 long ans = 0;
 for (long people = 2; people <= n; people++) {
 ans = (ans + k) % people;
 }
 System.out.println(ans + 1);
 }
}
```

**Why it works:** The loop ans = (ans + k) % people for people = 2..N takes O(N) time and O(1) space.

Verified against 3 test cases: `10 3⏎` → `4`; `14 2⏎` → `13`; `1 7⏎` → `1`

### Exercise 3 · Josephus with k = 2 and N up to $10^{18}$

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long n = sc.nextLong();
 System.out.println(2 * (n - Long.highestOneBit(n)) + 1);
 }
}
```

**Why it works:** After r eliminations 2^m people remain and the counting position is person 2r+1; with k = 2 and a power-of-two circle that person survives.

Verified against 4 test cases: `100⏎` → `73`; `1⏎` → `1`; `64⏎` → `1`

## 5.13 Block Swap Algorithm (Array Rotation)

### Exercise 1 · Left rotation

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int d = sc.nextInt() % n;
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) {
 sb.append(i == 0 ? "" : " ").append(a[(i + d) % n]);
 }
 System.out.println(sb);
 }
}
```

**Why it works:** The index formula (i + d) % n gives the result directly, without moving anything in place.

Verified against 3 test cases: `5 2⏎1 2 3 4 5⏎` → `3 4 5 1 2`; `4 0⏎9 8 7 6⏎` → `9 8 7 6`; `3 7⏎1 2 3⏎` → `2 3 1`

### Exercise 2 · Right rotation in place

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 static void reverse(int[] a, int lo, int hi) {
 while (lo < hi) {
 int t = a[lo];
 a[lo++] = a[hi];
 a[hi--] = t;
 }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int d = sc.nextInt() % n;
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 // right rotation by d == left rotation by n - d
 int left = (n - d) % n;
 reverse(a, 0, left - 1);
 reverse(a, left, n - 1);
 reverse(a, 0, n - 1);
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) sb.append(i == 0 ? "" : " ").append(a[i]);
 System.out.println(sb);
 }
}
```

**Why it works:** Convert to a left rotation by (n − d) mod n and apply the three-reversal trick.

Verified against 3 test cases: `5 2⏎1 2 3 4 5⏎` → `4 5 1 2 3`; `6 6⏎1 2 3 4 5 6⏎` → `1 2 3 4 5 6`; `2 1⏎1 2⏎` → `2 1`

### Exercise 3 · How many rotations?

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // The smallest element sits right after the "break" where a[i] > a[i+1].
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 int lo = 0, hi = n - 1;
 while (lo < hi) {
 int mid = (lo + hi) / 2;
 if (a[mid] > a[hi]) {
 lo = mid + 1; // the minimum is to the right of mid
 } else {
 hi = mid; // the minimum is at mid or to its left
 }
 }
 System.out.println(lo); // index of the minimum == number of left rotations
 }
}
```

**Why it works:** If a[mid] > a[hi] the minimum is right of mid; otherwise it is at mid or left of it. The loop narrows to the minimum's index in O(log n).

Verified against 3 test cases: `6⏎15 18 2 3 6 12⏎` → `2`; `5⏎1 2 3 4 5⏎` → `0`; `4⏎2 3 4 1⏎` → `3`

## 5.14 Leaders in an Array

### Exercise 1 · Print the leaders

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 int[] out = new int[n];
 int c = 0;
 int max = Integer.MIN_VALUE;
 for (int i = n - 1; i >= 0; i--) {
 if (a[i] > max) {
 out[c++] = a[i];
 max = a[i];
 }
 }
 StringBuilder sb = new StringBuilder();
 for (int i = c - 1; i >= 0; i--) sb.append(sb.length() == 0 ? "" : " ").append(out[i]);
 System.out.println(sb);
 }
}
```

**Why it works:** The running maximum of the suffix decides each element in O(1).

Verified against 4 test cases: `6⏎16 17 4 3 5 2⏎` → `17 5 2`; `5⏎14 20 3 6 5⏎` → `20 6 5`; `3⏎5 4 3⏎` → `5 4 3`

### Exercise 2 · Non-strict leaders

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Non-strict leaders: an element counts if it is >= every element to its right.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 int[] out = new int[n];
 int c = 0;
 int max = Integer.MIN_VALUE;
 for (int i = n - 1; i >= 0; i--) {
 if (a[i] >= max) {
 out[c++] = a[i];
 max = a[i];
 }
 }
 StringBuilder sb = new StringBuilder();
 for (int i = c - 1; i >= 0; i--) sb.append(sb.length() == 0 ? "" : " ").append(out[i]);
 System.out.println(sb);
 }
}
```

**Why it works:** Replace `>` by `>=` in the running-maximum test.

Verified against 3 test cases: `5⏎7 10 4 10 6⏎` → `10 10 6`; `3⏎2 2 2⏎` → `2 2 2`; `1⏎9⏎` → `9`

### Exercise 3 · Next greater element

Ex3.java

```java
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Scanner;

public class Ex3 {
 // Next greater element to the right (or -1), in O(n) with a stack of candidates scanned from the right.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 int[] ans = new int[n];
 Deque<Integer> stack = new ArrayDeque<>(); // values, decreasing from top... top is nearest candidate
 for (int i = n - 1; i >= 0; i--) {
 while (!stack.isEmpty() && stack.peek() <= a[i]) {
 stack.pop(); // can never be the next-greater for anything to the left
 }
 ans[i] = stack.isEmpty() ? -1 : stack.peek();
 stack.push(a[i]);
 }
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) sb.append(i == 0 ? "" : " ").append(ans[i]);
 System.out.println(sb);
 }
}
```

**Why it works:** The stack holds the elements that could still be "next greater" for something on the left, in decreasing order. Each element is pushed and popped at most once: O(n).

Verified against 3 test cases: `4⏎4 5 2 25⏎` → `5 25 25 -1`; `4⏎13 7 6 12⏎` → `-1 12 12 -1`; `3⏎3 2 1⏎` → `-1 -1 -1`

## 5.15 Majority Element (Boyer–Moore Voting)

### Exercise 1 · Majority with a HashMap

Ex1.java

```java
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 Map<Integer, Integer> freq = new HashMap<>();
 Integer winner = null;
 for (int i = 0; i < n; i++) {
 int x = sc.nextInt();
 int c = freq.merge(x, 1, Integer::sum);
 if (c > n / 2) winner = x;
 }
 System.out.println(winner == null ? "No majority element" : winner);
 }
}
```

**Why it works:** merge(x, 1, Integer::sum) increments the counter; any value whose count passes n/2 is the answer.

Verified against 3 test cases: `5⏎2 2 1 1 1⏎` → `1`; `4⏎1 2 1 2⏎` → `No majority element`; `1⏎7⏎` → `7`

### Exercise 2 · Boyer–Moore

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 int candidate = 0, count = 0;
 for (int x : a) {
 if (count == 0) {
 candidate = x;
 count = 1;
 } else if (x == candidate) count++;
 else count--;
 }
 int occ = 0;
 for (int x : a) if (x == candidate) occ++;
 System.out.println(occ > n / 2 ? String.valueOf(candidate) : "No majority element");
 }
}
```

**Why it works:** Phase 1 keeps a candidate and a vote counter; phase 2 counts the real occurrences.

Verified against 3 test cases: `5⏎2 2 1 1 1⏎` → `1`; `7⏎3 3 4 2 4 4 2⏎` → `No majority element`; `9⏎2 2 2 6 2 2 8 2 1⏎` → `2`

### Exercise 3 · Elements appearing more than n/3 times

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Elements appearing more than n/3 times: at most TWO of them exist, so keep two candidates.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 int c1 = 0, c2 = 1, n1 = 0, n2 = 0;
 for (int x : a) {
 if (x == c1) n1++;
 else if (x == c2) n2++;
 else if (n1 == 0) { c1 = x; n1 = 1; }
 else if (n2 == 0) { c2 = x; n2 = 1; }
 else { n1--; n2--; }
 }
 int o1 = 0, o2 = 0;
 for (int x : a) {
 if (x == c1) o1++;
 else if (x == c2) o2++;
 }
 StringBuilder sb = new StringBuilder();
 if (o1 > n / 3) sb.append(c1);
 if (o2 > n / 3) sb.append(sb.length() == 0 ? "" : " ").append(c2);
 if (sb.length() > 0 && o1 > n / 3 && o2 > n / 3 && c1 > c2) {
 sb = new StringBuilder().append(c2).append(' ').append(c1); // print in increasing order
 }
 System.out.println(sb.length() == 0 ? "None" : sb);
 }
}
```

**Why it works:** Generalised Boyer–Moore: removing three different values at a time cannot eliminate a value that has more than n/3 occurrences.

Verified against 3 test cases: `8⏎1 1 1 3 3 2 2 2⏎` → `1 2`; `3⏎3 2 3⏎` → `3`; `4⏎1 2 3 4⏎` → `None`

## 5.16 Maximum Product Subarray

### Exercise 1 · Count subarrays with positive product

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // Count subarrays whose product is strictly positive.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 int count = 0;
 for (int i = 0; i < n; i++) {
 int negatives = 0;
 for (int j = i; j < n; j++) {
 if (a[j] == 0) break; // a zero makes every longer subarray's product 0
 if (a[j] < 0) negatives++;
 if (negatives % 2 == 0) count++;
 }
 }
 System.out.println(count);
 }
}
```

**Why it works:** Track the parity of negatives while extending; stop at a zero.

Verified against 3 test cases: `5⏎2 3 -2 4 0⏎` → `4`; `3⏎-1 -2 -3⏎` → `2`; `3⏎0 0 0⏎` → `0`

### Exercise 2 · Maximum product subarray

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long mx = 0, mn = 0, best = 0;
 for (int i = 0; i < n; i++) {
 long x = sc.nextInt();
 if (i == 0) {
 mx = mn = best = x;
 continue;
 }
 if (x < 0) {
 long t = mx;
 mx = mn;
 mn = t;
 }
 mx = Math.max(x, mx * x);
 mn = Math.min(x, mn * x);
 best = Math.max(best, mx);
 }
 System.out.println(best);
 }
}
```

**Why it works:** The O(n) max/min recurrence from the lesson.

Verified against 4 test cases: `4⏎2 3 -2 4⏎` → `6`; `3⏎-2 0 -1⏎` → `0`; `4⏎-5 -4 -1 -2⏎` → `40`

### Exercise 3 · Which subarray?

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 // Print "product start end" for the maximum-product subarray (0-based, earliest start, then
     earliest end).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long[] a = new long[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextLong();
 long best = Long.MIN_VALUE;
 int bs = 0, be = 0;
 for (int i = 0; i < n; i++) {
 long p = 1;
 for (int j = i; j < n; j++) {
 p *= a[j];
 if (p > best) {
 best = p;
 bs = i;
 be = j;
 }
 }
 }
 System.out.println(best + " " + bs + " " + be);
 }
}
```

**Why it works:** Enumerate (i, j) in order and update the record only when the product is strictly greater.

Verified against 3 test cases: `4⏎2 3 -2 4⏎` → `6 0 1`; `1⏎5⏎` → `5 0 0`; `3⏎-1 -1 -1⏎` → `1 0 1`

## 5.17 Maximum Equilibrium Sum

### Exercise 1 · First equilibrium index

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long[] a = new long[n];
 long total = 0;
 for (int i = 0; i < n; i++) {
 a[i] = sc.nextLong();
 total += a[i];
 }
 long left = 0;
 for (int i = 0; i < n; i++) {
 if (left == total - left - a[i]) {
 System.out.println(i);
 return;
 }
 left += a[i];
 }
 System.out.println(-1);
 }
}
```

**Why it works:** Running left sum and a precomputed total give the right sum in O(1).

Verified against 3 test cases: `7⏎-7 1 5 2 -4 3 0⏎` → `3`; `3⏎1 2 3⏎` → `-1`; `1⏎5⏎` → `0`

### Exercise 2 · Maximum equilibrium sum

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 // Print the maximum equilibrium sum (one side's sum), or "none".
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long[] a = new long[n];
 long total = 0;
 for (int i = 0; i < n; i++) {
 a[i] = sc.nextLong();
 total += a[i];
 }
 long left = 0;
 boolean found = false;
 long best = Long.MIN_VALUE;
 for (int i = 0; i < n; i++) {
 if (left == total - left - a[i]) {
 found = true;
 best = Math.max(best, left);
 }
 left += a[i];
 }
 System.out.println(found ? String.valueOf(best) : "none");
 }
}
```

**Why it works:** Same scan; update the maximum whenever left == right.

Verified against 3 test cases: `9⏎1 2 3 4 5 4 3 2 1⏎` → `10`; `7⏎-7 1 5 2 -4 3 0⏎` → `0`; `2⏎1 2⏎` → `none`

### Exercise 3 · Balanced split points

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Count split points k (1 <= k <= n-1) where sum(a[0..k-1]) == sum(a[k..n-1]).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long[] a = new long[n];
 long total = 0;
 for (int i = 0; i < n; i++) {
 a[i] = sc.nextLong();
 total += a[i];
 }
 long prefix = 0;
 int count = 0;
 for (int k = 1; k < n; k++) {
 prefix += a[k - 1];
 if (prefix == total - prefix) count++;
 }
 System.out.println(count);
 }
}
```

**Why it works:** prefix == total − prefix at each split point; count them.

Verified against 3 test cases: `6⏎1 -1 1 -1 1 -1⏎` → `2`; `4⏎2 2 2 2⏎` → `1`; `3⏎1 2 3⏎` → `1`

## 5.18 Maximum Sum of an Hourglass in a Matrix

### Exercise 1 · One hourglass

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // Read a 3x3 matrix and print the single hourglass sum.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int[][] m = new int[3][3];
 for (int i = 0; i < 3; i++) for (int j = 0; j < 3; j++) m[i][j] = sc.nextInt();
 System.out.println(m[0][0] + m[0][1] + m[0][2] + m[1][1] + m[2][0] + m[2][1] + m[2][2]);
 }
}
```

**Why it works:** Add the two full outer rows and the centre cell.

Verified against 3 test cases: `1 1 1⏎0 1 0⏎1 1 1⏎` → `7`; `0 0 0⏎5 5 5⏎0 0 0⏎` → `5`; `-1 -1 -1⏎-1 -1 -1⏎-1 -1 -1⏎` → `-7`

### Exercise 2 · Maximum hourglass with negatives

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int r = sc.nextInt();
 int c = sc.nextInt();
 int[][] m = new int[r][c];
 for (int i = 0; i < r; i++) for (int j = 0; j < c; j++) m[i][j] = sc.nextInt();
 int best = Integer.MIN_VALUE;
 for (int i = 0; i + 2 < r; i++) {
 for (int j = 0; j + 2 < c; j++) {
 int s = m[i][j] + m[i][j + 1] + m[i][j + 2] + m[i + 1][j + 1] + m[i + 2][j] + m[i + 2][j + 1] +
 m[i + 2][j + 2];
 best = Math.max(best, s);
 }
 }
 System.out.println(best);
 }
}
```

**Why it works:** All (R−2)(C−2) hourglass sums are compared; the initial value must be below any possible sum.

Verified against 3 test cases: `3 3⏎-1 -1 -1⏎-1 -9 -1⏎-1 -1 -1⏎` → `-15`; `4 4⏎1 2 3 4⏎5 6 7 8⏎9 1 2 3⏎4 5 6 7⏎` → `41`; `3 4⏎1 1 1 1⏎1 9 1 1⏎1 1 1 1⏎` → `15`

### Exercise 3 · Largest k×k block

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 // Largest sum of any k x k block, using a 2-D prefix-sum table so each block costs O(1).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int r = sc.nextInt();
 int c = sc.nextInt();
 int k = sc.nextInt();
 long[][] p = new long[r + 1][c + 1]; // p[i][j] = sum of the rectangle rows 0..i-1, cols 0..j-1
 for (int i = 1; i <= r; i++) {
 for (int j = 1; j <= c; j++) {
 p[i][j] = sc.nextLong() + p[i - 1][j] + p[i][j - 1] - p[i - 1][j - 1];
 }
 }
 long best = Long.MIN_VALUE;
 for (int i = k; i <= r; i++) {
 for (int j = k; j <= c; j++) {
 long s = p[i][j] - p[i - k][j] - p[i][j - k] + p[i - k][j - k];
 best = Math.max(best, s);
 }
 }
 System.out.println(best);
 }
}
```

**Why it works:** The prefix table lets any rectangle sum be evaluated in four lookups, independent of its size.

Verified against 3 test cases: `3 3 2⏎1 2 3⏎4 5 6⏎7 8 9⏎` → `28`; `2 2 2⏎-1 -2⏎-3 -4⏎` → `-10`; `2 3 1⏎5 -1 3⏎0 2 -4⏎` → `5`

## 5.19 Toggle the Switch (100 Doors)

### Exercise 1 · Count the open doors

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 boolean[] open = new boolean[n + 1];
 for (int m = 1; m <= n; m++) for (int d = m; d <= n; d += m) open[d] = !open[d];
 int c = 0;
 for (int d = 1; d <= n; d++) if (open[d]) c++;
 System.out.println(c);
 }
}
```

**Why it works:** Simulate every monkey and count the true cells.

Verified against 3 test cases: `10⏎` → `3`; `100⏎` → `10`; `1⏎` → `1`

### Exercise 2 · A trillion doors

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long n = sc.nextLong();
 long r = (long) Math.sqrt((double) n);
 while (r * r > n) r--;
 while ((r + 1) * (r + 1) <= n) r++;
 System.out.println(r);
 }
}
```

**Why it works:** $\lfloor\sqrt{n}\rfloor$ with a floating-point estimate corrected by two while-loops.

Verified against 3 test cases: `1000000000000000000⏎` → `1000000000`; `99⏎` → `9`; `999999999999999999⏎` → `999999999`

### Exercise 3 · Only the first m monkeys

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Only the first m monkeys are released. Count doors that are open afterwards.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int m = sc.nextInt();
 boolean[] open = new boolean[n + 1];
 for (int monkey = 1; monkey <= Math.min(m, n); monkey++) {
 for (int d = monkey; d <= n; d += monkey) open[d] = !open[d];
 }
 int c = 0;
 for (int d = 1; d <= n; d++) if (open[d]) c++;
 System.out.println(c);
 }
}
```

**Why it works:** The shortcut no longer applies (a door's toggle count is the number of divisors ≤ m), so simulate only the m monkeys.

Verified against 3 test cases: `10 3⏎` → `4`; `10 10⏎` → `3`; `6 1⏎` → `6`

## 5.20 Alice's Apple Tree (Minimum Garden Perimeter)

### Exercise 1 · Apples in a square

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long r = sc.nextLong();
 System.out.println(2 * r * (r + 1) * (2 * r + 1));
 }
}
```

**Why it works:** Direct evaluation of the closed form in long arithmetic.

Verified against 3 test cases: `3⏎` → `168`; `1⏎` → `12`; `2⏎` → `60`

### Exercise 2 · Minimum perimeter (small)

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long need = sc.nextLong();
 long r = 0, sum = 0;
 while (sum < need) {
 r++;
 sum += 12 * r * r;
 }
 System.out.println(8 * r);
 }
}
```

**Why it works:** Ring-by-ring accumulation in long.

Verified against 3 test cases: `13⏎` → `16`; `1⏎` → `8`; `1000000000⏎` → `5040`

### Exercise 3 · Minimum perimeter (large)

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 static long inside(long r) {
 return 2 * r * (r + 1) * (2 * r + 1);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long need = sc.nextLong();
 long lo = 1, hi = 200_000;
 while (lo < hi) {
 long mid = (lo + hi) / 2;
 if (inside(mid) >= need) hi = mid;
 else lo = mid + 1;
 }
 System.out.println(8 * lo);
 }
}
```

**Why it works:** Binary search for the smallest r with apples(r) ≥ needed; answer 8r.

Verified against 3 test cases: `100000000000000⏎` → `503968`; `13⏎` → `16`; `1⏎` → `8`

## 5.21 Weighted Substring (Smallest String of a Given Weight)

### Exercise 1 · Weight of a string

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 // Total weight of a string made of A..Z, with weight(A)=1 and weight(next) = (index+1) *
     weight(previous).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 long[] w = new long[26];
 w[0] = 1;
 for (int i = 1; i < 26; i++) w[i] = w[i - 1] * (i + 2);
 long sum = 0;
 for (char c : s.toCharArray()) sum += w[c - 'A'];
 System.out.println(sum);
 }
}
```

**Why it works:** Table lookup per character; sum in long.

Verified against 3 test cases: `ACC⏎` → `25`; `AABBC⏎` → `20`; `D⏎` → `60`

### Exercise 2 · Smallest string

**Ex2.java**

```java
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long weight = sc.nextLong();
 List<Long> w = new ArrayList<>();
 long cur = 1;
 w.add(cur);
 for (int n = 2; n <= 26; n++) {
 if (cur > weight / (n + 1)) break;
 cur *= (n + 1);
 w.add(cur);
 }
 List<Character> out = new ArrayList<>();
 for (int i = w.size() - 1; i >= 0; i--) {
 while (weight >= w.get(i)) {
 out.add((char) ('A' + i));
 weight -= w.get(i);
 }
 }
 Collections.sort(out);
 StringBuilder sb = new StringBuilder();
 for (char c : out) sb.append(c);
 System.out.println(sb);
 }
}
```

**Why it works:** Each weight divides the next, so greedy gives the minimum count; ascending order gives the smallest string.

Verified against 4 test cases: `25⏎` → `ACC`; `20⏎` → `AABBC`; `1⏎` → `A`

### Exercise 3 · Normalise a string

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 // Normalise: print the shortest, lexicographically smallest string with the same weight as s.
 // Carry like a number system: (letter i+1 has weight (i+2) times letter i), so (i+2) copies of
     letter i
 // become one copy of letter i+1.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 long[] count = new long[27];
 for (char c : s.toCharArray()) count[c - 'A']++;
 for (int i = 0; i < 25; i++) {
 long base = i + 3; // weight(letter i+1) / weight(letter i): 3 for A->B, 4 for B->C, ...
 count[i + 1] += count[i] / base;
 count[i] %= base;
 }
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < 26; i++) {
 for (long k = 0; k < count[i]; k++) sb.append((char) ('A' + i));
 }
 System.out.println(sb);
 }
}
```

**Why it works:** A mixed-radix number system: carry count[i] / (i + 2) into the next letter and keep the remainder.

Verified against 4 test cases: `AAAAB⏎` → `ABB`; `AAA⏎` → `B`; `BBBB⏎` → `C`

## 5.22 Move Hyphen to the Beginning

### Exercise 1 · First hyphen to the front

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.nextLine();
 int i = s.indexOf('-');
 System.out.println(i < 0 ? s : "-" + s.substring(0, i) + s.substring(i + 1));
 }
}
```

**Why it works:** Rebuild the string as "-" + before + after.

Verified against 3 test cases: `face-prep⏎` → `-faceprep`; `abc⏎` → `abc`; `a-b-c⏎` → `-ab-c`

### Exercise 2 · All hyphens to the front

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.nextLine();
 StringBuilder h = new StringBuilder();
 StringBuilder o = new StringBuilder();
 for (char c : s.toCharArray()) (c == '-' ? h : o).append(c);
 System.out.println(h.append(o));
 }
}
```

**Why it works:** A stable partition into hyphens and non-hyphens, concatenated.

Verified against 3 test cases: `a-b-c⏎` → `--abc`; `---⏎` → `---`; `abc⏎` → `abc`

### Exercise 3 · Minimum adjacent swaps

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 // Minimum number of ADJACENT swaps to bring all hyphens to the front:
 // each hyphen must jump over every non-hyphen character that stands before it.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.nextLine();
 long nonHyphensSeen = 0;
 long swaps = 0;
 for (char c : s.toCharArray()) {
 if (c == '-') swaps += nonHyphensSeen;
 else nonHyphensSeen++;
 }
 System.out.println(swaps);
 }
}
```

**Why it works:** Hyphens never need to cross each other (that would be a wasted swap), so the cost is the number of (non-hyphen, hyphen) inversions.

Verified against 3 test cases: `a-b-c⏎` → `3`; `--ab⏎` → `0`; `ab--⏎` → `4`

## 5.23 Strobogrammatic Number

### Exercise 1 · Is it strobogrammatic?

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 String from = "01689";
 String to = "01986";
 int l = 0, r = s.length() - 1;
 boolean ok = true;
 while (l <= r) {
 int i = from.indexOf(s.charAt(l));
 if (i < 0 || to.charAt(i) != s.charAt(r)) {
 ok = false;
 break;
 }
 l++;
 r--;
 }
 System.out.println(ok ? "YES" : "NO");
 }
}
```

**Why it works:** The digit at the left pointer must rotate into the digit at the right pointer; the middle digit must be 0, 1 or 8.

Verified against 5 test cases: `69⏎` → `YES`; `818⏎` → `YES`; `962⏎` → `NO`

### Exercise 2 · How many of length n?

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 // Count of n-digit strobogrammatic numbers (n >= 1, no leading zero; the single digit 0 counts
     for n = 1).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long count;
 if (n == 1) {
 count = 3; // 0, 1, 8
 } else {
 count = 4; // outer pair: 11, 69, 88, 96
 for (int i = 1; i < n / 2; i++) count *= 5; // each inner pair: 00, 11, 69, 88, 96
 if (n % 2 == 1) count *= 3; // middle digit: 0, 1, 8
 }
 System.out.println(count);
 }
}
```

**Why it works:** Multiplication principle: 4 · 5^(pairs−1) · (3 if n is odd).

Verified against 4 test cases: `4⏎` → `20`; `1⏎` → `3`; `3⏎` → `12`

### Exercise 3 · Strobogrammatic in a range

**Ex3.java**

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ex3 {
 static List<String> build(int n, int total) {
 if (n == 0) return new ArrayList<>(List.of(""));
 if (n == 1) return new ArrayList<>(List.of("0", "1", "8"));
 List<String> res = new ArrayList<>();
 for (String inner : build(n - 2, total)) {
 if (n != total) res.add("0" + inner + "0");
 res.add("1" + inner + "1");
 res.add("6" + inner + "9");
 res.add("8" + inner + "8");
 res.add("9" + inner + "6");
 }
 return res;
 }

 // Count strobogrammatic numbers in [low, high] given as strings of up to 15 digits.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String low = sc.next();
 String high = sc.next();
 int count = 0;
 for (int len = low.length(); len <= high.length(); len++) {
 for (String s : build(len, len)) {
 if (len == low.length() && s.compareTo(low) < 0) continue; // same length: string order =
     numeric order
 if (len == high.length() && s.compareTo(high) > 0) continue;
 count++;
 }
 }
 System.out.println(count);
 }
}
```

**Why it works:** Generate all candidates for each length and filter the ones outside the bounds.

Verified against 3 test cases: `50 100⏎` → `3`; `0 0⏎` → `1`; `1 1000⏎` → `18`

## 5.24 Lexicographically First Palindromic Rearrangement

### Exercise 1 · Can it be a palindrome?

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next().toLowerCase();
 int[] freq = new int[256];
 for (char c : s.toCharArray()) freq[c]++;
 int odd = 0;
 for (int f : freq) if (f % 2 == 1) odd++;
 System.out.println(odd <= 1 ? "YES" : "NO");
 }
}
```

**Why it works:** At most one letter may have an odd frequency.

Verified against 3 test cases: `aabb⏎` → `YES`; `apple⏎` → `NO`; `Malayalam⏎` → `YES`

### Exercise 2 · Build the smallest palindrome

Ex2.java

```java
import java.util.Scanner;
import java.util.TreeMap;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 TreeMap<Character, Integer> f = new TreeMap<>();
 for (char c : sc.next().toLowerCase().toCharArray()) f.merge(c, 1, Integer::sum);
 String mid = "";
 int odd = 0;
 for (var e : f.entrySet()) {
 if (e.getValue() % 2 == 1) {
 odd++;
 mid = String.valueOf(e.getKey());
 }
 }
 if (odd > 1) {
 System.out.println("no palindromic string");
 return;
 }
 StringBuilder front = new StringBuilder();
 for (var e : f.entrySet()) front.append(String.valueOf(e.getKey()).repeat(e.getValue() / 2));
 System.out.println(front + mid + new StringBuilder(front).reverse());
 }
}
```

**Why it works:** Front half sorted, optional odd letter in the middle, mirrored back half.

Verified against 4 test cases: `Malayalam⏎` → `aalmymlaa`; `aabbcc⏎` → `abccba`; `apple⏎` → `no palindromic string`

### Exercise 3 · How many palindromes?

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Number of DISTINCT palindromic permutations: arrange the first half freely.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next().toLowerCase();
 int[] freq = new int[256];
 for (char c : s.toCharArray()) freq[c]++;
 int odd = 0;
 for (int f : freq) if (f % 2 == 1) odd++;
 if (odd > 1) {
 System.out.println(0);
 return;
 }
 long[] fact = new long[21];
 fact[0] = 1;
 for (int i = 1; i <= 20; i++) fact[i] = fact[i - 1] * i;
 int half = s.length() / 2;
 long ways = fact[half];
 for (int f : freq) ways /= fact[f / 2];
 System.out.println(ways);
 }
}
```

**Why it works:** The palindrome is determined by its left half; count the distinct arrangements of the half-multiset.

Verified against 4 test cases: `aabbcc⏎` → `6`; `aabb⏎` → `2`; `abc⏎` → `0`

## 5.25 Manacher's Algorithm (Longest Palindromic Substring)

### Exercise 1 · Is it a palindrome?

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 boolean ok = true;
 for (int l = 0, r = s.length() - 1; l < r; l++, r--) {
 if (s.charAt(l) != s.charAt(r)) {
 ok = false;
 break;
 }
 }
 System.out.println(ok ? "YES" : "NO");
 }
}
```

**Why it works:** Compare the characters at both ends and move inwards; any mismatch decides NO.

Verified against 3 test cases: `racecar⏎` → `YES`; `abca⏎` → `NO`; `a⏎` → `YES`

### Exercise 2 · Longest palindromic substring (expand)

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 int bestStart = 0, bestLen = 1;
 for (int c = 0; c < s.length(); c++) {
 for (int t = 0; t < 2; t++) {
 int l = c, r = c + t;
 while (l >= 0 && r < s.length() && s.charAt(l) == s.charAt(r)) {
 l--;
 r++;
 }
 if (r - l - 1 > bestLen) {
 bestLen = r - l - 1;
 bestStart = l + 1;
 }
 }
 }
 System.out.println(s.substring(bestStart, bestStart + bestLen));
 }
}
```

**Why it works:** Centre expansion with odd and even centres; O(n²).

Verified against 4 test cases: `cbbd⏎` → `bb`; `babad⏎` → `bab`; `a⏎` → `a`

### Exercise 3 · Count palindromic substrings

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 // Count ALL palindromic substrings (by position) in O(n) using Manacher's radii.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 StringBuilder sb = new StringBuilder("#");
 for (char c : s.toCharArray()) sb.append(c).append('#');
 String t = sb.toString();
 int n = t.length();
 int[] rad = new int[n];
 int center = 0, right = 0;
 long total = 0;
 for (int i = 0; i < n; i++) {
 if (i < right) rad[i] = Math.min(right - i, rad[2 * center - i]);
 while (i - rad[i] - 1 >= 0 && i + rad[i] + 1 < n && t.charAt(i - rad[i] - 1) == t.charAt(i +
     rad[i] + 1)) rad[i]++;
 if (i + rad[i] > right) {
 center = i;
 right = i + rad[i];
 }
 total += (rad[i] + 1) / 2; // a radius of r in the transformed string gives ceil(r/2) palindromes
 }
 System.out.println(total);
 }
}
```

**Why it works:** Manacher's radii give, for every centre, the number of palindromes centred there; summing them is O(n).

Verified against 3 test cases: `abba⏎` → `6`; `aaa⏎` → `6`; `abc⏎` → `3`

## 5.26 Natural Sort Order

### Exercise 1 · Sort file names naturally

**Ex1.java**

```java
import java.math.BigInteger;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ex1 {
 static List<String> chunks(String s) {
 List<String> out = new ArrayList<>();
 int i = 0;
 while (i < s.length()) {
 int j = i;
 boolean d = Character.isDigit(s.charAt(i));
 while (j < s.length() && Character.isDigit(s.charAt(j)) == d) j++;
 out.add(s.substring(i, j));
 i = j;
 }
 return out;
 }

 static int cmp(String a, String b) {
 List<String> x = chunks(a), y = chunks(b);
 for (int i = 0; i < Math.min(x.size(), y.size()); i++) {
 String p = x.get(i), q = y.get(i);
 int c = (Character.isDigit(p.charAt(0)) && Character.isDigit(q.charAt(0))) ? new BigInteger(p)
 .compareTo(new BigInteger(q)) : p.compareToIgnoreCase(q);
 if (c != 0) return c;
 }
 return Integer.compare(x.size(), y.size());
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 List<String> names = new ArrayList<>();
 for (int i = 0; i < n; i++) names.add(sc.next());
 names.sort(Ex1::cmp);
 for (String s : names) System.out.println(s);
 }
}
```

**Why it works:** The comparator from the lesson, applied with List.sort.

Verified against 3 test cases: `4⏎img12.png⏎img10.png⏎img2.png⏎IMG1.png⏎` → `IMG1.png⏎img2.png⏎img10.png⏎img12.png`; `2⏎b2⏎a10⏎` → `a10⏎b2`; `3⏎x9⏎x10⏎x1⏎` → `x1⏎x9⏎x10`

### Exercise 2 · Sort version numbers

**Ex2.java**

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ex2 {
 // Sort version numbers such as 1.2.10 and 1.10 numerically, field by field.
 static int cmp(String a, String b) {
 String[] x = a.split("\\.");
 String[] y = b.split("\\.");
 for (int i = 0; i < Math.max(x.length, y.length); i++) {
 long p = i < x.length ? Long.parseLong(x[i]) : 0;
 long q = i < y.length ? Long.parseLong(y[i]) : 0;
 if (p != q) return Long.compare(p, q);
 }
 return 0;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 List<String> v = new ArrayList<>();
 for (int i = 0; i < n; i++) v.add(sc.next());
 v.sort(Ex2::cmp);
 System.out.println(String.join(" ", v));
 }
}
```

**Why it works:** Numeric comparison per dotted component; pad the shorter version with zeros.

Verified against 3 test cases: `4⏎1.10⏎1.2.10⏎1.2.9⏎1.9⏎` → `1.2.9 1.2.10 1.9 1.10`; `2⏎2.0⏎1.9.9⏎` → `1.9.9 2.0`; `2⏎1.0⏎1.0.0⏎` → `1.0 1.0.0`

### Exercise 3 · Natural order with deterministic ties

**Ex3.java**

```java
import java.math.BigInteger;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ex3 {
 static List<String> chunks(String s) {
 List<String> out = new ArrayList<>();
 int i = 0;
 while (i < s.length()) {
 int j = i;
 boolean d = Character.isDigit(s.charAt(i));
 while (j < s.length() && Character.isDigit(s.charAt(j)) == d) j++;
 out.add(s.substring(i, j));
 i = j;
 }
 return out;
 }

 // Natural order, ignoring case. When two names are equal numerically (file01 vs file1),
 // the one with FEWER leading zeros (the shorter text) comes first, then plain compareTo.
 static int cmp(String a, String b) {
 List<String> x = chunks(a), y = chunks(b);
 for (int i = 0; i < Math.min(x.size(), y.size()); i++) {
 String p = x.get(i), q = y.get(i);
 int c = (Character.isDigit(p.charAt(0)) && Character.isDigit(q.charAt(0))) ? new BigInteger(p)
 .compareTo(new BigInteger(q)) : p.compareToIgnoreCase(q);
 if (c != 0) return c;
 }
 if (x.size() != y.size()) return Integer.compare(x.size(), y.size());
 if (a.length() != b.length()) return Integer.compare(a.length(), b.length());
 return a.compareTo(b);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 List<String> names = new ArrayList<>();
 for (int i = 0; i < n; i++) names.add(sc.next());
 names.sort(Ex3::cmp);
 System.out.println(String.join(" ", names));
 }
}
```

**Why it works:** Adding tie-breakers makes the comparator a strict total order, so the output is deterministic.

Verified against 3 test cases: `4⏎file01⏎file1⏎File2⏎file10⏎` → `file1 file01 File2 file10`; `3⏎a1⏎A1⏎a01⏎` → `A1 a1 a01`; `1⏎zzz⏎` → `zzz`

## 5.27 Maneuvering (Counting Grid Paths)

### Exercise 1 · Paths in a rectangle

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int m = sc.nextInt();
 int n = sc.nextInt();
 long[][] dp = new long[m][n];
 for (int i = 0; i < m; i++)
 for (int j = 0; j < n; j++)
 dp[i][j] = (i == 0 || j == 0) ? 1 : dp[i - 1][j] + dp[i][j - 1];
 System.out.println(dp[m - 1][n - 1]);
 }
}
```

**Why it works:** Bottom-up DP with long values.

Verified against 3 test cases: `3 7⏎` → `28`; `1 5⏎` → `1`; `18 18⏎` → `2333606220`

### Exercise 2 · Paths with obstacles

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 // Grid of 0 (free) and 1 (blocked). Count right/down paths from top-left to bottom-right.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int r = sc.nextInt();
 int c = sc.nextInt();
 int[][] g = new int[r][c];
 for (int i = 0; i < r; i++) for (int j = 0; j < c; j++) g[i][j] = sc.nextInt();
 long[][] dp = new long[r][c];
 for (int i = 0; i < r; i++) {
 for (int j = 0; j < c; j++) {
 if (g[i][j] == 1) {
 dp[i][j] = 0;
 } else if (i == 0 && j == 0) {
 dp[i][j] = 1;
 } else {
 dp[i][j] = (i > 0 ? dp[i - 1][j] : 0) + (j > 0 ? dp[i][j - 1] : 0);
 }
 }
 }
 System.out.println(dp[r - 1][c - 1]);
 }
}
```

**Why it works:** Same recurrence; blocked cells are forced to 0 so no path goes through them.

Verified against 3 test cases: `3 3⏎0 0 0⏎0 1 0⏎0 0 0⏎` → `2`; `2 2⏎0 1⏎1 0⏎` → `0`; `1 1⏎0⏎` → `1`

### Exercise 3 · Huge grid modulo a prime

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 static final long MOD = 1_000_000_007L;

 static long power(long b, long e) {
 long r = 1;
 b %= MOD;
 while (e > 0) {
 if ((e & 1) == 1) r = r * b % MOD;
 b = b * b % MOD;
 e >>= 1;
 }
 return r;
 }

 // C(m+n-2, m-1) modulo 1e9+7 for m, n up to 100000: factorials and a modular inverse (Fermat).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int m = sc.nextInt();
 int n = sc.nextInt();
 int total = m + n - 2;
 long[] fact = new long[total + 1];
 fact[0] = 1;
 for (int i = 1; i <= total; i++) fact[i] = fact[i - 1] * i % MOD;
 long denom = fact[m - 1] * fact[n - 1] % MOD;
 System.out.println(fact[total] * power(denom, MOD - 2) % MOD);
 }
}
```

**Why it works:** nCr mod p = n! · inverse(r!) · inverse((n−r)!) mod p, with the inverse by fast exponentiation.

Verified against 3 test cases: `3 7⏎` → `28`; `1 1⏎` → `1`; `100000 100000⏎` → `691090292`

## 5.28 Sorted Distinct Permutations

### Exercise 1 · How many distinct permutations?

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 // Number of distinct permutations = n! / (c1! * c2! * ...)
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 int[] freq = new int[256];
 for (char c : s.toCharArray()) freq[c]++;
 long[] f = new long[21];
 f[0] = 1;
 for (int i = 1; i <= 20; i++) f[i] = f[i - 1] * i;
 long ways = f[s.length()];
 for (int x : freq) ways /= f[x];
 System.out.println(ways);
 }
}
```

**Why it works:** Multinomial coefficient from letter counts.

Verified against 4 test cases: `AAB⏎` → `3`; `ABCD⏎` → `24`; `AAAA⏎` → `1`

### Exercise 2 · Print them sorted

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 static StringBuilder out = new StringBuilder();

 static void build(int[] counts, char[] cur, int pos) {
 if (pos == cur.length) {
 out.append(new String(cur)).append('\n');
 return;
 }
 for (int c = 0; c < counts.length; c++) {
 if (counts[c] == 0) continue;
 counts[c]--;
 cur[pos] = (char) c;
 build(counts, cur, pos + 1);
 counts[c]++;
 }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 int[] counts = new int[256];
 for (char c : s.toCharArray()) counts[c]++;
 build(counts, new char[s.length()], 0);
 System.out.print(out);
 }
}
```

**Why it works:** Count-based backtracking enumerates in sorted order without duplicates.

Verified against 3 test cases: `ABA⏎` → `AAB⏎ABA⏎BAA`; `BAC⏎` → `ABC⏎ACB⏎BAC⏎BCA⏎CAB⏎CBA`; `X⏎` → `X`

### Exercise 3 · The k-th permutation

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 static long[] fact = new long[21];

 // number of distinct arrangements of the remaining letters
 static long arrangements(int[] counts, int remaining) {
 long r = fact[remaining];
 for (int c : counts) r /= fact[c];
 return r;
 }

 // k-th (1-based) distinct permutation in sorted order, without generating the others.
 public static void main(String[] args) {
 fact[0] = 1;
 for (int i = 1; i <= 20; i++) fact[i] = fact[i - 1] * i;
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 long k = sc.nextLong();
 int[] counts = new int[256];
 for (char c : s.toCharArray()) counts[c]++;
 int n = s.length();
 if (k > arrangements(counts, n)) {
 System.out.println("NONE");
 return;
 }
 StringBuilder sb = new StringBuilder();
 for (int pos = 0; pos < n; pos++) {
 for (int c = 0; c < 256; c++) {
 if (counts[c] == 0) continue;
 counts[c]--;
 long block = arrangements(counts, n - pos - 1); // permutations that start with this letter here
 if (k <= block) {
 sb.append((char) c);
 break;
 }
 k -= block;
 counts[c]++;
 }
 }
 System.out.println(sb);
 }
}
```

**Why it works:** Ranking via multinomial counts: O(n · σ) instead of enumerating up to n! permutations.

Verified against 3 test cases: `DBCA 10⏎` → `BCDA`; `AAB 3⏎` → `BAA`; `AB 3⏎` → `NONE`

## 5.29 N-Queens (Backtracking)

### Exercise 1 · Validate a placement

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 // Read n and, for each row, the column of its queen. Print VALID if no two queens attack each other.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] col = new int[n];
 for (int i = 0; i < n; i++) col[i] = sc.nextInt();
 boolean ok = true;
 for (int i = 0; i < n && ok; i++) {
 for (int j = i + 1; j < n; j++) {
 if (col[i] == col[j] || Math.abs(col[i] - col[j]) == j - i) {
 ok = false;
 break;
 }
 }
 }
 System.out.println(ok ? "VALID" : "INVALID");
 }
}
```

**Why it works:** Check every pair of rows: equal columns or equal row and column distance mean attack.

Verified against 3 test cases: `4⏎2 4 1 3⏎` → `VALID`; `4⏎1 2 3 4⏎` → `INVALID`; `1⏎1⏎` → `VALID`

### Exercise 2 · Count solutions

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 static int n, solutions;
 static boolean[] c, d1, d2;

 static void place(int r) {
 if (r == n) {
 solutions++;
 return;
 }
 for (int col = 0; col < n; col++) {
 int a = r - col + n - 1, b = r + col;
 if (c[col] || d1[a] || d2[b]) continue;
 c[col] = d1[a] = d2[b] = true;
 place(r + 1);
 c[col] = d1[a] = d2[b] = false;
 }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 n = sc.nextInt();
 c = new boolean[n];
 d1 = new boolean[2 * n];
 d2 = new boolean[2 * n];
 place(0);
 System.out.println(solutions);
 }
}
```

**Why it works:** Exhaustive backtracking with O(1) conflict checks.

Verified against 4 test cases: `6⏎` → `4`; `8⏎` → `92`; `3⏎` → `0`

### Exercise 3 · Lexicographically first solution

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 static int n;
 static int[] pos;
 static boolean[] c, d1, d2;

 static boolean place(int r) {
 if (r == n) return true;
 for (int col = 0; col < n; col++) {
 int a = r - col + n - 1, b = r + col;
 if (c[col] || d1[a] || d2[b]) continue;
 c[col] = d1[a] = d2[b] = true;
 pos[r] = col;
 if (place(r + 1)) return true;
 c[col] = d1[a] = d2[b] = false;
 }
 return false;
 }

 // The lexicographically first solution as "column of the queen in each row" (1-based), or NONE.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 n = sc.nextInt();
 pos = new int[n];
 c = new boolean[n];
 d1 = new boolean[2 * n];
 d2 = new boolean[2 * n];
 if (!place(0)) {
 System.out.println("NONE");
 return;
 }
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) sb.append(i == 0 ? "" : " ").append(pos[i] + 1);
 System.out.println(sb);
 }
}
```

**Why it works:** Depth-first search trying columns left to right returns the lexicographically smallest solution first.

Verified against 3 test cases: `8⏎` → `1 5 8 6 3 7 2 4`; `4⏎` → `2 4 1 3`; `3⏎` → `NONE`

## 5.30 Maze Solving (Rat in a Maze)

### Exercise 1 · How many cells can the rat reach?

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 // Count the cells reachable from the entry (2) moving up/down/left/right through non-walls
     (flood fill).
 static int r, c;
 static int[][] g;
 static boolean[][] seen;

 static int fill(int i, int j) {
 if (i < 0 || j < 0 || i >= r || j >= c || g[i][j] == 1 || seen[i][j]) return 0;
 seen[i][j] = true;
 return 1 + fill(i + 1, j) + fill(i - 1, j) + fill(i, j + 1) + fill(i, j - 1);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 r = sc.nextInt();
 c = sc.nextInt();
 g = new int[r][c];
 seen = new boolean[r][c];
 int si = 0, sj = 0;
 for (int i = 0; i < r; i++) for (int j = 0; j < c; j++) {
 g[i][j] = sc.nextInt();
 if (g[i][j] == 2) { si = i; sj = j; }
 }
 System.out.println(fill(si, sj));
 }
}
```

**Why it works:** A flood fill counts every cell entered by the DFS exactly once.

Verified against 3 test cases: `3 3⏎2 0 1⏎1 0 1⏎1 0 3⏎` → `5`; `1 3⏎2 1 3⏎` → `1`; `2 2⏎2 0⏎0 3⏎` → `4`

### Exercise 2 · Is there a way out?

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 static int r, c;
 static int[][] g;
 static boolean[][] seen;

 static boolean reach(int i, int j) {
 if (i < 0 || j < 0 || i >= r || j >= c || g[i][j] == 1 || seen[i][j]) return false;
 if (g[i][j] == 3) return true;
 seen[i][j] = true;
 return reach(i + 1, j) || reach(i - 1, j) || reach(i, j + 1) || reach(i, j - 1);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 r = sc.nextInt();
 c = sc.nextInt();
 g = new int[r][c];
 seen = new boolean[r][c];
 int si = 0, sj = 0;
 for (int i = 0; i < r; i++) for (int j = 0; j < c; j++) {
 g[i][j] = sc.nextInt();
 if (g[i][j] == 2) { si = i; sj = j; }
 }
 System.out.println(reach(si, sj) ? "YES" : "NO");
 }
}
```

**Why it works:** Depth-first search with a visited array; true when the exit cell is reached.

Verified against 3 test cases: `3 3⏎2 1 3⏎0 1 0⏎0 0 0⏎` → YES; `1 3⏎2 1 3⏎` → NO; `1 2⏎2 3⏎` → YES

### Exercise 3 · Shortest path

**Ex3.java**

```java
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Scanner;

public class Ex3 {
 // Length (number of moves) of the SHORTEST route from entry (2) to exit (3), or -1.
     Breadth-first search.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int r = sc.nextInt();
 int c = sc.nextInt();
 int[][] g = new int[r][c];
 int si = 0, sj = 0;
 for (int i = 0; i < r; i++) for (int j = 0; j < c; j++) {
 g[i][j] = sc.nextInt();
 if (g[i][j] == 2) { si = i; sj = j; }
 }
 int[][] dist = new int[r][c];
 for (int[] row : dist) java.util.Arrays.fill(row, -1);
 Deque<int[]> q = new ArrayDeque<>();
 q.add(new int[] {si, sj});
 dist[si][sj] = 0;
 int[] dr = {1, -1, 0, 0};
 int[] dc = {0, 0, 1, -1};
 while (!q.isEmpty()) {
 int[] cur = q.poll();
 if (g[cur[0]][cur[1]] == 3) {
 System.out.println(dist[cur[0]][cur[1]]);
 return;
 }
 for (int d = 0; d < 4; d++) {
 int ni = cur[0] + dr[d], nj = cur[1] + dc[d];
 if (ni < 0 || nj < 0 || ni >= r || nj >= c || g[ni][nj] == 1 || dist[ni][nj] != -1) continue;
 dist[ni][nj] = dist[cur[0]][cur[1]] + 1;
 q.add(new int[] {ni, nj});
 }
 }
 System.out.println(-1);
 }
}
```

**Why it works:** BFS visits cells in order of distance; the first time the exit is dequeued its distance is minimal.

Verified against 3 test cases: `3 3⏎2 0 0⏎1 1 0⏎3 0 0⏎` → `6`; `1 3⏎2 1 3⏎` → `-1`; `1 2⏎2 3⏎` → `1`

## 5.31 Hamiltonian Cycle

### Exercise 1 · Verify a cycle

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 // Input: v, adjacency matrix, then a vertex sequence of length v+1. Is it a Hamiltonian cycle?
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int v = sc.nextInt();
 int[][] g = new int[v][v];
 for (int i = 0; i < v; i++) for (int j = 0; j < v; j++) g[i][j] = sc.nextInt();
 int[] p = new int[v + 1];
 for (int i = 0; i <= v; i++) p[i] = sc.nextInt();
 boolean ok = p[0] == p[v];
 boolean[] seen = new boolean[v];
 for (int i = 0; i < v && ok; i++) {
 if (seen[p[i]]) ok = false;
 seen[p[i]] = true;
 if (g[p[i]][p[i + 1]] == 0) ok = false;
 }
 System.out.println(ok ? "YES" : "NO");
 }
}
```

**Why it works:** Linear scan with a seen array.

Verified against 3 test cases: `4⏎0 1 0 1⏎1 0 1 0⏎0 1 0 1⏎1 0 1 0⏎0 1 2 3 0⏎` → `YES`; `4⏎0 1 0 1⏎1 0 1 0⏎0 1 0 1⏎1 0 1 0⏎0 2 1 3 0⏎` → `NO`; `3⏎0 1 1⏎1 0 1⏎1 1 0⏎0 1 2 0⏎` → `YES`

### Exercise 2 · Find a cycle

**Ex2.java**

```java
import java.util.Arrays;
import java.util.Scanner;

public class Ex2 {
 static int v;
 static int[][] g;
 static int[] path;

 static boolean extend(int pos) {
 if (pos == v) return g[path[v - 1]][path[0]] == 1;
 for (int c = 1; c < v; c++) {
 boolean ok = g[path[pos - 1]][c] == 1;
 for (int i = 0; i < pos && ok; i++) if (path[i] == c) ok = false;
 if (!ok) continue;
 path[pos] = c;
 if (extend(pos + 1)) return true;
 path[pos] = -1;
 }
 return false;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 v = sc.nextInt();
 g = new int[v][v];
 for (int i = 0; i < v; i++) for (int j = 0; j < v; j++) g[i][j] = sc.nextInt();
 path = new int[v];
 Arrays.fill(path, -1);
 path[0] = 0;
 if (!extend(1)) {
 System.out.println("NO");
 return;
 }
 StringBuilder sb = new StringBuilder();
 for (int x : path) sb.append(x).append(' ');
 sb.append(0);
 System.out.println("YES " + sb);
 }
}
```

**Why it works:** Depth-first backtracking, candidates in increasing order, close the cycle at the end.

Verified against 3 test cases: `5⏎0 1 0 1 0⏎1 0 1 1 1⏎0 1 0 0 1⏎1 1 0 0 0⏎0 1 1 0 0⏎` → `NO`; `5⏎0 1 0 1 0⏎1 0 1 1 1⏎0 1 0 0 1⏎1 1 0 0 1⏎0 1 1 1 0⏎` → `YES 0 1 2 4 3 0`; `3⏎0 1 1⏎1 0 1⏎1 1 0⏎` → `YES 0 1 2 0`

### Exercise 3 · Count Hamiltonian cycles

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 static int v, count;
 static int[][] g;
 static boolean[] used;

 static void dfs(int cur, int placed) {
 if (placed == v) {
 if (g[cur][0] == 1) count++;
 return;
 }
 for (int nxt = 1; nxt < v; nxt++) {
 if (!used[nxt] && g[cur][nxt] == 1) {
 used[nxt] = true;
 dfs(nxt, placed + 1);
 used[nxt] = false;
 }
 }
 }

 // Count Hamiltonian cycles that start and end at vertex 0, counting a cycle and its reverse separately.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 v = sc.nextInt();
 g = new int[v][v];
 for (int i = 0; i < v; i++) for (int j = 0; j < v; j++) g[i][j] = sc.nextInt();
 used = new boolean[v];
 used[0] = true;
 dfs(0, 1);
 System.out.println(count);
 }
}
```

**Why it works:** Count every complete path that closes back to vertex 0.

Verified against 3 test cases: `4⏎0 1 1 1⏎1 0 1 1⏎1 1 0 1⏎1 1 1 0⏎` → `6`; `3⏎0 1 1⏎1 0 1⏎1 1 0⏎` → `2`; `4⏎0 1 0 0⏎1 0 1 0⏎0 1 0 1⏎0 0 1 0⏎` → `0`

## 5.32 Knight's Tour & Warnsdorff's Rule

### Exercise 1 · Verify a tour

**Ex1.java**

```java
import java.util.Scanner;

public class Ex1 {
 // Input: n, then an n x n board of move numbers. Is it a valid knight's tour (1..n*n, each step
     a knight move)?
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] px = new int[n * n + 1];
 int[] py = new int[n * n + 1];
 boolean ok = true;
 for (int y = 0; y < n; y++) {
 for (int x = 0; x < n; x++) {
 int v = sc.nextInt();
 if (v < 1 || v > n * n || px[v] != 0) { // out of range or a repeated number
 ok = false;
 } else {
 px[v] = x + 1;
 py[v] = y + 1;
 }
 }
 }
 for (int v = 1; v < n * n && ok; v++) {
 if (px[v] == 0 || px[v + 1] == 0) {
 ok = false;
 break;
 }
 int dx = Math.abs(px[v] - px[v + 1]);
 int dy = Math.abs(py[v] - py[v + 1]);
 if (!((dx == 1 && dy == 2) || (dx == 2 && dy == 1))) ok = false;
 }
 System.out.println(ok ? "YES" : "NO");
 }
}
```

**Why it works:** Positions per number, then check each consecutive pair.

Verified against 3 test cases: `5⏎1 6 15 10 21⏎14 9 20 5 16⏎19 2 7 22 11⏎8 13 24 17 4⏎25 18 3 12 23⏎` → YES; `3⏎1 2 3⏎4 5 6⏎7 8 9⏎` → NO; `1⏎1⏎` → YES

### Exercise 2 · Degree of a square

**Ex2.java**

```java
import java.util.Scanner;

public class Ex2 {
 // How many squares can a knight on (x, y) reach on an empty n x n board? (the "degree"
     Warnsdorff uses)
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int x = sc.nextInt();
 int y = sc.nextInt();
 int[] dx = {1, 1, 2, 2, -1, -1, -2, -2};
 int[] dy = {2, -2, 1, -1, 2, -2, 1, -1};
 int c = 0;
 for (int m = 0; m < 8; m++) {
 int nx = x + dx[m], ny = y + dy[m];
 if (nx >= 0 && ny >= 0 && nx < n && ny < n) c++;
 }
 System.out.println(c);
 }
}
```

**Why it works:** Count the legal moves; corners have 2, the centre 8.

Verified against 4 test cases: `8 0 0⏎` → `2`; `8 3 3⏎` → `8`; `5 2 2⏎` → `8`

### Exercise 3 · Reachable squares

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 // Count the squares of an n x n board that a knight can reach from (0,0) in exactly k moves
 // (squares may be visited again). Plain breadth-first layers.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int k = sc.nextInt();
 int[] dx = {1, 1, 2, 2, -1, -1, -2, -2};
 int[] dy = {2, -2, 1, -1, 2, -2, 1, -1};
 boolean[][] cur = new boolean[n][n];
 cur[0][0] = true;
 for (int step = 0; step < k; step++) {
 boolean[][] next = new boolean[n][n];
 for (int y = 0; y < n; y++) {
 for (int x = 0; x < n; x++) {
 if (!cur[y][x]) continue;
 for (int m = 0; m < 8; m++) {
 int nx = x + dx[m], ny = y + dy[m];
 if (nx >= 0 && ny >= 0 && nx < n && ny < n) next[ny][nx] = true;
 }
 }
 }
 cur = next;
 }
 int c = 0;
 for (boolean[] row : cur) for (boolean b : row) if (b) c++;
 System.out.println(c);
 }
}
```

**Why it works:** Layer-by-layer breadth-first expansion: the set of squares reachable in exactly i moves.

Verified against 3 test cases: `8 2⏎` → `10`; `8 1⏎` → `2`; `8 0⏎` → `1`

## 5.33 Selection Sort

### Exercise 1 · Sort and count swaps

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // Selection sort; print how many swaps that actually move something (minIndex != i) were performed.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 int swaps = 0;
 for (int i = 0; i < n - 1; i++) {
 int m = i;
 for (int j = i + 1; j < n; j++) if (a[j] < a[m]) m = j;
 if (m != i) {
 int t = a[m];
 a[m] = a[i];
 a[i] = t;
 swaps++;
 }
 }
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) sb.append(i == 0 ? "" : " ").append(a[i]);
 System.out.println(sb);
 System.out.println(swaps);
 }
}
```

**Why it works:** Standard selection sort with a counter around the swap.

Verified against 3 test cases: `5⏎64 25 12 22 11⏎` → `11 12 22 25 64⏎3`; `3⏎1 2 3⏎` → `1 2 3⏎0`; `4⏎4 3 2 1⏎` → `1 2 3 4⏎2`

### Exercise 2 · Largest first

SelEx2.java

```java
import java.util.Scanner;

public class SelEx2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 for (int i = 0; i < n - 1; i++) {
 int maxIndex = i;
 for (int j = i + 1; j < n; j++) {
 if (a[j] > a[maxIndex]) maxIndex = j;
 }
 int t = a[maxIndex];
 a[maxIndex] = a[i];
 a[i] = t;
 }
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) sb.append(i == 0 ? "" : " ").append(a[i]);
 System.out.println(sb);
 }
}
```

**Why it works:** Select the largest remaining value in each pass.

Verified against 3 test cases: `5⏎3 1 4 1 5⏎` → `5 4 3 1 1`; `1⏎7⏎` → `7`; `3⏎-1 -5 -3⏎` → `-1 -3 -5`

### Exercise 3 · K smallest, cheaply

**SelEx3.java**

```java
import java.util.Scanner;

public class SelEx3 {
 // The k smallest values in increasing order using only k selection passes: O(n * k).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int k = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 for (int i = 0; i < k; i++) {
 int m = i;
 for (int j = i + 1; j < n; j++) {
 if (a[j] < a[m]) m = j;
 }
 int t = a[m];
 a[m] = a[i];
 a[i] = t;
 }
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < k; i++) sb.append(i == 0 ? "" : " ").append(a[i]);
 System.out.println(sb);
 }
}
```

**Why it works:** Reuse selection: after k passes the first k positions hold the k smallest in order.

Verified against 3 test cases: `6 3⏎7 10 4 3 20 15⏎` → `3 4 7`; `3 3⏎3 2 1⏎` → `1 2 3`; `4 1⏎9 8 7 6⏎` → `6`

## 5.34 Quick Sort

### Exercise 1 · Count the partitions

**QuickSortEx1.java**

```java
import java.util.Scanner;

public class QuickSortEx1 {
 static int partition(int[] a, int low, int high) {
 int pivot = a[high];
 int i = low - 1;
 for (int j = low; j < high; j++) {
 if (a[j] <= pivot) {
 i++;
 int t = a[i];
 a[i] = a[j];
 a[j] = t;
 }
 }
 int t = a[i + 1];
 a[i + 1] = a[high];
 a[high] = t;
 return i + 1;
 }

 static void quickSort(int[] a, int low, int high) {
 if (low < high) {
 int p = partition(a, low, high);
 quickSort(a, low, p - 1);
 quickSort(a, p + 1, high);
 }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 quickSort(a, 0, n - 1);
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) sb.append(i == 0 ? "" : " ").append(a[i]);
 System.out.println(sb);
 }
}
```

**Why it works:** The lesson's algorithm with the output formatted as one line.

Verified against 3 test cases: `7⏎8 3 1 7 0 10 2⏎` → `0 1 2 3 7 8 10`; `1⏎5⏎` → `5`; `5⏎2 2 2 2 2⏎` → `2 2 2 2 2`

### Exercise 2 · Descending quick sort

**QuickSortEx2.java**

```java
import java.util.Scanner;

public class QuickSortEx2 {
 static void quick(int[] a, int lo, int hi) {
 if (lo >= hi) return;
 int pivot = a[hi], i = lo - 1;
 for (int j = lo; j < hi; j++) {
 if (a[j] >= pivot) { // descending order: larger values go to the left part
 i++;
 int t = a[i];
 a[i] = a[j];
 a[j] = t;
 }
 }
 int t = a[i + 1];
 a[i + 1] = a[hi];
 a[hi] = t;
 quick(a, lo, i);
 quick(a, i + 2, hi);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 quick(a, 0, n - 1);
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) sb.append(i == 0 ? "" : " ").append(a[i]);
 System.out.println(sb);
 }
}
```

**Why it works:** Partition by "≥ pivot goes left" and recurse as before.

Verified against 3 test cases: `5⏎3 1 4 1 5⏎` → `5 4 3 1 1`; `2⏎1 2⏎` → `2 1`; `1⏎0⏎` → `0`

### Exercise 3 · k-th smallest with quickselect

**Ex3.java**

```java
import java.util.Scanner;

public class Ex3 {
 // Quickselect: k-th smallest (1-based) without sorting everything. Average O(n).
 static int select(int[] a, int k) {
 int lo = 0, hi = a.length - 1;
 while (true) {
 int pivot = a[hi], i = lo - 1;
 for (int j = lo; j < hi; j++) {
 if (a[j] <= pivot) {
 i++;
 int t = a[i];
 a[i] = a[j];
 a[j] = t;
 }
 }
 int t = a[i + 1];
 a[i + 1] = a[hi];
 a[hi] = t;
 int p = i + 1;
 if (p == k - 1) return a[p];
 if (p > k - 1) hi = p - 1;
 else lo = p + 1;
 }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int k = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 System.out.println(select(a, k));
 }
}
```

**Why it works:** Quickselect discards the side that cannot contain the k-th element, so the work forms a geometric series ≈ 2n.

Verified against 3 test cases: `6 3⏎7 10 4 3 20 15⏎` → `7`; `3 1⏎9 8 7⏎` → `7`; `4 4⏎1 2 3 4⏎` → `4`

# Quiz answer key

## 5.1 Euclid's Algorithm (GCD)

| | Answer | Why |
|---|---|---|
| Q1 | B. 6 | 48 = 2·18 + 12, 18 = 1·12 + 6, 12 = 2·6 + 0 → gcd = 6. |
| Q2 | 12 | 84 mod 36 = 12 → (36, 12); 36 mod 12 = 0 → (12, 0). a = 12. |
| Q3 | line 4 | Line 4 runs after a was overwritten, so `a % b` is `b % b` = 0. Save the remainder first. |
| Q4 | 0 \| a \| b | gcd(a, 0) = a, otherwise gcd(a, b) = gcd(b, a mod b). |
| Q5 | B. O(log min(a, b)) | The remainder is more than halved every two steps, so only about $2 \cdot \log_2(\min)$ steps are needed. |
| Q6 | a after iteration 1 = 12; b after iteration 1 = 2; b after iteration 2 = 0; final a = 2 | (50, 12) → (12, 2) → (2, 0). The answer is the last a. |

## 5.2 Euler's Phi (Totient) Function

| | Answer | Why |
|---|---|---|
| Q1 | C. 16 | 17 is prime: every k from 1 to 16 is coprime to it. φ(p) = p − 1. |
| Q2 | A. 16 | 60 = $2^2 \cdot 3 \cdot 5$ → 60 × 1/2 × 2/3 × 4/5 = 16. (The lecture slide says 32 — that is a mistake.) |
| Q3 | 12 | 36 = $2^2 \cdot 3^2$ → 36 − 18 = 18 (p = 2); then 18 − 6 = 12 (p = 3). |
| Q4 | -= | result − result/p = result · (1 − 1/p). |
| Q5 | B. A prime factor larger than √n can remain | The loop only searches p ≤ √n. A single bigger prime factor can survive as the remaining value of n. |
| Q6 | φ(1) → 1; φ(9) → 6; φ(10) → 4; φ(13) → 12 | φ(9) = 9 − 3 = 6; φ(10) = 4 (1,3,7,9); φ(13) = 12 (prime). |

## 5.3 Chinese Remainder Theorem

| | Answer | Why |
|---|---|---|
| Q1 | A. 23 | 233 mod 105 = 23 and 23 checks out for all three. |
| Q2 | C. Pairwise coprime | Pairwise coprime moduli make each M_i invertible modulo m_i. |
| Q3 | 17 | Numbers ≡ 1 (mod 4): 1, 5, 9, 13, 17 → 17 % 5 = 2. |
| Q4 | B. M / m_i | M_i = M / m_i is the product of all the other moduli. |
| Q5 | C. no solution | x ≡ 1 (mod 6) forces x odd; x ≡ 2 (mod 4) forces x even. |

## 5.4 Combinations (nCr)

| | Answer | Why |
|---|---|---|
| Q1 | B. 15 | 6!/(2!·4!) = 720/(2·24) = 15. |
| Q2 | C. 30 | 6 · 5 = 30. Dividing by 2! gives the 15 combinations. |
| Q3 | 15 | i=1: 1·5/1 = 5; i=2: 5·6/2 = 15. |
| Q4 | B. fact(15) overflows int | 15! ≈ $1.3 \times 10^{12}$ > $2.1 \times 10^9$. |
| Q5 | A. C(n, n−r) | Choosing r to include is the same as choosing n−r to leave out. |

## 5.5 Karatsuba's Multiplication

| | Answer | Why |
|---|---|---|
| Q1 | B. 3 | ac, bd and (a+b)(c+d). |
| Q2 | A. (a+b)(c+d) − ac − bd | Expanding (a+b)(c+d) = ac + ad + bc + bd. |
| Q3 | C. O(n^1.585) | T(n) = 3T(n/2) + O(n) gives n^($\log_2$ 3). |
| Q4 | 2840 | 46 × 134 = 6164; 6164 − 672 − 2652 = 2840 = ad + bc. |
| Q5 | B. Karatsuba has overhead that dominates for small n | Recursion and extra additions cost more than 1-2 machine multiplications. |

## 5.6 Booth's Algorithm (signed binary multiplication)

| | Answer | Why |
|---|---|---|
| Q1 | C. 10 | "10" marks the start of a run of 1s. |
| Q2 | 1001 | 1000 + 0001 = 1001 = −7. |

| | Answer | Why |
|---|---|---|
| Q3 | B. 1110 | The sign bit (1) is copied into the vacated left position. |
| Q4 | A. 00010101 | 0001 0101 = 21 = 7 × 3. |
| Q5 | B. 11111111 | A single long run of 1s needs just one subtract (and the result is just a shift). |

## 5.7 Simple Sieve of Eratosthenes

| | Answer | Why |
|---|---|---|
| Q1 | B. Because every composite ≤ n has a prime factor ≤ √n | A composite m = a·b with a ≤ b has a ≤ √m ≤ √n. |
| Q2 | 9 12 15 18 21 24 27 30 | Multiples of 3 starting from 9. |
| Q3 | B. O(n log log n) | Σ n/p over primes ≈ n log log n. |
| Q4 | line 4 | Starting at m = p crosses out p itself, so every prime is removed. Start at 2·p (or p·p). |
| Q5 | B. 15 | 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47. |

## 5.8 Segmented & Incremental Sieve

| | Answer | Why |
|---|---|---|
| Q1 | C. √R | Every composite ≤ R has a prime factor ≤ √R. |
| Q2 | 105 | (100 + 6) / 7 = 15; 15 · 7 = 105. |
| Q3 | B. To avoid marking p itself and values below p·p | Smaller multiples were handled by smaller primes; p itself must stay unmarked. |
| Q4 | B. Produces primes one after another, remembering upcoming composites | It stores for each upcoming composite the primes that will cross it out. |

## 5.9 Binary Palindrome

| | Answer | Why |
|---|---|---|
| Q1 | B. 9 (1001) | 1001 reads the same both ways. |
| Q2 | 3 | 6 = 110 → reversed 011 = 3. |
| Q3 | B. multiplies by 2 | A left shift moves every bit one place to the left: multiplication by 2. |
| Q4 | & | `rest & 1` isolates the lowest bit. |
| Q5 | B. O(log n) | One iteration per bit; the number of bits is $\log_2 n$. |

## 5.10 Swap Two Nibbles in a Byte

| | Answer | Why |
|---|---|---|
| Q1 | B. 4 | A nibble is half a byte: 4 bits. |
| Q2 | 195 | 0x3C → 0xC3 = 195. |
| Q3 | B. 0xF0 | 0xF0 = 1111 0000. |
| Q4 | B. To remove sign extension and get 0–255 | Bytes are signed; & 0xFF keeps the low 8 bits as an unsigned value. |

## 5.11 Longest Sequence of 1s (and after flipping a bit)

| | Answer | Why |
|---|---|---|
| Q1 | 3 | 01101110: runs of 1s are 3 (bits 1–3) and 2 (bits 5–6). |
| Q2 | C. previous + current + 1 | Left run + flipped bit + right run. |
| Q3 | B. 32 | All 32 bits are already 1; a 32-bit int cannot hold more. |
| Q4 | B. O(log n) (32 steps) | One step per bit; the number of bits is logarithmic in the value. |

## 5.12 Josephus Problem

| | Answer | Why |
|---|---|---|
| Q1 | C. 5 | Order: 2, 4, 6, 3, 1 → survivor 5. |
| Q2 | 3 | ans: 0 → (0+2)%2=0 → (0+2)%3=2 → (2+2)%4=0 → (0+2)%5=2; +1 = 3. |
| Q3 | C. O(n) | One loop iteration per person. |
| Q4 | + | Shift by k places and wrap around the circle of `people`. |
| Q5 | A. 9 | 2·4 + 1 = 9. |

## 5.13 Block Swap Algorithm (Array Rotation)

| | Answer | Why |
|---|---|---|
| Q1 | 3 4 5 1 2 | Element i of the output is element (i + 2) mod 5 of the input. |
| Q2 | C. Three reversals | Reverse(A), reverse(B), reverse(all) uses only swaps. |
| Q3 | B. 5 | n − d = 5. |
| Q4 | C. O(n·d) | d passes of n element moves. |
| Q5 | 3 | Final array: 3 4 5 6 1 2. |

## 5.14 Leaders in an Array

| | Answer | Why |
|---|---|---|
| Q1 | A. 17 5 2 | 17 > all to its right; 5 > 2; 2 is last. 16 is smaller than 17. |
| Q2 | line 1 | Initialising max to 0 hides negative leaders; use Integer.MIN_VALUE. |
| Q3 | B. O(n) | One pass. |
| Q4 | A. Yes | Nothing to its right can be larger. |

## 5.15 Majority Element (Boyer–Moore Voting)

| | Answer | Why |
|---|---|---|
| Q1 | B. at least 5 | More than 9/2 = 4.5 → at least 5. |
| Q2 | count after 2nd element (1) = 0; candidate after 3rd element (2) = 2; count after 4th element (4) = 0; final candidate = 7 | (2,1) → (2,0) → (2,1) → (2,0) → (7,1). 7 is only a candidate; verification fails. |
| Q3 | B. The candidate may not be a majority | If no majority exists, phase 1 still returns some value. |
| Q4 | C. Boyer–Moore | Voting needs only a candidate and a counter. |

| | Answer | Why |
|---|---|---|
| Q5 | B. O(n log n) | The sort dominates. |

### 5.16 Maximum Product Subarray

| | Answer | Why |
|---|---|---|
| Q1 | B. 15 | n(n+1)/2 = 15. |
| Q2 | 24 | (−2)·3·(−4) = 24. |
| Q3 | B. A negative number can turn it into the maximum | Most negative × negative = largest positive. |
| Q4 | A. O(n) | One pass with O(1) work per element. |
| Q5 | 2 | The subarray {2} has product 2. |

### 5.17 Maximum Equilibrium Sum

| | Answer | Why |
|---|---|---|
| Q1 | B. 2 | Left 1+3 = 4, right 2+2 = 4 at index 2. |
| Q2 | 3 | 8 − 4 − 1 = 3 (the elements 2 and 1). |
| Q3 | B. total − left − a[i] | Everything not on the left and not a[i] is on the right. |
| Q4 | B. O(n) | Two linear passes. |

### 5.18 Maximum Sum of an Hourglass in a Matrix

| | Answer | Why |
|---|---|---|
| Q1 | A. 12 | (5−2)(6−2) = 12. |
| Q2 | B. two full rows of 3 and the centre cell | Top row, centre of the middle row, bottom row. |
| Q3 | 35 | 1+2+3+5+7+8+9 = 35 (the slide's own example). |
| Q4 | B. All sums may be negative | Otherwise the result 0 would not correspond to any hourglass. |

### 5.19 Toggle the Switch (100 Doors)

| | Answer | Why |
|---|---|---|
| Q1 | B. 10 | The perfect squares 1, 4, 9, …, 100. |
| Q2 | B. 9 | Divisors of 36: 1, 2, 3, 4, 6, 9, 12, 18, 36 — nine (odd), so it is open. |
| Q3 | B. Divisors pair up as (d, n/d) except when d = n/d | The unpaired divisor √n makes the count odd. |
| Q4 | B. O(n log n) | Σ n/k = n H(n). |

### 5.20 Alice's Apple Tree (Minimum Garden Perimeter)

| | Answer | Why |
|---|---|---|
| Q1 | B. 48 | $12 \cdot 2^2 = 48$ (and 12 + 48 = 60 inside the r = 2 square). |
| Q2 | 60 | 2·2·3·5 = 60. |
| Q3 | B. apples(r) is monotonically increasing | More rings always mean more apples. |

| | Answer | Why |
|---|---|---|
| Q4 | B. 8r | Side length 2r, four sides: 8r. |

## 5.21 Weighted Substring (Smallest String of a Given Weight)

| | Answer | Why |
|---|---|---|
| Q1 | C. 60 | D = 4·C + C = 5·12 = 60. |
| Q2 | B. ACC | 12 + 12 + 1 = 25 with just 3 letters. |
| Q3 | ABB | Greedy takes B (3), B (3), then A (1): 3 + 3 + 1 = 7. Sorted: ABB. |
| Q4 | B. Each weight divides the next | Divisibility makes the coin system canonical. |

## 5.22 Move Hyphen to the Beginning

| | Answer | Why |
|---|---|---|
| Q1 | -abcd | Before: "ab", after: "cd". |
| Q2 | B. -1 | indexOf returns −1 when there is no match. |
| Q3 | B. Strings are immutable so every += copies the string | Each step creates a new String of increasing length. |
| Q4 | B. O(n) | Every character is read once and written once. |

## 5.23 Strobogrammatic Number

| | Answer | Why |
|---|---|---|
| Q1 | C. 69 | 69 rotated by 180° is 69. 25 and 1962 contain 2 (no valid rotation); 962 contains 2 as well. |
| Q2 | A. 0, 1, 8 | The middle digit must rotate into itself. |
| Q3 | true | 8 rotates to 8 and 88 reads the same. |
| Q4 | B. 4 | 11, 69, 88, 96. |

## 5.24 Lexicographically First Palindromic Rearrangement

| | Answer | Why |
|---|---|---|
| Q1 | B. level | level: l:2, e:2, v:1 — one odd count. |
| Q2 | abcba | front "ab", middle "c", back "ba". |
| Q3 | C. O(n) | One pass to count plus a pass over the alphabet. |
| Q4 | A. 0 | The counts add up to an even number, so the number of odd counts is even; for a palindrome it must be ≤ 1, hence 0. |

## 5.25 Manacher's Algorithm (Longest Palindromic Substring)

| | Answer | Why |
|---|---|---|
| Q1 | B. To make every palindrome odd-length | Even palindromes get a "#" centre. |
| Q2 | C. O(n) | The right edge only moves forward. |
| Q3 | #a#b#b#a# | n characters → 2n+1 characters. |
| Q4 | B. aaaaaa | Every centre expands to the borders. |

## 5.26 Natural Sort Order

| | Answer | Why |
|---|---|---|
| Q1 | B. file10 | Compare "1" with "2": 1 < 2, so file10 comes first. |
| Q2 | B. equal numerically (then tie-break) | Numerically they are both 7; add a tie-breaker for a deterministic order. |
| Q3 | C. a negative number | Negative, zero, positive: less, equal, greater. |
| Q4 | A. stable | It uses TimSort, a stable sort. |

## 5.27 Maneuvering (Counting Grid Paths)

| | Answer | Why |
|---|---|---|
| Q1 | B. 20 | C(6, 3) = 20. |
| Q2 | 6 | Rows: 1 1 1 / 1 2 3 / 1 3 6. |
| Q3 | B. It recomputes the same sub-grids many times | Overlapping sub-problems → exponential calls. |
| Q4 | B. O(m·n) | Each cell is filled once. |

## 5.28 Sorted Distinct Permutations

| | Answer | Why |
|---|---|---|
| Q1 | B. 6 | 4!/(2!·2!) = 6. |
| Q2 | BAC | Pivot is A (A < C); smallest larger letter to its right is B; swap → BCA, reverse the tail "CA" → "AC" → BAC. |
| Q3 | B. It prints every distinct permutation once, but not in sorted order | The HashSet prevents duplicates; swapping disturbs the order. |
| Q4 | C. O(8·8!) | 8! permutations × O(8) to build/print each. |

## 5.29 N-Queens (Backtracking)

| | Answer | Why |
|---|---|---|
| Q1 | B. 2 | Two mirror-image solutions. |
| Q2 | C. same diagonal | \|Δrow\| = \|Δcol\| = 2. |
| Q3 | 0 | Remove the queen again (un-choose). |
| Q4 | B. Undoing the last choice and trying the next option | Return to the previous decision and change it. |
| Q5 | C. Backtracking | Systematic search with pruning. |

## 5.30 Maze Solving (Rat in a Maze)

| | Answer | Why |
|---|---|---|
| Q1 | B. To avoid walking in circles forever | Without it the search can bounce between two cells indefinitely. |
| Q2 | B. BFS | BFS explores in rings of increasing distance. |
| Q3 | B. Un-mark a dead-end cell (backtrack) | The cell is not on any path to the exit. |
| Q4 | B. O(R·C) | Each cell is expanded once. |

## 5.31 Hamiltonian Cycle

| | Answer | Why |
|---|---|---|
| Q1 | B. every vertex exactly once and returns to the start | Edges once = Eulerian; vertices once = Hamiltonian. |
| Q2 | B. A cycle can be rotated to start anywhere | Any cycle through all vertices contains vertex 0. |
| Q3 | C. O(V!) | Up to (V−1)! orderings. |
| Q4 | B. impossible | A cycle needs two edges at every vertex. |

## 5.32 Knight's Tour & Warnsdorff's Rule

| | Answer | Why |
|---|---|---|
| Q1 | C. 8 | Eight L-shaped jumps. |
| Q2 | B. the fewest onward moves | Visit the most constrained square first. |
| Q3 | B. No, it is a heuristic | It can get stuck; check the result. |
| Q4 | B. $O(N^2)$ | $N^2$ moves with constant work each. |

## 5.33 Selection Sort

| | Answer | Why |
|---|---|---|
| Q1 | 10 13 14 37 29 | Pass 1: swap 10 with 29 → 10 29 14 37 13. Pass 2: min of rest is 13 → swap with 29 → 10 13 14 37 29. |
| Q2 | C. 15 | 5 + 4 + 3 + 2 + 1 = 15. |
| Q3 | B. At most n−1 swaps | One swap per pass at most. |
| Q4 | C. $O(n^2)$ | It always scans the entire unsorted part. |

## 5.34 Quick Sort

| | Answer | Why |
|---|---|---|
| Q1 | B. in its final sorted position | Everything left is ≤ and everything right is >. |
| Q2 | C. $O(n^2)$ | Highly unbalanced partitions. |
| Q3 | 5 | Five elements (3, 2, 5, 1, 4) are ≤ 6 so the pivot goes to index 5. |
| Q4 | B. Divide and conquer | Partition, then solve the two halves independently. |
| Q5 | B. No | Partitioning swaps distant elements, so equal keys may be reordered. |